微网与外部电网电力调控策略

摘　要

微网在分时电价与光伏出力双重不确定性下运行：光伏盈余集中在电价低谷的午间，负载高峰落在电价尖峰的傍晚，而储能容量与充放功率均有限。本文按信息结构由弱到强给出四层输出：确定性最优计划 → 不确定性下的安全边际 → 多阶段滚动调整 → 波动电价下的风险决策。

问题 1 中电价、负载与光伏预测给定且要求 0:00 与 24:00 储电量相同，目标与约束均为线性，故建立**线性规划**模型，由其**对偶变量**给出储电水价值；在储电量界不活跃时由互补松弛条件得到最优充放电的**阈值型刻画**（死区比 $1/(\eta_c\eta_d) = 1.2346$）。问题 2 的计划依据为 0:00 的光伏预报、结算依据为当日实际值，缺口按 5 倍电价紧急购电，两阶段在时间上分离，故建立**两阶段随机规划**模型，并用**报童型临界分位数**给出安全边际的解析近似。问题 3 有 0:00、6:00、12:00、18:00 四个预报时刻，故建立含**非预期性**的**多阶段滚动优化**模型，并以预报频次对照实验量化**信息价值**。问题 4 中电价本身随机，故以**分时骨架加随机残差**两层结构建模电价，用二维分层抽样的**联合情景**保留电价—净负荷相关结构，在**条件风险价值**下重算问题 2、3。

问题 1 最优全天购电费 **35 126.95 元**，相对无储能基线 48 052.05 元节省 **12 925.10 元（26.90%）**，全天购电量 59 482.70 kWh，充、放电量 20 740.67 与 16 799.94 kWh（比值 1.2346 与往返效率互为倒数），弃光为 0。问题 2 在 2025-02-01 至 12-31（334 天）总费用 **14 264 792.13 元**（计划购电费 13 637 552.90 元、紧急购电费 627 239.24 元），紧急购电量 172 015.82 kWh，出现在 296 天，相对基线 16 407 319.63 元节省 13.06%。问题 3 引入四个预报时刻后总费用降至 **13 932 402.95 元**，信息价值 332 389.18 元（2.33%），已实现完全信息上界价值的 85.1%，继续加密预报最多再省 58 063 元（0.42%），故**不建议**再增加预报时刻。问题 4 在波动电价下问题 2、3 的费用分别为 **15 094 255.84 元**与 **14 722 153.17 元**，相对波动价基线 17 026 782.86 元节省 11.35% 与 13.54%，风险—费用前沿给出不同风险偏好下的策略族。

全部结果通过**原始可行性**（供给缺口残差 $\le 10^{-6}$ kWh）、**对偶最优性**与**费用独立复算**三重检验（审计项 A1–A10 在 10 min 级内部解上全部通过：供给缺口残差 3.41×10^{-13} kWh、越界与充放互斥违反数均为 0、费用复算相对误差 0；交付物层受 4 h 聚合与四位小数取整限制，块级残差 $\le 2.96 \times 10^{-4}$ kWh），并与无储能、完美信息、典型日预报、全年单一线性规划四类基线对照；对紧急购电倍数、效率口径、容量与功率、场景数与终端条件做灵敏度分析，结论在上述范围内不变。

在本题参数范围内，储能的核心价值是**把午间光伏盈余与低谷电量搬运到傍晚高价时段**；**容量摆幅是主要瓶颈**（解除容量上限可再省 22.1%），充放电功率在部分时段同时构成约束（解除后仅再省 1.7%），两者量级不同；当峰谷价比高于盈亏平衡阈值 $1/(\eta_c\eta_d) \approx 1.235$ 时套利有利。结论适用于价格接受型、不外送、单一储能的并网微网，允许上网售电或引入需求响应时需重新校准。

关键词： 微网；储能调度；线性规划；两阶段随机规划；报童模型；多阶段滚动优化；CVaR；信息价值

一、问题重述与分析

1.1 问题背景与决策矛盾

本文研究的并网型微网既与外部电网（以下简称外网）交换功率，又配有储能设备（BESS），其经济性来自**电价的时间价差**：低价或分布式能源有剩余电量的时段充电、高价时段放电。

在新能源全面参与市场交易的背景下 [1]，储能已由“并网配套”转变为**必须依靠市场价差生存的资产**（发改价格〔2025〕136 号）；题面既未给出上网电价，也未给出容量补偿与辅助服务收益，其收入只能来自把电量从低价时段搬到高价时段。

“价差是否足以支撑这笔搬运”可直接用附件数据检验：按附件 1 的固定电价，全天最高电价 1.3952 元/kWh 与最低电价 0.3713 元/kWh 之比为 $1.3952/0.3713 \approx 3.76$（口径：全时段极值之比，不依赖峰谷时段划分）。储能搬 1 kWh 一个来回须克服往返损耗（在 p_1 处充电、p_2 处放电时单位电量净收益为 $p_2\eta_c\eta_d - p_1$），故 **盈亏平衡价比阈值为** $1/(\eta_c\eta_d) = 1/0.81 \approx 1.2346$（效率读法见假设 A2）；3.76 远高于 1.2346，说明本题中**价差本身不是瓶颈**——真正的约束是储能的可用容量摆幅、充放电功率，以及无法事先确知的电价与光伏波动。

本题的决策矛盾可以概括为一句话：**光伏盈余集中在电价低谷的午间，而负载高峰落在电价尖峰的傍晚，储能容量与充放功率均有限，因此必须把“何时购电、何时充放、预留多少安全余量”作为一个整体来优化。**

- **问题 1（确定性计划）**：电价与负载每天相同、光伏给出一天的预测数据，要求 0:00 与 24:00 储电量相同，制定每天 0:00 的计划购电策略。
- **问题 2（不确定性下的日前计划）**：电价仍固定，负载与光伏实际值逐日逐时段变化，购电不足须按交易时刻电价的 5 倍**紧急购电**；计划在 0:00 制定、偏差在实时结算，属**两阶段决策**。
- **问题 3（多阶段滚动调整）**：0:00、6:00、12:00、18:00 可获取未来 24 小时整点光伏预报，其他时刻可**调整**购电策略；计划高于调整的部分按 0.5 倍计价、调整高于计划的部分按 1.5 倍计价，核心追问是 **是否需要引入其他时刻的预报**。
- **问题 4（波动电价下的风险决策）**：外网电价本身实时波动，需重新求解问题 2 与问题 3，问题由“确定性价格 + 随机出力”升级为**价格与出力的联合不确定性**。

1.2 四问的差别在于信息结构，而不在于模型复杂度

决定建模方式的只有三件事：**决策时刻有几个、每个时刻知道什么、偏差如何被惩罚**（图 1）。



图 1　微网决策时序与信息结构（横轴为一天 24 小时；上行泳道为信息到达时刻，中行为决策时刻，下行为结算依据；$\Delta t = 1/6$ h，全天共 144 个 10 min 时段；样本范围为 2025-02-01 至 12-31 共 334 天）

同一递进（图 1）：决策时刻由 1 个增至 4 个，调整的计价基准始终为 0:00 的计划值（假设 A7）；已知量由当日真值变为预报值、再变为只剩电价的 **分时骨架**（假设 A10）；模型由确定性线性规划变为两阶段随机规划、再变为带 **非预期性约束**的多阶段滚动。

四问的差别不在于模型复杂度，而在于信息结构：决策时刻从 1 个增加到 4 个，计划依据从确定值变为预报值，且引入了违约（0.5 倍）与紧急购电（5 倍）两类不对称惩罚。问题 3 另有第三条价格信号：超出计划量的部分按 1.5 倍计价、被减部分按 0.5 倍计价，即追加的边际代价是削减的边际收益的 $1.5/0.5 = 3$ 倍，故**频繁、大幅地改计划本身就要付钱**。问题 2 与问题 4-2 相反：计划量按正常电价全额结算（口径 口径 C-03），买少了要按 5 倍电价补缺口，故最优计划会**主动多买、留出安全边际**——消融实验显示，取消安全边际后 334 天总费用升至 17 398 559.68 元（比无储能基线还高 6.04%）、紧急购电量升至 1 156 618.02 kWh，保留边际的方案为 14 264 792.13 元与 172 015.82 kWh，安全边际的边际价值为 $17\,398\,559.68 - 14\,264\,792.13 = 3\,133\,767.55$ 元（第九章、（见 C-Q2-01））。两侧惩罚方向相反，不能用同

一套边际规则处理。

1.3 四问的依赖关系与误差传播

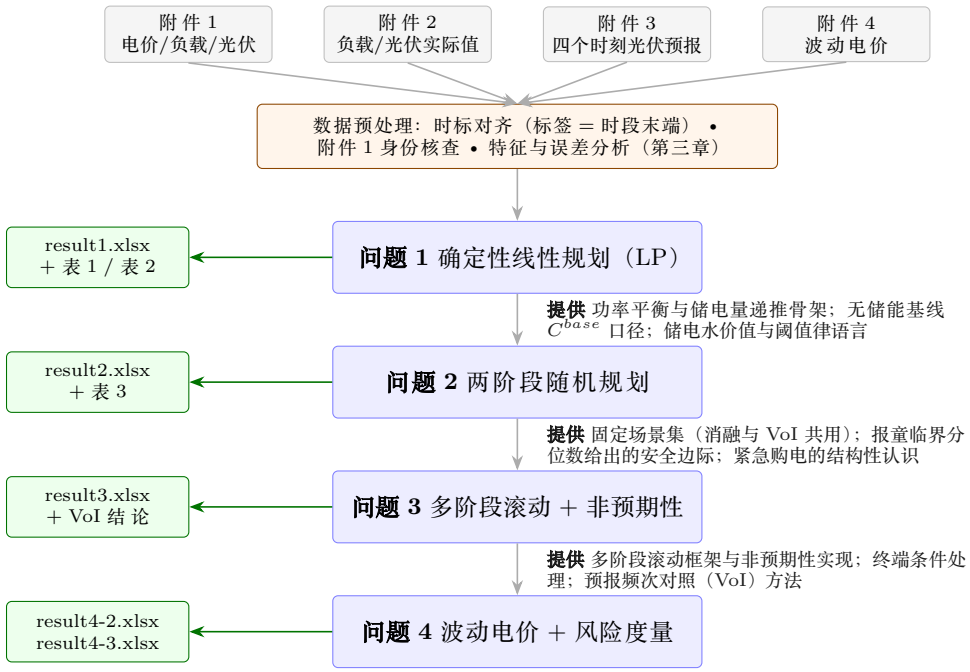


图 2 四问的模型依赖关系与复用内容（箭头标注上游向下游提供的模型要素；附件 1–4 经数据预处理后进入问题 1–4，四个问题在 2025-02-01 至 12-31 共 334 天的同一窗口上报告结果）

图 2 表明四问共享同一套物理模型骨架（并网点电量平衡、储电量递推与充放电上下界约束），逐层增加信息结构模块（问题 1 的骨架与基线 C^{base}、问题 2 的两阶段结构、问题 3 的滚动框架、问题 4 的随机电价），使每层新增贡献都可被**单独消融**（见第九章）。

表 1 四问之间的误差传播路径与本文的处理方式（“本文的处理”列均指向正文对应小节）

上游 → 下游	传递的内容	潜在的误差传播	本文的处理
附件 → 问题 1	电价、负载、光伏的时标与单位	时标错位一个槽或丢掉 0:00+1 一列，会让全部合计系统性偏离，而“全天汇总 = $\sum$ 144 格”的自检对整体错位不可见	统一按“标签 = 时段末端”的**位置读法**对齐，并断言每天恰有 144 个时段；末列 0:00+1 保留（§3.2）
问题 1 → 问题 2	模型骨架、C^{base} 口径、水价值语言	问题 1 用“附件 1 全年平均日”，问题 2 用“334 天窗口”，两套口径量级不同（单日 48 052.0466 元对 334 天 16 407 319.63 元）；混排一次，全文所有“节省率”即失去可比性	两套口径各自命名并在表 2 中显式声明；报告数值时同时注明窗口天数与电价来源（§1.3、§4.4）
问题 2 → 问题 3	场景生成与削减方法、安全边际	场景分布偏离真实分布时，各问各自生成场景会把“场景噪声”误读成“信息结构改进”	场景集**一次性固定并存盘**，问题 3 与消融实验读同一份（§9.4）；稳健性上把场景数由 $K = 60$ 增至 $K = 120$，紧急购电量由 172 015.82 kWh 变为 141 707.94 kWh，而总费用仅由 1 426.48 万元变为 1 423.52 万元（§5.2）
问题 3 → 问题 4	多阶段滚动框架、终端条件	电价随机性与光伏预报误差叠加；若两者分别抽样，会丢失价格与出力的相关结构，并低估“高价又缺电”的联合尾部	用“日净需求残差 × 日均电价”二维分层取历史代表日生成**联合**情景，保留 $\rho(p, \text{净负荷})$（实际 0.2620、情景 0.2597）；分块 Bootstrap 仅作条件对照（§7.2）
问题 1–4 → 全文结论	各问的“总费用”与“节省率”	基线定义不一致时，同一个“节省率”在不同章节指向不同分母，形成内部矛盾	四问统一使用 口径 C-01 定义的无储能基线，节省率按 口径 C-04 计算，并逐处注明电价口径（附件 1 固定价 / 附件 4 波动价）

表 1 第二行与末行指向同一风险：问题 1 的输入是**全年平均日**、问题 2–4 的窗口只有 334 天，两者是**两套口径**而非同一量的两个估计，必须各自命名、分别报告（表 2）。

表 2 第一行是**单日**量、后两行是 **334 天累计**量（2025-02-01 至 12-31），三者不可相减相除，只在同一行内部比较；节省率一律写成“相对同一窗口、同一电价口径下的无储能基线”。

表 2　无储能基线的两套口径声明（费用单位：元；口径 C-01 定义见第四章）

口径名称	适用问题	负载与光伏来源	电价来源	基线数值
典型日口径	问题 1	附件 1 的 144 槽典型日（全年逐时段均值）	附件 1（固定价）	48 052.0466
窗口口径（固定价）	问题 2、3	附件 2 的当日实际负载与实际光伏，逐日计	附件 1（固定价）	16 407 319.63
窗口口径（波动价）	问题 4	同上一行	附件 4（波动价）	17 026 782.86

本文与既有做法的差异　常见的两种处理（各问单独选口径与参考值；直接用遗传算法、粒子群等元启发式求解）要么使“节省率”分母不可核对，要么拿不到对偶信息。本文的差异在四点：①四问压在**同一个物理模型骨架与同一个基线口径**上、逐层增加模块，使每层贡献可被单独消融，而非并列四套模型；②全部模型为**线性规划**并用 HiGHS 精确求解，可输出对偶量（储电边际价值 ν_t，第四章记法 μ_t）与阈值型策略的最优性证据；③计划依据与结算依据分离为**两套数据**（假设 A9）：三种信息结构在同一 334 天窗口与场景集上的费用为 R1 1 341.16、R2 2 348.46、R3 1 426.48 万元，R3 与 R1 之差 $14\,264\,792.13 - 13\,411\,560.90 = 853\,231.23$ 元即**预测误差的代价**，R2 比无储能基线高 43.14%（口径使用错误）；④直接回答题面“是否需要引入其他时刻的预报”：同一场景集与同一实际轨迹下，预报时刻数由 1 增至 2、再增至 4 时，总费用为 1 426.48 万元 → 1 413.16 万元 → 1 393.24 万元，四次预报已实现 **85.1% 的可获得信息价值**，继续加密最多再省 58 063 元（0.42%），故在本题的预报精度与惩罚结构下**不建议继续加密**，除非新增预报的日成本低于该额度（(见 C-Q3-01)）。

二、模型假设与符号说明

2.1　模型假设

题面已给定储能最大容量 12000 kWh、最大充放电功率 5000 kW、2025 年 1 月 1 日 0:00 储电量 6000 kWh、储电量运行区间 1200–10800 kWh 与充放电效率 90%；本文取运行区间为额定容量的 10%–90%（可用摆幅 $10800 - 1200 = 9600$ kWh），二者与功率上限构成两类硬约束，谁更紧由第四章容量/功率双向扫描判定。

假设 A2 影响最大：三种效率读法下，典型日相对无储能基线的节省额为 12 925.10 元（单边各 0.9，**主口径**）、14 635.52 元（仅充电侧 0.9）、13 883.44 元（仅放电侧 0.9），极差 13.2%；把题面“90%”读作往返效率、等效为单边 $\sqrt{0.9}$ 时为 14 250.99 元。主口径写入全部结果表，其余读法只在灵敏度节出现（(见 C-Q1-03)）。

2.2　信息结构声明

问题 2–4 中“每天 0:00 制定计划”有三种解读，本文各跑一次全窗口实验，条件完全一致（同一 334 天窗口、同一场景集、同一终端条件、同一电价口径），只有“0:00 已知什么”不同，三行可直接相减（表 4）。

本文采用**信息结构 R3**：R1 假设 0:00 即知当天真值，紧急购电量恒为 0、问题 2 退化为问题 1；R2 用事后平均值冒充事前信息，总费用比无储能基线还高 43.14%，属口径错误；R3 用附件 3 中**当天** 0:00 **发布**的真实预报，全部信息先于决策时刻可得，无泄漏也不夸大。三点补充：①计划依据（附件 3 预报 + 负载曲线）与结算依据（附件 2 当日实际负载与光伏）是**两套数据**，差额即紧急购电来源；②“预测误差的代价”为 $14\,264\,792.13 - 13\,411\,560.90 = 853\,231.23$ 元、占 R1 费用的 6.36%（(见 C-Q2-01)）；③负载按“已知”处理是对 R3 的乐观简化（题面未给负载预报），会低估紧急购电量，第九章另作预报误差灵敏度。

2.3　符号说明

公式符号均在表 5 登记，单位口径见本节末段。

时标与单位口径声明　本节符号遵循同一套时标与单位口径（四问一致）：① **位置读法**——附件 1/2/4 的标签为 **时段末端**，槽位 j（$j = 0, 1, \ldots, 143$）表示区间 $[10j,\ 10(j+1))$ 分钟，故题面表 1 的“10:00–10:10”对应槽位 60；末列 0:00+1 属当日第 144 个槽位 [23:50, 24:00)、不得丢弃（逐格对照见 §3.2 表 6）。② **电量口径**——决策变量为电量（kWh），$\Delta t = 1/6$ h，功率上限 $\overline{P} = 5000$ kW 对应单槽电量上限 $E^{\max} = \overline{P}\Delta t = 833.3333$ kWh；目标函数的电价系数为 p_t（元/kWh）、**不再乘** Δt；等价的功率口径为“变量取功率、目标系数取 $p_t\Delta t$、上限取 5000 kW”，两套口径全文只用一套。③ **单位与精度**——功率 kW、电量 kWh、电价元/kWh、费用元；表格费用保留 2 位小数，内部计算不取整。

三、数据预处理与特征分析

3.1　数据来源与基本规模

四个附件的用途见表 40。时间序列均按 **10 min** 粒度、以**时段末端**为标签对齐，全天 144 个时段；负载与光伏为**功率**（kW）、电价为**元/kWh**，不得混用。

表 3　模型假设及其必要性、现实/题目依据、影响方向与验证方式（A1–A10）

编号	假设	必要性（不用会怎样）	现实/题目依据	影响方向	验证或灵敏度方式
A1	不允许向外部电网售电	目标函数将多出一项售电收益，盈余有了出路，弃光量恒为 0，模型的可行域与最优值都改变	题面未给出上网电价；就近消纳“优先自用”的政策导向	使费用**偏高**，结果保守	对结果文件作非负性审计（审计项 A7）：五个结果文件的负购电量数均为 0；并对引入上网电价 θp_t 的政策情景作扫描
A2	充电效率与放电效率**单边各取** 0.9（往返 0.81）	效率决定套利是否有利可图，是收益的核心参数；不声明读法则同一模型可给出三套结果	题面给出“充放电效率为 90%”，**未区分充、放方向**	效率越低，往返损耗越大，储能套利收益越低	三种主口径读法的节省额对照（单边各 0.9、仅充电侧 0.9、仅放电侧 0.9）见本节末段；另报“往返 0.9 等效单边”口径作条件对照
A3	同一时段不同时充放电	若允许同时充放，须引入 0–1 变量，模型由线性规划变为混合整数规划，规模与求解时间上升，且对偶信息不再直接可用	效率乘积 $\eta_c\eta_d < 1$ 时同时充放严格劣于只做净动作（命题 1）	不影响最优目标值，只影响求解方式	数值断言 $\min(c_t, d_t) = 0$ 全部成立：问题 1 互斥违反数为 0，问题 2/3/4 的内部解互斥违反数也均为 0（审计项 A5）
A4	忽略自放电与循环衰减	否则目标函数须加入与循环次数（放电量）相关的衰减成本项，并需要衰减参数	题面未给出任何衰减参数；结果窗口仅 334 天	高估长期收益，且放电量越大高估越多	加入循环成本 c_{cyc}（元/kWh 放电量）作单参数灵敏度，给出“储能节省额归零”的临界 c_{cyc}
A5	储电量**跨日连续递推**，不逐日清零	若每日清零，则与“2025-01-01 0:00 储电量 6000 kWh”的初值条件矛盾，并丢弃跨日套利空间	附录 1 给出的初值 + 储能设备连续运行的常识；题面只要求 0:00 与 24:00 储电量相同	影响全窗口费用与每天的储电量初值，跨日套利越大影响越大	与“每日重置初值”变体对照（消融层级 B2/B3）；审计项 A6 逐日核对“末值不低于初值”的违反天数（问题 2/3/4 均为 0）
A6	每日终端条件 $S_{144} \geq S_0$；问题 1 取严格日周期 $S_0 = S_{144} = 6000$ kWh	不设终端条件时，模型会在每天末尾与窗口末尾“掏空”储能以压低当期费用，形成短视解，且该解在真实连续运行中不可执行	储能运营常识；问题 1 题面直接要求 $S_0 = S_{144}$	影响每天末尾与窗口末尾的费用；本数据上日周期口径比自由环状口径贵 8.35 元（0.02%）	终端处理四组对照：完全自由、$S_T \geq S_0$、日周期、水价值终端（§9.4 层级 B5）
A7	调整决策“后覆盖前”，调整费用的计价基准始终为 0:00 的**计划购电量**	若基准随上一次调整漂移，调整费与违约费的口径不可复现，也无法与题面表述对应	题面“计划购电量高于调整购电量的部分”一句明确了基准	影响调整与违约相关费用；四次预报口径下违约费 210 674.71 元、超额费 262 628.79 元	与“相对上一次调整量”口径并列报告并各自命名（第九章）
A8	2025 年 1 月作为储电量递推的**预热期**，其终值即 2025-02-01 0:00 的初值	结果窗口自 2025-02-01 起，而初值自 2025-01-01 起算；若不作预热声明，2 月 1 日的储电量没有来源，跨日递推无从开始	附件 2 覆盖 2025-01-01 至 12-31，结果文件要求 2025-02-01 起共 334 天	影响窗口初期的储电量与初期费用；1 月本身不计入任何结果	论文与结果文件中明示 1 月不计入结果，并公开 2 月 1 日初值的来源；审计项 A6 逐日核对终端条件
A9	**信息结构取 R3**：问题 2/4-2 的 0:00 计划依据为附件 3 中当日 0:00 发布的光伏预报，负载曲线按“已知”处理；问题 3/4-3 另有 6:00/12:00/18:00 三次滚动的当次预报	题面只说“每天 0:00 制定计划”，未说明 0:00 知道什么；不声明则问题 2–4 的任何数值都无法解释（三种解读的窗口总费用相差近千万元）	题面“根据预测数据制定计划”；附件 3 提供 0:00 发布的 24 h 光伏预报；题面未给负载预报，故按已知处理	费用落在完美信息（下界）与典型日预报（上界）之间；负载按已知处理会 **低估**紧急购电量，使结果偏乐观	三种信息结构 R1/R2/R3 在同一 334 天窗口与同一场景集上的完整对照（表 4）；**非预期性置换检验**：置换 6:00/12:00/18:00 发布的预报后，其发布时刻之前各槽位的计划量逐位不变（最大差值 0.0），置换 0:00 预报则只改变 0:00 之后的决策（第九章）；并对预报误差水平另作灵敏度
A10	波动电价（问题 4）下 0:00 可用的**电价预报取“分时骨架”**，即电价的逐时段日内均值曲线；结算仍按附件 4 的实际电价	附件 4 未给出日前电价预报；若不声明电价预报口径，问题 4 的“计划”将使用结算时的实际电价，构成数据泄漏，结果会被系统性压低	分时骨架与附件 4 的逐时段均值最大偏差仅 5.15×10^{-5} 元/kWh，且等于附件 1 的电价剖面，是可获得的日前信息	忽略电价的残差波动，使问题 4 的费用与节省率**偏乐观**	结算侧改用附件 4 实际电价，价格随机性只在结算体现；并以“是否保留残差自相关”作对照（分块 Bootstrap 条件对照的分位数最大相对偏差达 20.44%）

表 4　信息结构的三种解读及其在 2025-02-01 至 12-31（334 天）上的实测后果（费用单位：元，电量单位：kWh，电价均为附件 1 固定价；紧急购电量口径见 口径 C-05，“出现在多少天”一并列出）

代号	解读	总费用	相对基线节省率	紧急购电量	本文的使用方式
R1	完美信息：0:00 即知当日负载与光伏真值	13 411 560.90	18.26%	0.00（0/334 天）	**不可作为主方案**；用作“预测误差的代价”的**下界对照**
R2	典型日预报：以附件 1 的日均形状作为预报	23 484 646.63	−43.14%	2 525 548.76（286/334 天）	仅作悲观对照；附件 1 是实际值的全年逐时段均值而**不是预报**，用它当预报须额外论证并做泄漏检验，本文**不采用**
R3	日前预报：0:00 使用附件 3 当日 0:00 发布的光伏预报，负载曲线按已知处理	14 264 792.13	13.06%	172 015.82（296/334 天）	**本文主方案**：最符合工程实际与题面“制定计划”的语义

表 5　符号说明（集合、参数、决策变量、对偶变量与指标量；“—”表示无量纲）

类别	符号	含义	单位	来源/取值
集合	$t \in \mathcal{T}$	时段（10 min 粒度）	—	$\lvert\mathcal{T}\rvert = 144$
集合	$d \in \mathcal{D}$	日期	—	334 天（2025-02-01 至 12-31）
集合	$m \in \mathcal{M}$	预报发布/决策时刻	—	$\{0, 6, 12, 18\}$ 时
集合	$\omega \in \Omega$	情景	—	$\lvert\Omega\rvert = K$
参数	$L_{d,t}$	小区负载（功率）	kW	附件 2
参数	$PV_{d,t}$	光伏实际功率	kW	附件 2
参数	$\widehat{PV}_{d,h}^{(m)}$	m 时发布的光伏预报功率（整点值）	kW	附件 3
参数	$p_{d,t}$	电价	元/kWh	附件 1（固定价）/ 附件 4（波动价）
参数	$\bar{p}_t$	电价的日内分时骨架（逐时段均值）	元/kWh	附件 1 电价剖面（假设 A10）
参数	$\varepsilon_{d,t}$	电价相对骨架的残差	元/kWh	$p_{d,t} - \bar{p}_t$
参数	Δt	时间步长	h	1/6
参数	η_c, η_d	充电效率、放电效率（单边）	—	0.9/0.9（假设 A2）
参数	$\underline{S}, \overline{S}$	储电量运行下限、上限	kWh	1200/10800
参数	$\overline{S}^{\max}$	储能设备最大容量	kWh	12000
参数	$\overline{P}$	最大充放电功率	kW	5000
参数	$E^{\max}$	单槽充/放电量上限	kWh	$\overline{P}\Delta t = 833.3333$
参数	S^{init}	初始储电量	kWh	6000 @ 2025-01-01 0:00
参数	κ	紧急购电电价倍数	—	5
参数	κ^+, κ^-	调整超额 / 违约电价倍数	—	1.5/0.5
参数	β	CVaR 置信水平（预先锁定）	—	0.95
参数	π_ω	情景权重	—	分层样本占比，$\sum_\omega \pi_\omega = 1$
参数	$\rho(\cdot,\cdot)$	相关系数	—	用于刻画电价与净负荷的共动
决策	$E_{d,t}^{plan}$	计划购电量（第一阶段，与 ω 无关）	kWh	≥ 0
决策	$E_{d,t}^{adj}$	调整购电量（最后覆盖该槽位的决策）	kWh	≥ 0
决策	$g_{d,t}$	紧急购电量（第二阶段追索）	kWh	≥ 0
决策	c_t, d_t	充电量、放电量	kWh	$0 \leq c_t, d_t \leq E^{\max}$
决策	s_t	弃光/盈余处置电量	kWh	≥ 0
决策	S_t	时段末储电量	kWh	$\in [\underline{S}, \overline{S}]$
决策	u_t, v_t	调整量相对计划量的增、减分解	kWh	$E_t^{adj} - E_t^{plan} = u_t - v_t$，均 ≥ 0
对偶	α_t	电量平衡约束的影子价格（$E_t > 0$ 时等于 p_t）	元/kWh	LP 对偶；实测取值 $[0.3713, 1.2561]$，恒等式偏差 0.000
对偶	ν_t	储电量递推约束的乘子，即储电的边际价值（水价值）	元/kWh	LP 对偶；实测 $\nu_t \in [0.4711, 1.1305]$
对偶	μ_t	水价值的等价记法（$\mu_t \equiv \nu_t$），第四章阈值律沿用	元/kWh	LP 对偶
对偶	$\lambda_t^{\pm}$	储电量上/下界的影子价格	元/kWh	LP 对偶
对偶	ν_t^c, ν_t^d	充/放电功率界的乘子	元/kWh	LP 对偶
对偶	z_t^c, z_t^d	充/放互补松弛量	元/kWh	$z_t^c = \alpha_t - \eta_c\nu_t$；$z_t^d = -\alpha_t + \nu_t/\eta_d$（阈值律：$\alpha_t \leq \eta_c\nu_t$ 充电、$\alpha_t \geq \nu_t/\eta_d$ 放电，144 槽违反次数 0）
指标	C^{base}	无储能基线购电费	元	口径 口径 C-01
指标	C^{plan}, C^{emg}	计划购电费、紧急购电费	元	口径 口径 C-03
指标	V^{arb}	储能与弃光带来的相对节省	元	$C^{base} - C^*$，节省率按 口径 C-04

3.2　时标语义与对齐

附件 1、2、4 的时段标签为 $0{:}10, 0{:}20, \ldots, 23{:}50, 0{:}00{+}1$（后者为次日 0:00），本文采用**位置读法**：

- 标签表示**时段末端**：槽位 j（$j = 0, 1, \ldots, 143$）表示区间 $[10j,\ 10(j+1))$ 分钟；第 $j+1$ 个数据行/列对应槽位 j，标签只作显示、**不参与重映射**；
- 每行 144 个值**无缝铺满** 0:00 至 24:00；末列 0:00+1 属当日 23:50–24:00、**不得丢弃**，否则合计偏低 $1/144 \approx 0.6944\%$。

附件 3 的列标签为“预报 1 小时”至“预报 24 小时”，约定**预报 k 小时对应发布时刻之后的第 k 个小时**，即 $[h_0 + k - 1,\ h_0 + k)$（$h_0 \in \{0, 6, 12, 18\}$）；若误按 $[h_0 + k,\ h_0 + k + 1)$ 对齐，则**全部预报整体错**

1 小时。18:00 的预报中 $k = 7, \ldots, 24$ 落在**次日** 0:00–17:00（18 个整点），跨日用绝对时间轴映射（目标整点序号 $d \times 24 + h_0 + k - 1$），不丢配对样本。表 6 给出题面时段与附件标签的对照。

表 6　题面表 1 的时段写法与附件时段标签对照（示例；时段序号 i 从 1 起计，槽位下标 $j = i - 1$）

题面写法	附件标签	对应区间	数据点序号 i
10:00–10:10	10:10	[10:00, 10:10)	61
12:00–12:10	12:10	[12:00, 12:10)	73
14:00–14:10	14:10	[14:00, 14:10)	85
16:00–16:10	16:10	[16:00, 16:10)	97
20:00–20:10	20:10	[20:00, 20:10)	121
23:50–24:00	0:00+1	[23:50, 24:00)	144

对齐结果由两条独立校验确认：结构断言（附件 1 恰 144 行；附件 3 前向填充后每天恰 4 行）与结果文件层面的汇总校验——审计项 A10 核对"全天购电量/全天购电费"与逐时段求和，实测最大相对偏差为 3.40×10^{-7}（问题 2、3）与 4.32×10^{-7}（问题 4-2），远低于 10^{-4} 的阈值。该汇总校验对"整体错位一个槽"不可见，该风险由位置读法与表 6 逐格对照锁定。

3.3　附件 1 的身份核查

附件 1 第 4 列的表头写作"光伏发电**预测**功率"，但逐项核查表明它并非预报：

其**光伏**、**负载**、**电价**列分别等于附件 2 全年（365 天）逐时段实际光伏功率、负载与附件 4 逐时段电价的均值，最大偏差分别为 0.0431 kW、5.04×10^{-5} kW 与 5.15×10^{-5} 元/kWh。

三处偏差都在数值精度量级（10^{-5} 及以下），故附件 1 是**全年平均日（典型日）**而非日前预报（电价列的一致同时是假设 A10"分时骨架"的数据依据）：附件 1 **只能**用于问题 1 与基线口径，**不可**用作问题 2–4 的预报输入或验证集；问题 1 因此是 **确定性优化问题**，其最优值是问题 2–4 的对照基准。

换用窗口口径（2025-02-01 至 12-31 共 334 天）的逐时段均值作比较，电价偏差升到 3.07×10^{-2} 元/kWh，为**全年 365 天**口径的 $3.07 \times 10^{-2} / 5.15 \times 10^{-5} \approx 600$ 倍：**全年口径**支撑"附件 1 即附件 2/4 逐时段均值"的身份断言，**窗口口径**则说明问题 1 的典型日结果**不可**直接当作问题 2–4 的窗口基准（§1.3 表 1 第二行）。

3.4　电价与负载的结构特征

按附件 4 统计，波动电价均值 0.7662 元/kWh、最小 0.0076、最大 1.7936 元/kWh；按附件 1 统计，固定电价均值同为 0.7662 元/kWh、最小 0.3713、最大 1.3952 元/kWh。按附件 2 统计，负载均值 4626.0337 kW、最小 1995.7176、最大 7978.8849 kW，日均用电量 110 802.6332 kWh；光伏均值 2311.7857 kW、峰值 10 216.2000 kW，日均发电量 57 092.4850 kWh。

两类结构性事实直接影响建模方式。其一，**套利是否有利可图**：盈亏平衡价比阈值为 $1/(\eta_c \eta_d) = 1/0.81 \approx 1.2346$，而固定电价的极值之比 $1.3952/0.3713 \approx 3.76$（口径见 §1.1）远高于它。其二，**约束落在哪一侧**：光伏峰值 10 216.2000 kW 明显高于充放电功率上限 5000 kW，故有光伏功率超出充电能力的时段，盈余只能靠储能吸纳或弃光；容量摆幅与功率谁更紧由第四章的容量/功率双向扫描判定。

电价与负载的关系具有**尺度依赖性**（表 7 按三个尺度报告）。

表 7　电价与负载、净负荷、光伏的相关系数（三个尺度；池化与去形状口径的样本量为 $365 \times 144 = 52\,560$ 个时段观测，日级口径为 365 个日观测；全样本为 2025 年全年 365 天口径，与情景保真度检验（0.2620 → 0.2597）验证，见 §7.2）

变量对	池化（全部时段）	日级（逐日先算再合并）	剔除日内形状后
电价 — 负载	+0.5077	+0.8977	+0.4554
电价 — 净负荷 $L - PV$	+0.2602	+0.9824	+0.1583
电价 — 光伏	+0.0284	−0.3431	+0.0484

读法如下。① **电价与负载共动**：日级 +0.8977，剔除日内共同形状后仍保留 +0.4554。② **"电价—光伏"的池化值具有误导性**：池化仅 +0.0284、日级 −0.3431，方向相反；真实结构是**光伏出力大的午间对应电价低谷**，独立抽样会生成"午间光伏大发同时电价高企"的不可能情景并低估尾部费用，故问题 4 用保留相关结构的联合情景（§7.2）。③"电价—净负荷"随尺度递进：池化 +0.2602、日级 +0.9824、剔除日内形状后 +0.1583；④"电价—光伏"去形状后为 +0.0484，接近 0，说明同向关系几乎全部来自日尺度。以上只描述**共动/同向变动**，本文不主张其中任何一方为因。

3.5　光伏预报误差的统计刻画

本节把附件 3 中**当天** 0:00 **发布**的 24 h 预报（即问题 2/4-2 的日前计划依据）与附件 2 实际功率逐槽配对：整点预报先展开为 6 个 10 min 槽，再逐槽相减得 $r = \widehat{PV} - PV$（单位 kW）。夜间光伏接近 0、相对误差无意义，故同时报告两个口径，并统一以**目标整点** $h \geq 5$（5:00–24:00，每天 19 个整点）为白天口径、不按功率阈值筛选，使两口径分母固定、可比（表 8）。

表 8　光伏预报误差统计（样本为 2025 年每天 0:00 发布的 24 h 预报与附件 2 对应实际功率的逐槽配对；MAE、RMSE、bias 单位均为 kW）

口径	样本量 n	MAE	RMSE	bias
全时段（目标整点 0–23 时，144 槽/天）	52 560	361.58	636.37	−26.96
白天（目标整点 ≥ 5 时，114 槽/天）	41 610	456.72	715.22	−34.06

白天口径的 MAE 为全时段口径的 $456.72/361.58 \approx 1.26$ 倍（夜间接近 0 的样本摊薄了全时段指标），建模应以白天口径为依据。更关键的是 *bias* 的符号：按目标整点聚合后正负号系统性变号，见表 32。

表 32 的形状有直接的决策含义：**上午被系统性高估**（7 时达 +733.00 kW）使 0:00 计划的光伏出力偏大、购电量偏少，缺口更易触发 5 倍紧急购电（口径 口径 C-05）；**下午被系统性低估**（15 时达 −801.87 kW）使计划购电偏多，多买电量按计划电价白付，或在问题 3 中触发 0.5 倍违约电价与 1.5 倍超额电价。全天聚合 bias 只有 −26.96 kW（白天 −34.06 kW），是两类相反偏差抵消后的净额，**不能据此判断风险方向**；19–23 时的平均偏差落在 $[-0.15, 0.00]$ kW 内，出力均接近 0，不触发任何惩罚。故决策层必须按**目标整点**设置安全边际而非按全天平均误差一次性纠偏，场景生成直接用逐槽预报残差的经验分布（§5.2）以保留这种双向结构（(见 C-Q2-01)）。

数据质量与缺失值　三条结构断言全部通过：附件 1 恰 144 行；附件 3 日期列前向填充后每天恰有 4 个发布时刻（合计 1460 行）；结果窗口恰 334 天（2025-02-01 至 12-31），结果表恰 147 列（日期 + 144 个时段 + 全天购电量 + 全天购电费）。断言通过后不再插补：附件 3 稀疏日期列以 `ffill` 补齐，其余附件按位置读法逐列读入。数据侧唯一实质性质量问题是**预报质量的非均匀性**（上午与下午偏差方向相反、傍晚之后无有效预报信息、误差水平白天高于全时段），故本文不采用"预报误差独立同分布且无偏"的简化，而把逐槽残差的经验分布直接用于场景生成（§5.2、§7.2），并在决策层按目标整点设安全边际与滚动更新（§5.4、§6.3）。

四、问题 1：确定性条件下的计划购电模型

4.1　模型建立

建模口径　本文采用**电量口径**建模：决策变量均以电量（kWh）为单位，步长 $\Delta t = 1/6$ h，充放电功率上限 $\overline{P} = 5000$ kW 折算为单槽电量上限 $\overline{P}\Delta t = 833.3333$ kWh；电价 p_t（元/kWh）与电量相乘即为费用（元），**不再乘** Δt；负载与光伏（kW）进入模型前一律乘以 Δt 化为电量。

数据与信息结构　电价与负载逐日相同、光伏只给出一日预测，故只需求解**一个典型日**（附件 1，144 个 10 min 时段），信息结构为"当日真值已知"的确定性情形。该日电价均值 0.7662 元/kWh、范围 $[0.3713, 1.3952]$ 元/kWh；储能额定容量 12 000 kWh、最大充放电功率 5000 kW，储电量运行区间 $[1200, 10800]$ kWh（可用摆幅 9600 kWh），充放电效率单边各 0.9（往返 0.81，假设 A2）。结果窗口（2025-02-01 至 2025-12-31，共 334 天）背景量：负载日均电量 110 802.6332 kWh、光伏日均电量 57 092.4850 kWh、光伏峰值功率 10 216.2000 kW。

决策变量　计划购电量 E_t、充电量 c_t、放电量 d_t、弃光电量 s_t 与时段末储电量 S_t（单位均为 kWh），$t \in \mathcal{T}$，$|\mathcal{T}| = 144$。

目标函数　以全天购电费用最小为目标（购电量均按当日实际电价结算）：

$$\min_{E,c,d,s,S} \quad C^{\text{plan}} = \sum_{t=1}^{144} p_t \, E_t \tag{1}$$

约束条件

$$E_t + \big(PV_t - L_t\big)\Delta t + d_t - c_t - s_t = 0, \qquad \forall t \in \mathcal{T} \tag{2a}$$

$$S_t = S_{t-1} + \eta_c \, c_t - \frac{d_t}{\eta_d}, \qquad \forall t \in \mathcal{T} \tag{2b}$$

$$\underline{S} \leq S_t \leq \overline{S}, \qquad \forall t \in \mathcal{T} \tag{2c}$$

$$0 \leq c_t \leq \overline{P}\Delta t, \qquad 0 \leq d_t \leq \overline{P}\Delta t, \qquad \forall t \in \mathcal{T} \tag{2d}$$

$$S_0 = S_{144} = S^{init} = 6000 \text{ kWh} \tag{2e}$$

$$E_t \geq 0, \qquad s_t \geq 0 \qquad \forall t \in \mathcal{T} \tag{2f}$$

式 (2a) 即" 微网提供的电能不低于小区负载" 的代数表达，其中 $\big(PV_t - L_t\big)\Delta t$ 为净负荷电量（盈余为负）。光伏盈余可能超过充电功率上限与负载之和，故以**等式加松弛量** $s_t \ge 0$ 写出以保证可行；附件 1 典型日的日内光伏盈余低于容量摆幅 9600 kWh 与单槽吸纳能力 $\overline{P}\Delta t = 833.3333$ kWh，最优弃光 $s_t \equiv 0$（口径 C-06 的处置量在问题 1 中即弃光电量，见 §4.5）。同一典型日上" 含松弛量的等式"" 不含松弛量的等式"" 不等式" 三种写法的最优值完全相同（均为 35 126.9486 元），保留 s_t 的意义在于**实际日期序列**（窗口内光伏峰值功率 10 216.2000 kW）的可行性。式 (2e) 即题面" 储能设备在 0:00 和 24:00 的储电量相同"，本文进一步取两者均为附录给定的初始储电量 6000 kWh，敏感性见 §4.5。

模型规模 求解采用 $4 \times 144 = 576$ 个决策变量（E_t, c_t, d_t, S_t）、144 个等式约束（式 (2b)）与 144 个不等式约束（式 (2a) 与式 (2d) 的合并形式），是连续线性规划 (LP)；显式保留 s_t 时变量数为 $5 \times 144 = 720$ 个。充放互斥由模型结构自动保证（§4.2），**无需 0–1 变量**，模型保持纯 LP。

4.2 模型性质

命题 1 (充放互斥)**.** 当 $\eta_c \eta_d < 1$ 时，最优解中不存在同一时段既充电又放电的情形。

证明. 设某时段 t 同时有 $c_t > 0$ 与 $d_t > 0$。取 $\delta = \min\{c_t,\ d_t/(\eta_c\eta_d)\} > 0$，构造新解 $c_t' = c_t - \delta$、$d_t' = d_t - \eta_c\eta_d\delta \ge 0$，其余时段变量不变。由式 (2b)，储电量变化为 $\eta_c(-\delta) - (-\eta_c\eta_d\delta)/\eta_d = 0$，故整条储电量轨迹与全部上下界约束均不变，新解仍可行。由式 (2a)，购电量变化为 $\Delta E_t = \Delta c_t - \Delta d_t = -\delta + \eta_c\eta_d\delta = -\delta(1 - \eta_c\eta_d) < 0$，目标函数下降 $p_t\delta(1 - \eta_c\eta_d) > 0$，与原解最优矛盾。 □

命题 1 的证明只用 $\eta_c\eta_d < 1$、$p_t > 0$ 与平衡式的结构，对问题 1–4 的全部变体（含波动电价与调整成本）一致成立。数值验证：同时充放时段数 0，储电量递推残差 9.09×10^{-13} kWh，充放比偏差 4.44×10^{-16}（审计项 A5，见 §4.5）。

4.3 对偶分析与阈值型最优策略

引入两个对偶量：ν_t 为式 (2b) 的影子价格，即**时段 t 末单位储电量的边际价值**（水价值，元/kWh）；α_t 为式 (2a) 的影子价格，即**并网点单位电量的边际价值**（元/kWh）。符号约定：α_t 取非负值，与该约束的标准拉格朗日乘子相差一个负号（标准乘子的非正性由 $s_t \ge 0$ 的互补松弛给出）。在充放电功率界与储电量界均不活跃时，对 c_t, d_t 的平稳性条件给出互补松弛式：

$$z_t^c = \alpha_t - \eta_c\,\nu_t \ \ge 0, \qquad c_t\, z_t^c = 0 \tag{3a}$$

$$z_t^d = -\alpha_t + \frac{\nu_t}{\eta_d} \ \ge 0, \qquad d_t\, z_t^d = 0 \tag{3b}$$

式 (3a)：多充 1 kWh 的边际代价为 α_t，换来 η_c kWh 储电、价值 $\eta_c\nu_t$，故**并网点电量边际价值低于储电边际价值折算水平时才充电**；式 (3b)：多放 1 kWh 可替代 α_t 的购电成本，代价是储电量减少 $1/\eta_d$ kWh。两条阈值可直接与电价比较：**当 $E_t > 0$（正在向外网购电）时** $\alpha_t = p_t$（实测偏差 0.000）；当 $E_t = 0$（光伏与放电已满足负载）时 $\alpha_t \le p_t$，实测 $\alpha_t - p_t$ 最大值为 -0.0042 元/kWh，恒不为正。由此得**阈值型最优策略**：

命题 2 (阈值律)**.** 在储电量上下界均不活跃（$\underline{S} < S_t < \overline{S}$）的时段，最优充放电策略由两条阈值刻画：当 $\alpha_t \le \eta_c\nu_t$ 时充电，当 $\alpha_t \ge \nu_t/\eta_d$ 时放电；当

$$\eta_c\,\nu_t \ < \ \alpha_t \ < \ \frac{\nu_t}{\eta_d} \tag{4}$$

时 $c_t = d_t = 0$，构成" 死区"。

式 (4) 的死区宽度随往返效率下降而变宽：两条阈值之比为 $(\nu_t/\eta_d)/(\eta_c\nu_t) = 1/(\eta_c\eta_d)$，$\eta_c = \eta_d = 0.9$ 时为 $1/0.81 = 1.2346$。即**同一水价值水平下，放电时段的并网点电量边际价值须高于充电时段的 1.2346 倍，跨时段套利才有利可图**；购电时段该判据直接化为放电电价高于充电电价 1.2346 倍，此即全文的盈亏平衡比。

命题 2 的**适用边界**：它在" 纯套利子问题（变量仅 c_t, d_t, S_t）且储电量界不活跃" 时精确成立；$\underline{S}$ 或 $\overline{S}$ 活跃时须回到互补松弛形式式 (3a)–(3b) 并额外计入界乘子 $\lambda_t^{\pm}$；充放电功率界活跃时 z_t^c 或 z_t^d 不再为零，简单阈值关系失效。数值验证分两层：全部 144 个时段上按 α_t 判定充电/放电/死区的**违反次数为 0**；储电量界不活跃的 97 个时段子集（充电槽 42 个、放电槽 36 个）上违反次数同样为 0；两条互补松弛恒等式的最大残差均为 0.000 元/kWh。实测 $\nu_t \in [0.4711, 1.1305]$、$\alpha_t \in [0.3713, 1.2561]$ 元/kWh 均**时变**，故无" 全天统一的电价阈值"：正确表述是两级结构——先由 LP 对偶确定时变水价值曲线，再逐时段比较 α_t 与两条阈值 （见 C-Q1-02）。

水价值 ν_t 随储电量水平单调不增；充放电功率的边际价值须由式 (2d) 的对偶量另行报告（§4.5）。

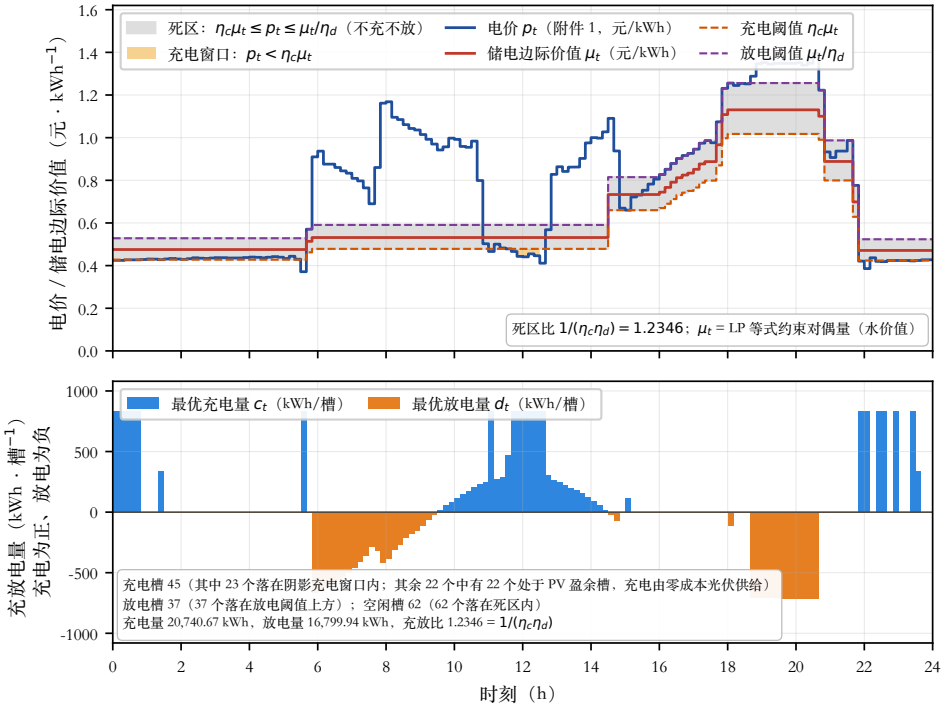


图 3　最优性证书：电价、水价值与阈值型策略（上子图：电价 p_t 与并网点电量边际价值 α_t，虚线为充电阈值 $\eta_c\nu_t$ 与放电阈值 ν_t/η_d，阴影为死区；下子图：最优充电量与放电量；对象为附件 1 典型日的 144 个 10 min 时段，电价单位 元/kWh，电量单位 kWh；阈值一致性检验在全部 144 个时段上的违反次数为 0，储电量界不活跃的 97 个时段子集内违反次数亦为 0）

4.4　无储能基线的闭式解

口径 C-01　无储能基线购电费 C^{base}
定义　$C^{base} = \sum_{t=1}^{144} p_t\,\Delta t\,\max\big(L_t - PV_t,\,0\big)$，即不配储能、逐时段按净负荷购电的费用；**数据**　典型日口径取附件 1 的负载、光伏与电价；**时段**　$t = 1, \ldots, 144$，$\Delta t = 1/6$ h；结果窗口 2025-02-01 至 2025-12-31（334 天）；**单位**　元（表内保留 2 位小数，内部计算不取整）；**不含**　储能充电成本、紧急购电费、调整与违约费用、容量补偿与循环衰减；**对照**　三个变体的数值见表 9；窗口口径下波动电价较固定电价高 619 463.23 元（相对固定价口径 3.78%）；**复算**　`code/config.py::baseline_cost`，审计项 A1。

表 9　无储能基线 C^{base} 的三个变体及其适用问题（单位：元）

变体	口径	电价来源	适用问题	数值
V1	附件 1 典型日，单日周期	附件 1（固定价）	问题 1	48 052.0466
V2	334 天窗口，逐日独立	附件 1（固定价）	问题 2、3	16 407 319.63
V3	334 天窗口，逐日独立	附件 4（波动价）	问题 4	17 026 782.86

三个变体不得混用，否则论文中出现的多个”节省率”互不可比。

成本分解恒等式　由式 (2a) 解出 E_t 并代入目标，可得

$$\sum_t p_t E_t = \underbrace{\sum_t p_t \Delta t \max(L_t - PV_t, 0)}_{C^{base}} + \underbrace{\sum_t p_t\big(\Delta t \min(L_t - PV_t, 0) + c_t - d_t + s_t\big)}_{C^{batt}}, \tag{5}$$

其中 C^{batt} 为储能与弃光带来的成本增量（典型日取负值，即费用下降）。式 (5) 是**逐项恒等式**而非”两个独立最优值之和”，C^{batt} 的最小化仍需求解式 (1)–(2f)：代入 $C^{base} = 48\,052.0466$ 元与最优值 35 126.9486 元得 $C^{batt} = -12\,925.0980$ 元，与目标函数直接相减一致。

4.5　求解结果

在附件 1 典型日上求解式 (1)–(2f)，得：全天购电量 59 482.6990 kWh（口径 C-02）、全天购电费 35 126.9486 元，相对无储能基线 $C^{base} = 48\,052.0466$ 元（口径 C-01 变体 V1）节省 12 925.0980 元，**节省率 26.90%**（口径 C-04，相对 口径 C-01）（见 C-Q1-03）。储能充电量 20 740.6661 kWh、放电量 16 799.9396 kWh，之比 $20\,740.6661/16\,799.9396 = 1.2346 = 1/(\eta_c\eta_d) = 1/0.81$，即**放电量与充电量之比**

等于往返效率 0.81，与式 (2b) 的效率方向一致（充放比偏差 4.44×10^{-16}）。最优弃光量 0.0000 kWh（口径 C-06）（见 C-Q1-01）；储电量全程位于 [1200.0000, 10800.0000] kWh，0:00 与 24:00 均为 6000.0000 kWh（严格日周期，口径 C-08）。

表 10　微网在指定时间段的购电量及全天的购电量和购电费（问题 1，单位：kWh / 元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.0000	12:00–12:10	480.4124	14:00–14:10	0.0000
16:00–16:10	445.4317	18:00–18:10	531.8940	20:00–20:10	0.0000
全天购电量（kWh）：59 482.6990		全天购电费（元）：35 126.9486			

表注：时段按题面表述取区间左端时刻，如 10:00–10:10 表示 [10:00, 10:10)；`result1.xlsx` 沿用模板既有的时段末端标签（首个标签 0:10–0:20、末个标签 0:00+1–0:10+1），其数值与本文槽位按**位置**一一对应（槽 j 写入第 $j+1$ 个数据行），标签仅作显示，两者相差一个标签位。

表 11　储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（问题 1，单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4500.0000	0.0000	4:00–8:00	833.3333	6365.8412
8:00–12:00	4787.9643	1702.9970	12:00–16:00	5286.0352	91.1014
16:00–20:00	0.0000	5780.1319	20:00–24:00	5333.3333	2859.8681
0:00 储电量：6000.0000			24:00 储电量：6000.0000		

表注：每段含 36 个 10 min 槽；6 段之和与逐槽求和一致——充电 20 740.6661 kWh、放电 16 799.9396 kWh（口径 C-07）。

物理机制见图 4：充电时段与光伏盈余时段、低价时段对齐，放电时段与高价时段对齐，储电量全程位于 [1200, 10800] kWh 内；两组时段只是**部分重合**，说明主机制是面向价格的跨时段套利，而非简单的”把午间盈余搬到傍晚”。

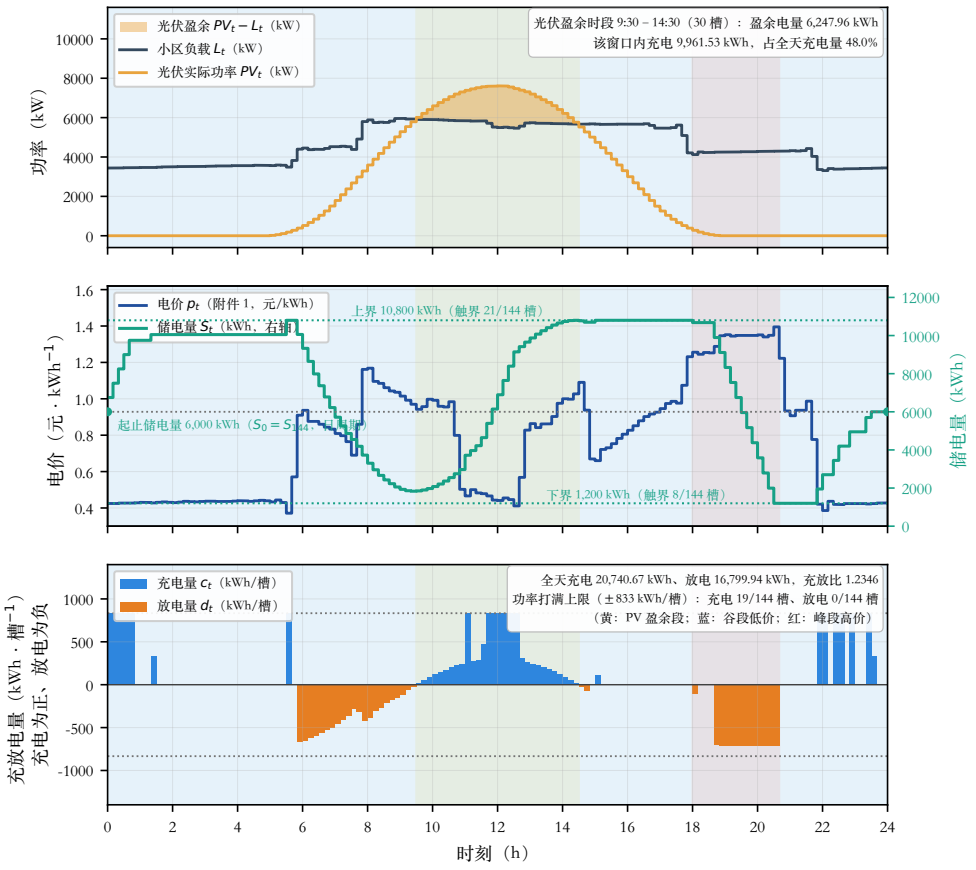


图 4　最优策略的机制解释（三子图共享时间轴：光伏与负载功率（kW）、电价与储电量（元/kWh、kWh，储电量右轴标出上下界 1200/10800 kWh）、充放电量（kWh）；对象为附件 1 典型日的 144 个 10 min 时段，竖直虚线连接光伏盈余时段、充电时刻与低价时段）

成本分解与节省来源见图 9（口径 C-01 变体 V1 为底柱，末柱为最优全天购电费 35 126.9486 元，差额即储能搬运与弃光处置的收益）。

解的最优性与可行性复核 除目标值与独立复算一致外，另做四类残差检验（§9.3 与附录 D）：供需平衡残差 1.49×10^{-13} kWh、最小供给盈余 -2.27×10^{-13} kWh、储电量递推残差 9.09×10^{-13} kWh、日周期偏差 0.0000 kWh；储电量越界、充放电功率越界、同时充放与负购电的时段数均为 0；费用相对复算误差 0.0000。

日周期条件的敏感性 题面只要求 0:00 与 24:00 的储电量**相等**，未要求等于 6000 kWh。放松为”两端相等、取值自由”时最优费用 35 118.5986 元、两端储电量 8550.0000 kWh，比主口径低 8.3500 元（−0.0238%）。相差不足 0.03% 仍须显式声明口径：若允许日末自由（只固定日初），储能会被”掏空”到下界，费用虚低、节省率不可比。

容量与充放功率的双向扫描 储能能否进一步降费，取决于”容量摆幅”与”充放功率”哪个更紧。本文在保持 10%–90% 放电深度（$\underline{S}=0.1E_{cap}$、$\overline{S}=0.9E_{cap}$ 同步移动）下扫描额定容量，并单独扫描充放功率上限，结果列于表 34 与图 5。

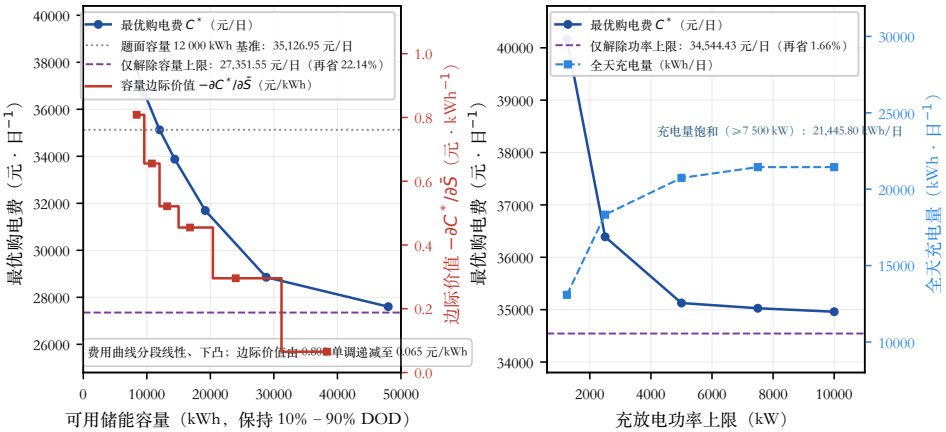


图 5 储能容量与充放功率上限的双向扫描（左：以额定容量为横轴的最优全天购电费与边际价值 $\partial C^*/\partial E_{cap}$；右：以充放功率上限为横轴的最优购电费；对象为附件 1 典型日的 144 个 10 min 时段，日周期条件 $S_0=S_{144}=6000$ kWh，容量与功率各 5–7 个扫描点，费用单位：元）

扫描结果（表 34）：①容量由 12 000 kWh 增至 28 800 kWh 可再省 6272.42 元（17.86%），充放功率上限由 5000 kW 提高到 10 000 kW 只能再省 166.36 元（0.47%）；②单独解除储电量上限可多省 22.1%，单独解除充放功率上限只多省 1.7%；③充电功率界在 19/144 个时段被打满、放电功率界 0/144，储电量上界活跃 21/144、下界活跃 8/144。故**容量摆幅是主要瓶颈，充放电功率在部分时段同时构成约束，两者量级不同**；与第九章 Sobol 全局灵敏度（典型日模型，$N=256$）一致：全天购电费的一阶指数中储电量上限 0.2510、下限 0.1099，充放功率上限 0.0263（电价、光伏、负载为常数缩放，指数为 0）。

效率口径的敏感性 题面”充放电效率为 90%”未区分充电侧与放电侧。本文主口径取**单边各 0.9（往返 81%）**，并给出对照：仅充电侧 0.9 时最优购电费 33 416.53 元、节省 14 635.52 元（30.46%）；仅放电侧 0.9 时 34 168.61 元、节省 13 883.44 元（28.89%）；主口径 35 126.95 元、节省 12 925.10 元（26.90%）；若把 90% 读作往返效率折算为单边 0.9487，最优购电费 33 801.06 元、节省 14 250.99 元。三种单边读法的节省额极差 $14\,635.52-12\,925.10=1710.42$ 元，占主口径节省额 13.2%，故效率口径必须显式声明，否则节省率不可比。

五、问题 2：不确定性下的两阶段计划购电模型

5.1 问题分析与决策结构

问题 2 与问题 1 的差别有三点：电价仍逐日相同（附件 1 固定价，均值 0.7662 元/kWh），但负载与光伏**逐日逐时段变化**；计划仍在每天 0:00 制定；若微网提供的电能低于负载，须按**交易时刻电价的 5 倍**紧急购电。故决策分两个层次：

- **第一阶段（日前 0:00）**：只知预报，一次性确定全天 144 个时段的 **计划购电量** E_t 与**充放电量** c_t, d_t，该决策**不依赖场景**；
- **第二阶段（实时结算）**：实际值揭示后，计划与实际之差由 **紧急购电量** $g_{t,\omega}$（缺口，按 $5p_t$ 计价）与**光伏盈余及超购部分的处置**吸收。

信息结构声明 本文采用**信息结构 R3**：计划依据为**附件 3 中当日 0:00 发布的光伏功率预报** $\widehat{PV}_t^{\mathrm{fcst}} = \widehat{PV}_t^{(0:00)}$；题面未给负载预报，故计划阶段按" 当日负载曲线已知" 处理（等价于负载预报无误差），并在假设节显式声明；**结算依据为附件 2 的当日实际负载与实际光伏**。两者是**两套数据**，其差异即紧急购电的来源。作为对照，另设完美信息 R1（0:00 即知当日真值）与典型日预报 R2（附件 1 的日均形状），结果见 §5.5。

时标解释：紧急购电发生在结算时刻 紧急购电不是在某个决策时刻" 决定" 的，而是在**结算时刻**由缺口 $g_{t,\omega} = \max\{0, (L_{\omega,t} - PV_{\omega,t})\Delta t + c_t - d_t - E_t\}$ 唯一确定的：计划量照常执行，缺口只能以 $5p_t$ 的价格即时补足，不能回溯改写计划。这一时标差异是下文报童模型第三条适用边界的来源（§5.4）。

一个必须显式声明的建模选择 **充放电计划同样在第一阶段确定，实时不允许重调度**。依据是题面" 其他时间段的购电费用均按计划购电量计算"，以及结果文件要求填报逐日逐时段的充放电量。据此，第一阶段买到的高价电不能因实际光伏更好而转手，计划购电量**允许高于**预测口径的需求：超出部分既不产生售电收益、也不再被后续时段使用（超购即被处置），其成本为 p_t。这一" 超购即弃" 的设定使模型天然具有 **安全边际**，也是报童型权衡在本问成立的结构性原因（经济价值见 §5.4）。

5.2 场景生成与削减

光伏预报误差是本问唯一的不确定性来源。定义**逐日逐时段残差**

$$r_{d,t} = \big(\widehat{PV}_{d,t}^{\mathrm{fcst}} - PV_{d,t}^{\mathrm{act}}\big)\Delta t, \qquad d \in \mathcal{D},\ t \in \mathcal{T}, \tag{6}$$

单位为 kWh，样本为窗口内全部 $|\mathcal{D}| = 334$ 天 $\times 144$ 槽。场景集分三层生成：

1. **按日总残差分层**：按 $s_d = \sum_t r_{d,t}$ 升序把 334 天**等频**划分为 K 层（主口径 $K = 60$，每层 5–6 天），保留残差水平分布的**全部区间**（尾部不被均值化抹平）；
2. **层内取 medoid**：每层取" 到本层其他样本欧氏距离之和最小" 的 **真实历史日残差向量**（不做中心化与平滑），逐槽保留残差的**日内形状**与时段间相关结构；
3. **权重 = 层样本占比**：$W_\omega = n_\omega/334$，$\sum_\omega W_\omega = 1$。

情景 ω 下实际净需求 $ND_{\omega,t} = ND_t^{\mathrm{fcst}} + r_{\omega,t}$，其中 $ND_t^{\mathrm{fcst}} = (L_t^{\mathrm{fcst}} - \widehat{PV}_t^{\mathrm{fcst}})\Delta t$。场景集一次性生成后**存盘固定**，问题 3、4 与全部消融实验读取同一份，以保证" 改进" 来自模型而非场景噪声。

三种层数下总费用极差 13.65 万元，仅为主口径的 0.96%，**结论对场景层数稳健**；紧急购电量对 K 更敏感（$K = 40$ 时为 $K = 120$ 时的 1.68 倍）：层数越少、单层样本越多，代表场景越难贴近尾部。场景集退化为" 残差恒为零" 的单一场景（$K = 1$，即严格按预报口径购电）即下文的**无安全边际**消融（§5.4）。

5.3 两阶段随机规划模型

$$\min_{E,c,d,S} \quad \sum_{t=1}^{144} p_t E_t \ + \ \sum_{\omega\in\Omega} W_\omega \sum_{t=1}^{144} 5\, p_t\, g_{t,\omega} \tag{7a}$$

$$E_t,\ c_t,\ d_t,\ S_t \text{ 与 } \omega \text{ 无关}, \qquad \forall t \in \mathcal{T} \tag{7b}$$

$$E_t - c_t + d_t \ \geq\ \big(L_t^{\mathrm{fcst}} - \widehat{PV}_t^{\mathrm{fcst}}\big)\Delta t, \qquad \forall t \in \mathcal{T} \tag{7c}$$

$$g_{t,\omega} \ \geq\ \big(L_{\omega,t} - PV_{\omega,t}\big)\Delta t + c_t - d_t - E_t, \qquad g_{t,\omega} \geq 0 \tag{7d}$$

$$S_t = S_{t-1} + \eta_c c_t - \frac{d_t}{\eta_d}, \quad \underline{S} \leq S_t \leq \overline{S}, \quad 0 \leq c_t, d_t \leq \overline{P}\Delta t, \quad S_0 = S^{init}, \quad S_{144} \geq S_0 \tag{7e}$$

$$E_t \geq 0, \qquad g_{t,\omega} \geq 0 \tag{7f}$$

式 (7b) 是两阶段结构的代数表达：带 ω 下标的量为**事后**量（结算量），第一阶段变量 E_t, c_t, d_t, S_t 在全部 60 个场景间共享。式 (7c) 用 **不等号**：计划购电量只要求不低于" 预测口径净需求 + 充电量 − 放电量"；式 (7d) 的缺口由紧急购电补齐，$5p_t > 0$ 使最优解中 $g_{t,\omega}$ 取其下界，即

$$g_{t,\omega} = \max\Big\{0,\ \big(L_{\omega,t} - PV_{\omega,t}\big)\Delta t + c_t - d_t - E_t\Big\}, \tag{8}$$

是式 (7d) 的紧形式；式 (7d) 用的是**计划充放电量** c_t, d_t（不含 ω），与 §5.1 一致。模型规模（$K = 60$）为每个子问题 $576 + 144 \times 60 = 9216$ 个变量、144 个等式与 $144 \times (1 + 60)$ 个不等式，仍是纯 LP，用 SciPy 的 HiGHS 求解器精确求解；结果窗口内共求解 334 个子问题（逐日顺序推进，日初储电量取前一日末值，1 月作预热期，假设 A8），终端条件取 $S_{144} \geq S_0$（假设 A6）。

5.4 报童型安全边际的可解释刻画

暂不考虑储能的跨期耦合，把每个时段看作独立的”报童问题”：计划购电量 E_t 相当于订货量，当日实际需求 $D_t = (L_t - PV_t)\Delta t + c_t - d_t$ 相当于需求量。缺 1 kWh 要多付 $(5-1)p_t = 4p_t$（**缺货成本** $c_u = 4p_t$，正常电价 p_t 已计入计划口径）；多买 1 kWh 的损失记为 c_o。由报童模型，最优计划量是需求分布的**临界分位数**：

$$q_t^* = F_{D_t}^{-1}\left(\frac{c_u}{c_u + c_o}\right) = F_{D_t}^{-1}\left(\frac{4p_t}{4p_t + c_o}\right), \tag{9}$$

与之互补的是**最优缺货概率** $1 - q_t^*$；两者不可混写。

过剩成本 c_o 的两种口径构成临界分位数的**上下界**：**口径一**（超购全废弃）$c_o = p_t$，临界分位数 $4/5 = 0.800$、缺货概率 $1/5 = 0.200$；**口径二**（超购被储能吸纳并在后续时段替代购电，单位回收价值 p^{low}）$c_o = p_t - p^{low} \approx 0.37$ 元/kWh，取全天电价均值 $p_t = 0.7662$ 元/kWh 得临界分位数 $3.0648/(3.0648+0.37) = 0.892$、缺货概率 $0.108 \approx 1/10$。故**缺货概率的两个界是** $1/10$ **与** $1/5$，覆盖率对应 0.892 与 0.800。

表 12　报童近似的两条界与两阶段模型的实测缺货频率（窗口 2025-02-01 至 2025-12-31）

过剩成本口径	临界分位数	最优缺货概率	样本量	与实测的关系
口径一：超购全废弃 $c_o = p_t$	0.800	0.200（1/5）	—	缺货概率上界
口径二：超购可回收 $c_o \approx 0.37$ 元/kWh	0.892	0.108（≈ 1/10）	—	缺货概率下界
两阶段随机规划实测	覆盖率 0.9145	**0.0855**	48 096 槽	低于下界（更保守）

表注：实测缺货频率 = 紧急购电量大于 10^{-6} kWh 的槽数占 334 天 × 144 槽的比例；覆盖率 = 1− 缺货频率；电量满足率为 99.04%（口径与 口径 C-05 一致）。

两阶段随机规划的实测行为：**逐槽缺货频率 8.55%**（覆盖率 91.45%）低于下界 1/10，**电量满足率 99.04%**，全窗口**安全边际合计 576 299.67 kWh**（平均 11.98 kWh/槽）（见 C-Q2-01）。实测更保守的原因在结构而非参数：**储能同时承担对冲功能**，同一份计划可由电池放电在缺口时段补足，单期报童模型无法刻画”计划购电量”与”电池吞吐”的替代关系。故式 (9) 与表 12 只作**方向性近似与上下界**，**不是**精确最优解。

安全边际的经济价值由**无安全边际消融**量化：场景集退化为单一情景（$K = 1$，残差恒为零，严格按预报口径购电）后，全年计划购电费 12 570 533.91 元、紧急购电费 4 828 025.76 元、合计 17 398 559.68 元，比无储能基线 16 407 319.63 元高 6.04%，紧急购电量 1 156 618.02 kWh、334/334 天全部触发；主口径费用 14 264 792.13 元、紧急购电量 172 015.82 kWh。分解为：**安全边际把计划购电费推高 1 067 018.99 元（边际电量 576 299.67 kWh 按均价 0.6176 元/kWh 计 355 918.38 元，其余 711 100.60 元来自储能排产与购电时序的同步调整），却把紧急购电费压低 4 200 786.52 元，净收益 3 133 767.55 元（相对不设边际降低** 18.0%**）**。即边际电量只占全窗口计划购电量 22 220 333.14 kWh 的 2.59%，却消掉 87.0% 的紧急购电费。惩罚倍数灵敏度方向一致：同一典型日与同一残差场景集上把倍数由 $k = 2$ 提高到 $k = 10$，单日最优安全边际由 671 kWh 增至 3 334 kWh，期望紧急购电量由 1 246 kWh 降至 187 kWh，目标值由 39 224.54 元升至 41 758.99 元（$k = 5$ 时为 40 747.45 元）。

报童模型在本问题中**不是精确解**，其偏离来源有四条，也就是它的适用边界：

1. **多时段耦合**：144 个时段通过储电量递推耦合，本时段多买的电会占用电池容量、影响其后所有时段，不能拆成 144 个独立报童问题；
2. **储能缓冲**：过剩电量可被储能吸纳并在后续高价时段替代购电，故 c_o 严格小于 p_t，临界分位数应高于口径一的 0.800；
3. **紧急购电的时标在结算时刻**：缺口只能事后按 $5p_t$ 补足，受当日计划充放电量与放电功率限制，不能回溯改写计划；
4. **残差分布未知**：本文用 334 天样本的**样本分位数**替代总体分位数，每个时段仅 334 个观测，分位数估计带误差，须报置信区间。

5.5 对照方案与风险维度

在同一批实际轨迹上比较三种计划依据：

- **R1 完美信息**：0:00 即知当日实际负载与实际光伏，紧急购电量恒为 0。在与主方案相同的日决策结构下逐日求解，总费用 **13 411 560.90 元**，相对无储能基线节省 18.26%，是”预测误差成本”的**下界**；若放弃日决策解耦、把 334 天写成单一 LP（单一储电量轨迹），费用 12 228 166.56 元，为更松的下界参考；
- **R3 本文主方案**（附件 3 的 0:00 预报）：总费用 14 264 792.13 元，相对基线节省 13.06%；
- **R2 典型日预报**：用附件 1 的日均形状作预报，总费用 **23 484 646.63 元**，比无储能基线高 43.14%，紧急购电量 2 525 548.76 kWh（286/334 天触发）。附件 1 的数值是附件 2 实际功率的 **全年逐时段均值**而非日前预报，只用于说明”预报质量决定成本”，**不可作为主方案**。

R3 与 R1 之差 $14\,264\,792.13 - 13\,411\,560.90 = 853\,231.23$ 元（相对 R1 高 6.36%）即**预测误差的成本**，即"5 倍紧急购电" 机制对应的金额。

风险维度上，期望费用最小化对极端场景不敏感：在 334 天中每 6 天取 1 天的子样本（56 天）上做 CVaR（置信水平 $\alpha = 0.95$，口径 C-11）与 λ 扫描，λ 由 0 增至 1 时期望费用由 41 995.59 元/日升至 42 411.56 元/日（+0.99%），$\mathrm{CVaR}_{0.95}$ 由 52 176.36 元/日降至 50 510.46 元/日（−3.19%），日均紧急购电量由 480.2 kWh 降至 313.8 kWh；完整前沿与回测见 §7.4，本节只保留与日前决策直接相关的 R1/R2/R3 对照。

5.6 求解结果

在 2025-02-01 至 2025-12-31（334 天）窗口内求解式 (7)，总费用 **14 264 792.13 元**（1 426.48 万元），其中**计划购电费 13 637 552.90 元、紧急购电费 627 239.24 元**；相对无储能基线 $C^{base} = 16\,407\,319.63$ 元（口径 C-01 变体 V2）节省 2 142 527.50 元，节省率 13.06%（口径 C-04），两项费用占比 95.60% 与 4.40%（口径 C-03）。全窗口计划购电量 22 220 333.14 kWh（口径 C-02），紧急购电量 172 015.82 kWh，出现在 296/334 天（口径 C-05）；逐槽缺货频率 8.55%、电量满足率 99.04%、安全边际合计 576 299.67 kWh，与 §5.4 一致。

口径 C-05　紧急购电量 G

定义　$G = \sum_{d\in\mathcal{D}} \sum_{t=1}^{144} g_{d,t}$，其中 $g_{d,t}$ 由式 (8) 在**结算时刻**确定，是" 含电池吞吐项" 的完整缺口，而不是 $\max\{0, \text{实际净负荷} - \text{计划购电量}\}$；**数据**　结算依据为附件 2 的实际负载与实际光伏，计划量与充放电量取 `result2.xlsx` 的对应工作表；**时段**　334 天 × 144 槽（2025-02-01 至 2025-12-31）；**单位**　kWh（表内保留 2 位小数）；**不含**　不含问题 3 调整机制引入的缺口（调整量单独统计，口径 C-09）；**对照**　完美信息 R1 下为 0.0000 kWh（0/334 天）；典型日预报 R2 下为 2 525 548.76 kWh（286/334 天）；**复算**　`code/lp_engine.py::realized_slot`，审计项 A1 与 A9。

可行性复核　逐日检查：计划口径残差（供给缺口）最大 3.41×10^{-13} kWh，储电量净递推残差最大 1.09×10^{-11} kWh，最差单日综合残差 1.82×10^{-12}；同时充放、SOC 越界、功率越界、负购电的时段数均为 0，末值低于初值的天数 0、费用相对复算误差 0.0000；结果文件回读校验的全天购电量最大偏差 0.0011 kWh、全天购电费最大相对偏差 3.40×10^{-7}。

题面指定日期的结果（表 1 / 表 2 / 表 3 格式）　四个指定日期 2025-03-20、2025-06-21、2025-09-23、2025-12-21 的计划购电量（表 13）、储能充放电量与 0:00/24:00 储电量（表 14）、紧急购电量（表 15）见表。四日均满足计划口径供需平衡（最大残差 1.82×10^{-12}）与日周期终端条件，0:00 与 24:00 储电量均为 6000.0000 kWh。

表 13　微网在指定时段与指定日期的购电量、全天购电量和购电费（问题 2；电量单位：kWh，费用单位：元）

时段 / 指标	2025-03-20	2025-06-21	2025-09-23	2025-12-21
10:00–10:10	0.0000	0.0000	0.0000	0.0000
12:00–12:10	804.3029	0.0000	536.2415	997.0335
14:00–14:10	0.0000	0.0000	0.0000	0.0000
16:00–16:10	817.3061	255.2254	902.7969	1001.2240
18:00–18:10	697.9973	0.0000	764.3298	715.4144
20:00–20:10	0.0000	0.0000	0.0000	0.0000
全天购电量	74 178.1298	35 701.5391	74 744.7963	99 692.6601
全天购电费	45 229.90	20 324.26	47 063.39	64 376.98
紧急购电量	1025.8805	187.0843	193.3669	22.5250
全天总费用	49 654.02	21 255.45	47 717.70	64 432.28

表注：时段按题面表述取区间左端时刻；全天购电费为计划口径（$\sum_t p_t E_t$），全天总费用为计划购电费与紧急购电费之和；四个日期的无储能基线分别为 52 290.65、29 668.18、54 741.30、71 176.92 元。

六、问题 3：多阶段滚动调整模型

6.1 多阶段决策结构与费用口径

问题 3 在问题 2 上增加两个要素：0:00、6:00、12:00、18:00 四个时刻均有未来 24 小时整点光伏预报，决策随之**滚动修正**；并引入**两类不对称惩罚**（总购电费用含计划购电费、紧急购电费与调整购电量相关费用）。

沿用第三章的槽位读法：槽位 $t = 0, \ldots, 143$ 表示 $[10t, 10(t+1))$ 分钟，$\Delta t = 1/6$ h；电量单位 kWh、电价元/kWh、费用元。四类决策量为：0:00 制定的**计划购电量** E_t^{plan}（第一阶段承诺）、滚动修正后的**调整购电量** E_t^{adj}、实际负载与光伏决定的**紧急购电量** g_t，以及储能充放电量 c_t, d_t 与时段末储电量 S_t。

表 14　储能设备在指定日期的充放电量及 0:00 与 24:00 的储电量（问题 2，单位：kWh）

日期	时段	充电量	放电量	日期	时段	充电量	放电量
2025-03-20	0:00–4:00	5333.3333	828.0303	2025-09-23	0:00–4:00	5333.3333	790.6233
2025-03-20	4:00–8:00	6500.0000	7811.9697	2025-09-23	4:00–8:00	5833.3333	7778.1258
2025-03-20	8:00–12:00	6905.5931	2218.5304	2025-09-23	8:00–12:00	6728.4081	1606.2615
2025-03-20	12:00–16:00	5333.3333	8640.0000	2025-09-23	12:00–16:00	5333.3333	8640.0000
2025-03-20	16:00–20:00	0.0000	0.0000	2025-09-23	16:00–20:00	0.0000	0.0000
2025-03-20	20:00–24:00	0.0000	0.0000	2025-09-23	20:00–24:00	0.0000	0.0000
2025-06-21	0:00–4:00	0.0000	2723.8246	2025-12-21	0:00–4:00	5333.3333	0.0000
2025-06-21	4:00–8:00	5348.6522	1710.2113	2025-12-21	4:00–8:00	6666.6667	9315.0000
2025-06-21	8:00–12:00	5458.7995	0.0000	2025-12-21	8:00–12:00	8333.3333	2835.0000
2025-06-21	12:00–16:00	5333.3333	8640.0000	2025-12-21	12:00–16:00	5333.3333	8640.0000
2025-06-21	16:00–20:00	0.0000	0.0000	2025-12-21	16:00–20:00	0.0000	0.0000
2025-06-21	20:00–24:00	0.0000	0.0000	2025-12-21	20:00–24:00	0.0000	0.0000

表注：四个指定日期的 0:00 与 24:00 储电量均为 6000.0000 kWh；每段含 36 个 10 min 槽，充放电量为交流侧电量（口径 C-07）；各日充放电量之比均为 1.2346（对应往返效率 0.81）。

表 15　微网在指定日期的紧急购电量（问题 2；购电量单位：kWh，电价单位：元/kWh）

日期	购电时间段	紧急购电量	交易时刻电价（加权平均）
2025-03-20	6:00–8:00	228.9914	0.8625
2025-03-20	8:10–8:30	49.1586	0.8625
2025-03-20	8:50–10:30	531.2739	0.8625
2025-03-20	10:40–11:40	216.4567	0.8625
2025-06-21	4:00–4:50	5.2416	0.9955
2025-06-21	14:00–14:30	181.8427	0.9955
2025-09-23	6:50–7:40	132.5948	0.6768
2025-09-23	11:00–11:40	60.7721	0.6768
2025-12-21	11:00–11:30	22.5250	0.4910
四个日期合计		1428.8567	—

表注：本表按”日期 × 紧急购电事件”逐行展开，与题面表 3 的横向版式一一对应；同一日期可有多行，连续触发的相邻槽按事件聚合，时间段为该事件涉及的槽位区间；交易时刻电价为该事件内按电量加权的平均电价（事件电价按 $5p_t$ 计价，本列为 p_t）。无紧急购电的时段不列入本表（全窗口 334 天中 296 天有触发）。

后覆盖前的决策结构。四个预报时刻把一天切成四段，槽位 t 的最终执行量由**最后一次覆盖它的那次决策**给出（表 16）：0:00 用当次预报制定全天 144 槽计划，此后每个决策时刻只用 **当次预报**对**其后至当日** 24:00 **的剩余时段**重新决策，但**只执行到下一个决策时刻**。

表 16　四次预报的决策时刻、生效槽位与可用预报（时间范围：2025-02-01 至 12-31 的每一天；槽位单位：10 min 槽，共 144 槽；预报来源：附件 3）

决策时刻	决策内容	生效槽位	生效时段	重解范围（只执行到下一时刻）
0:00	计划购电量 E^{plan}（144 槽）	0–35	0:00–6:00	0–143
6:00	调整购电量 E^{adj}	36–71	6:00–12:00	36–143
12:00	调整购电量 E^{adj}	72–107	12:00–18:00	72–143
18:00	调整购电量 E^{adj}	108–143	18:00–24:00	108–143

非预期性在实现上自然满足。非预期性要求任一时刻的决策只依赖该时刻及之前已实现的信息，本文实现有两个环节天然满足，无需显式施加式 (12)：其一，取预报值的函数只按**提前期**索引——槽位 t 在时刻 m 取附件 3 中 m 时刻发布行的第 k 列，$k=(h_t-m) \bmod 24$，故 m 时刻的子问题**在数据结构上取不到**其后发布的列；其二，每个子问题只覆盖 $t \geq s_m = 6m$，已执行槽位被冻结，决策不可能回头改写 $t < s_m$ 的历史。两条合起来即**前视在实现层面被排除**。

为可证伪地验证这一点，本文做了**预报置换检验**（`code/exp_na.py`，输出 `intermediate/exp_na.json`）：把某一次预报在 12 天之间随机置换后重解，记录四个控制区（表 16 的四段槽位，分别由 0:00、6:00、12:00、18:00 的决策控制）内决策量的最大变化。结果是：置换 0:00 预报会显著改变全天决策（说明检验本身有效力），而**任何一次置换都不改变”早于该预报发布时刻”的槽位决策**（该区变化恰为 0.000 kWh），故实现中不存在前视。置换 18:00 预报时四个控制区变化全部为 0.000 kWh，这是数据事实而非检验失效：附件 3 中 18:00 预报在其覆盖的槽 108–143 上几乎恒为 0（19:00 以后全年恒为 0，18:00–19:00 仅 20/365 天非零、最大 8.7375 kW），对晚间决策本无新信息，其信息价值贡献很小（见 §6.5）。四个控制区的完整 4×4 变化矩阵见第九章”非预期性测试”。

口径 C-09　调整与违约费用

定义　$c_t^{\mathrm{buy}} = p_t \min\big(E_t^{\mathrm{plan}}, E_t^{\mathrm{adj}}\big) + 0.5\,p_t\big(E_t^{\mathrm{plan}} - E_t^{\mathrm{adj}}\big)^+ + 1.5\,p_t\big(E_t^{\mathrm{adj}} - E_t^{\mathrm{plan}}\big)^+$（**偏差量定价**：计划与调整中较小者按正常电价 p_t 结算，调减部分按 $0.5p_t$ 计违约、调增部分按 $1.5p_t$ 计超额），缺口另按 $5p_t$ 紧急购电；**数据**　电价取附件 1（问题 3）或附件 4（问题 4-3）；结算基准恒为 0:00 的计划量 E^{plan}（假设 A7），不随调整次数漂移；**时段**　2025-02-01 至 12-31 共 334 天，逐日 144 槽；**单位**　元（保留 2 位小数，内部计算不取整）；**不含**　不含储能投资与运维成本、容量补偿、辅助服务收益与绿电环

境价值；**对照**　另一读法”计划购电量按全额结算 + 差额惩罚”与本文口径的差额恒等于 $\sum_t p_t v_t = 421\,349.41$ 元（违约量按 p_t 与 $0.5p_t$ 计价之差），该读法下调整机制不再产生净收益，对照见第九章；**复算**　`q3.py` 的 cost_buy、cost_breach、cost_excess、cost_emg 四项之和等于 total_cost，且 $\sum_t p_t v_t = 2\times$cost_breach 恒成立。

分段线性凸的偏差费用。 c_t^{buy} 关于 E_t^{adj} 是**凸的分段线性函数**：计划不高于调整时（$E_t^{\text{adj}} \ge E_t^{\text{plan}}$）斜率为 $1.5p_t$，计划高于调整时（$E_t^{\text{adj}} < E_t^{\text{plan}}$）斜率为 $0.5p_t$，故式 (10) 成立：

$$c_t^{\text{buy}} = p_t \min\big(E_t^{\text{plan}}, E_t^{\text{adj}}\big) + 0.5\,p_t\big(E_t^{\text{plan}} - E_t^{\text{adj}}\big)^+ + 1.5\,p_t\big(E_t^{\text{adj}} - E_t^{\text{plan}}\big)^+ \tag{10}$$

凸性意味着**偏差费用不惩罚”偏离”本身，只惩罚偏离的方向**：调减按半价结算差额、调增按 1.5 倍结算，这正是题面”两类惩罚”的数学形式。

6.2　分段线性凸成本的线性化

引入非负辅助变量 $u_t, v_t \ge 0$（u_t 表示超额量，v_t 表示违约量），令其满足

$$E_t^{\text{adj}} - E_t^{\text{plan}} = u_t - v_t, \qquad u_t \ge 0, \quad v_t \ge 0 \tag{11a}$$

$$c_t^{\text{buy}} = p_t E_t^{\text{plan}} + 1.5\,p_t u_t - 0.5\,p_t v_t \tag{11b}$$

其中式 (11b) 由恒等式 $p_t \min(E_t^{\text{plan}}, E_t^{\text{adj}}) = p_t E_t^{\text{plan}} - p_t v_t$ 直接得到。

式 (11) 的线性化是**精确的**，无需引入 0–1 变量：由式 (11a)–(11b)，费用中与 (u_t, v_t) 有关的部分为 $1.5p_t u_t - 0.5p_t v_t$，而 u_t, v_t 同时为正时两边同时减去 $\min(u_t, v_t)$ 可使目标下降 $p_t \min(u_t, v_t) > 0$，故**最优解中必然有** $\min(u_t, v_t) = 0$，式 (10) 与该线性化完全等价。互斥性的关键是两条分段斜率跨越基准电价 $1 \times p_t$（$0.5 < 1 < 1.5$），使代入 $d_t = E_t^{\text{adj}} - E_t^{\text{plan}}$ 后 u_t 与 v_t 的有效系数都为正，最优解只能落在折点上。**适用边界**：若违约倍数取到 1 及以上（本题为 0.5），折点两侧斜率不再跨过基准电价，互斥性不再自动成立，必须引入 0–1 变量；本文两次用到同一互补松弛手法（另一次是储能充放电互斥），以保持 LP 的精确可解性。

数值上，334 天 × 144 槽 = 48 096 个”槽-日”中 $\min(u_t, v_t) > 10^{-6}$ kWh 的违反数为 **0**，最大同时正偏差 0.000000 kWh。

6.3　场景树与非预期性约束

以四个决策时刻为分支点构造**场景树**（根节点 0:00、子节点 6:00 的预报实现，依此类推），记 $\tau(t)$ 为不晚于槽位 t 的最后一个决策时刻，四个时刻依次控制表 16 的四段槽位。

第一个决策时刻 0:00 的计划量对所有场景相同；槽位 t 的调整量只能依赖**截至** $\tau(t)$ **的信息**。设 $\omega|_{[0,\tau]}$ 表示场景 ω 在时刻 τ 之前的历史，则非预期性约束为

$$E_t^{\text{adj}}(\omega) = E_t^{\text{adj}}(\omega'), \qquad \forall\, \omega, \omega' \in \Omega:\ \ \omega|_{[0,\tau(t)]} = \omega'|_{[0,\tau(t)]} \tag{12}$$

式 (12) 展开成 LP 等价于按场景树中**深度为** $\tau(t)$ **的节点**把场景分组、对同组场景施加变量相等的线性约束。本文不显式展开场景树，而采用与它在确定性等价下一致的滚动实现（§6.4）：变量规模与场景数解耦，非预期性则由”数据可取范围 + 已执行槽位冻结”自动满足。

6.4　多阶段滚动模型

记 Ω_m 为时刻 m 用于剩余时段的场景集（权重 w_ω；按”日总预报残差”分 60 层、层内取 medoid 代表、权重取层样本占比生成），$s_m = 6m$ 为该时刻起始槽位。综合式 (10) –(12)，问题 3 的多阶段模型为

$$\min \ \sum_{t=0}^{143} p_t E_t^{\text{plan}} + \sum_{\omega\in\Omega} w_\omega \sum_{t=0}^{143} \Big[1.5\,p_t\,u_t(\omega) - 0.5\,p_t\,v_t(\omega) + 5\,p_t\,g_t(\omega)\Big] \tag{13a}$$

$$E_t^{\text{adj}}(\omega) - E_t^{\text{plan}} = u_t(\omega) - v_t(\omega), \qquad u_t(\omega), v_t(\omega) \ge 0 \tag{13b}$$

$$E_t^{\text{plan}} - c_t + d_t \ \ge\ \big(L_t - \widehat{PV}_t^{(m)}\big)\Delta t \qquad \text{（计划口径可行性，预报取当次发布的附件 3 数据）} \tag{13c}$$

$$g_t(\omega) \ \ge\ \big(L_{\omega,t} - PV_{\omega,t}\big)\Delta t + c_t - d_t - E_t^{\text{adj}}(\omega), \qquad g_t(\omega) \ge 0 \tag{13d}$$

$$S_t = S_{t-1} + \eta_c c_t - \frac{d_t}{\eta_d}, \quad \underline{S} \le S_t \le \overline{S}, \quad 0 \le c_t, d_t \le \overline{P}\Delta t, \quad S_0 = S^{init},\ S_{143} \ge S_0 \tag{13e}$$

$$E_t^{\text{plan}} \text{ 与 } \omega \text{ 无关}; \qquad E_t^{\text{adj}} \text{ 满足式 (12)} \tag{13f}$$

模型中 $L_t, PV_{\omega,t}$ 为实际负载与实际光伏功率（kW），$L_t - \widehat{PV}_t^{(m)}$ 为当次预报口径的净需求功率，乘 $\Delta t = 1/6$ h 后为电量；参数取 $\eta_c = \eta_d = 0.9$、$\overline{P}\Delta t = 833.3333$ kWh、$\underline{S} = 1200$ kWh、$\overline{S} = 10800$ kWh、

$S^{init} = 6000$ kWh。式 (13d) 的缺口以**调整后**的 $E_t^{\mathrm{adj}}(\omega)$ 为基准（假设 A7），式 (13e) 的终端条件 $S_{143} \geq S_0$ 防止滚动优化在窗口末端放空储能换取短期收益（假设 A6）。

滚动时域（MPC）实现。式 (13) 的场景树 LP 按决策时刻分解为四个子问题：在时刻 m 以当前储电量 S_{s_m-1} 为初值、当日初值 S_0 为终端下界，对槽位 $t \geq s_m$ 求解

$$\min \sum_{t=s_m}^{143} \Big[(1-\delta_m)\, p_t E_t + \delta_m \big(1.5\, p_t u_t - 0.5\, p_t v_t\big)\Big] + \sum_{\omega \in \Omega_m} w_\omega \sum_{t=s_m}^{143} 5\, p_t\, g_{t,\omega}, \qquad \delta_m = \begin{cases} 0, & m = 0 \\ 1, & m > 0 \end{cases} \tag{14}$$

约束为式 (13b)–(13e) 在剩余槽位上的限制；$m > 0$ 时计划项已是常数故从目标中略去，只保留偏差项与紧急购电项，并由式 (13b) 与 0:00 的计划量耦合。式 (14) 中”重解范围”与”执行范围”分离：求解后**只把** $[s_m, s_{m+1})$ **上的决策写入实际轨迹**（购电量与充放电量、储电量轨迹一并执行），其余解丢弃，等下一个预报时刻用更新的预报重优化——充放电量因此随信息滚动修正，而非冻结在 0:00。槽位 t 的最终执行量由 $\tau(t)$ 对应的决策给出（表 16）。全部物理约束用结果文件独立复算：供给缺口残差 $\leq 3.5 \times 10^{-13}$ kWh、储电量递推残差 $\leq 1.1 \times 10^{-11}$ kWh、储电量与功率越界数均为 0，334 天的日末储电量均不低于日初值；每个结果日重解 4 次子问题（共 $334 \times 4 = 1336$ 次，另加 2025 年 1 月预热期），每次均为 LP，用 SciPy 的 `linprog(method="highs")` 精确求解。

6.5 信息价值：是否需要引入其他时刻的预报

题面直接追问”是否需要引入其他时刻的预报制定调整购电策略”，故必须给出**量化答案**与**成本侧**。本文设计**预报时刻数对照实验**：$n \in \{1, 2, 4\}$ 三组使用**同一场景集、同一实际轨迹回放、同一费用口径**（口径 C-09），只有”计划时能用到几次预报”不同——$n = 1$ 只用 0:00 预报（即问题 2 的结构）、$n = 2$ 用 0:00 与 12:00、$n = 4$ 为题面给定的四次预报。附件 3 只提供这四个发布时刻，故 $n = 8$、$n = 24$ 在本数据下**并不存在**：任何加密实验都须自行合成预报，其结论取决于合成规则而非数据。本文不做这种合成，而用**完全信息上界**代替：0:00 计划仍按 0:00 预报制定，但从 6:00 起的调整假定已知当日实际净需求，故其费用**不高于**任何有限预报时刻数配置，是”还能再省多少”的**与数据无关的上界**。

$$\mathrm{VoI}(n) = \frac{C(1) - C(n)}{C(1)} \times 100\% \tag{15}$$

其中 $C(n)$ 为可用 n 个预报时刻时、在同一场景集与同一实际轨迹下的总费用。

口径 C-10　信息价值（VoI）

定义　$\mathrm{VoI}(n) = \big(C(1) - C(n)\big)/C(1) \times 100\%$；同时报”已实现的可获得信息价值”$\big(C(1) - C(n)\big)/\big(C(1) - C_{\mathrm{PI}}\big)$，分母 $C(1) - C_{\mathrm{PI}}$ 是完全信息上界给出的**可节省总额**；**数据**　四组运行（$n = 1$、$n = 2$、$n = 4$、完全信息），同一场景集（每时刻 60 个分层 medoid 代表场景）与同一实际轨迹（附件 2 实际负载与实际光伏）；**时段**　2025-02-01 至 12-31 共 334 天；**单位**　元（费用）、%（相对量）；**不含**　不含预报的获取成本（通信、预测与运维），故 VoI 只回答收益侧，成本侧必须以”盈亏平衡”的形式单独给出；**对照**　以完全信息上界为可获得信息价值的上限，恒有 $\mathrm{VoI}(n) \leq \mathrm{VoI}(\mathrm{PI})$；**复算**　`q3.py` 四组运行的 total_cost 之差，分母取 base_cost 或完全信息费用。

结果见表 17 与图 6。

表 17　预报时刻数对照实验：总费用、边际节省与信息价值（同一场景集、同一实际轨迹、同一偏差量定价口径 口径 C-09；时间范围：2025-02-01 至 12-31 共 334 天；样本：60 个分层 medoid 代表场景 × 4 组配置；单位：费用元，电量 kWh）

可用预报时刻	总费用/元	边际节省/元	累计 VoI	已实现占比	紧急购电量/kWh
仅 0:00（$n = 1$）	14 264 792.13	—	0	0	172 015.82
0:00、12:00（$n = 2$）	14 131 564.16	133 227.98	0.93%	34.1%	122 524.83
四时刻（$n = 4$，题面给定）	13 932 402.95	199 161.20	2.33%	85.1%	64 522.09
完全信息（可获信息上界）	13 874 340.39	58 062.56	2.74%	100%	6 134.76

”已实现占比”一列的分母 $C(1) - C_{\mathrm{PI}} = 390\,451.74$ 元是完全信息下**最多能省**的钱。

表 17 与图 6 显示：滚动调整**有效**（总费用降 2.33%、紧急购电量降 62.49%），收益**边际递减**（$1 \to 2$ 次加密省 133 227.98 元、$2 \to 4$ 次省 199 161.20 元，四次预报已实现可获得信息价值的 **85.1%**），而**再增加预报最多只能再省** 58 062.56 **元**（占四次预报总费用的 0.42%）。

边际增益来自哪一次预报。 199 161.20 元不是三次预报的平均贡献：由 §6.1 的置换检验，18:00 预报在其控制的槽上几乎恒为 0，置换它不改变任何决策，故 $n = 2 \to n = 4$ 的增益**主要来自** 6:00 **对上午时段的更新**——上午正是 0:00 预报系统性偏离最大的时段（见第三章）与紧急购电最集中的时段。故若有额外投入，应加在**上午的预报**上，而非简单增加发布次数。

成本侧比较与结论。把收益上界 58 062.56 元摊到 334 天，等价于”每天新增预报投入不超过 173.84 元”；只要每新增一个预报时刻的日均获取成本（通信、数值天气预报接口、预测与运维人力）超

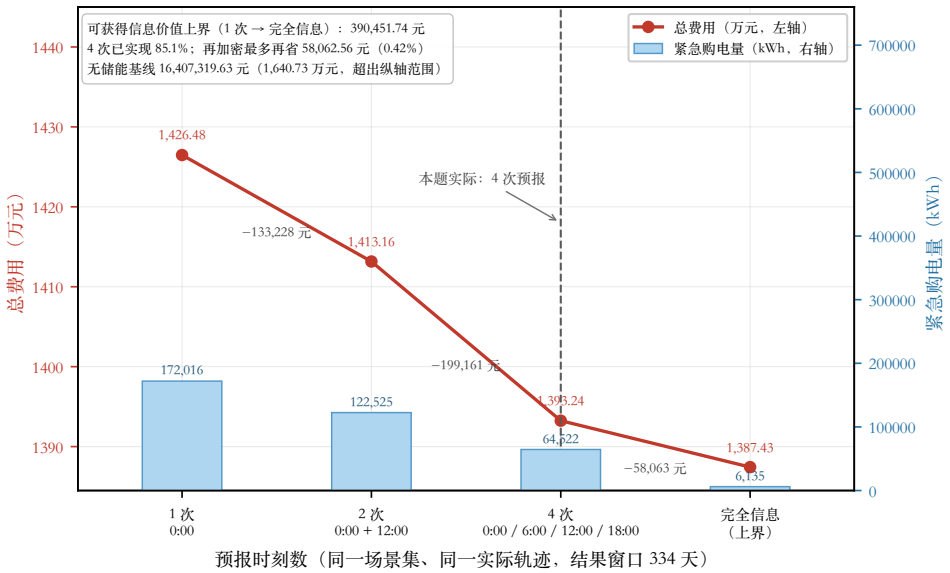


图 6　信息价值曲线（对象：预报时刻数 $n = 1/2/4$ 与完全信息上界的四组配置，竖直虚线标出题面给定的 $n = 4$；范围：2025-02-01 至 12-31 共 334 天，四组使用同一场景集与同一实际轨迹；单位：左轴总费用万元、右轴紧急购电量 kWh，相邻档位间标注边际节省元；样本：每组 60 个分层 medoid 代表场景）

过这个量级，加密预报在本题参数下就是**亏本**的。故本文的回答是：**四次预报已经足够，在预报获取成本可以忽略之前不建议继续加密预报时刻**；若要追加投入，应优先投向**提高现有四次预报的精度**——相同时刻数下剩余误差仍对应 58 062.56 元（0.42%），只能靠精度改进消除（见 （见 C-Q3-01））。**适用边界**：①只在本题惩罚倍数（违约 $0.5p_t$、超额 $1.5p_t$、紧急 $5p_t$）下成立；②只在附件 3 的预报质量下成立，预报误差成倍减小会使边际收益更早饱和；③ $n = 8$、$n = 24$ 为数据中不存在的配置，以其上界（完全信息）代替，故” 不建议加密” 是对**收益侧上界**的判断。

实现正确性自检。在同一场景集、同一口径下同时计算四组费用，可验证不等式链

$$C_{\mathrm{PI}} \;\le\; C_{n=4} \;\le\; C_{n=1}, \tag{16}$$

代入数值为 13 874 340.39 ≤ 13 932 402.95 ≤ 14 264 792.13（单位：元）；$n = 1$ 即” 后续不再调整” 的开环情形，链条正确退化为三段。若数值违反该链，说明非预期性、信息集或结算口径实现有误，须先修正再解读 VoI。

6.6　求解结果

在同一场景集上求解式 (13)，四项费用为：计划相关 13 216 203.48 元、违约 210 674.71 元、超额 262 628.79 元、紧急购电 242 895.98 元，合计 13 932 402.95 元（1 393.24 万元），相对无储能基线 16 407 319.63 元省 15.08%；紧急购电量 64 522.09 kWh，出现在 302/334 天（口径 C-09）。

调整机制省在哪里。与” 只按 0:00 计划结算” 的问题 2 结构（计划购电费 13 637 552.90 元 + 紧急购电费 627 239.24 元）相比，调整机制把**紧急购电费压低** 384 343.26 **元**，代价是计划相关费用净增 51 954.08 元（计划项减少 421 349.41 元、违约与超额合计增加 473 303.50 元），相抵净省 332 389.18 元。

四次预报下 0:00 计划购电量合计 22 220 333.20 kWh，调整后 21 843 259.36 kWh（净减 377 073.84 kWh），其中调减 645 614.42 kWh、调增 268 540.58 kWh，两者之差恰为净减少量（口径自检）。而 0:00 计划量在四种信息结构下 **完全相同**，信息价值的全部来源都在**调整**环节。

与问题 2 的对照。两者同为附件 1 的固定电价、同一场景集、同一实际轨迹与同一费用口径，唯一差别是三次滚动预报是否可用；费用差额见表 17（§6.5）：**节省 332 389.18 元（2.33%）**，相对无储能基线的节省率由 13.06% 提高到 15.08%。该降幅**依赖于 口径 C-09 的偏差量定价口径**：在” 计划购电量全额结算 + 差额惩罚” 的读法下，计划项不再随调整量下降，净收益会减少 $\sum_t p_t v_t$ = 421 349.41 元并转为负值（第九章给出对照），故” 是否需要引入其他时刻预报” 的答案与市场规则（惩罚倍数与结算方式）绑定，不能脱离口径单独引用。按题面表 1、表 2、表 3 格式给出的指定日期结果见第八章表 22，其中” 调整购电量” 一列填的是**最后一次覆盖该槽位的决策给出的购电量**（表 16），与 0:00 计划量之差即该槽的偏差量。

七、问题 4：波动电价下的风险决策模型

7.1 电价过程的统计检验

问题 4 中电价与光伏出力同时随机；若把附件 4 的电价序列直接当作” 另一组已知价格” 代入模型，等于假设 0:00 就知道全天的实际电价，与题意矛盾。故本文先把电价当作**随机过程**检验并建模（表 36）：检验在 2025 年全年（52 560 个观测）上进行，费用结果仍只在 334 天上报告。

四项检验的建模含义见表 36 末列。其中最需强调的是 T4：电价与光伏的池化相关 +0.028 几乎为 0——这只是” 夜间光伏恒为 0” 稀释后的假象，日聚合尺度上的真实结构是 −0.343；若据此独立抽样，就会生成” 午间光伏大发、同时电价高企” 这类不可能情景并**低估尾部风险**（图 7）。

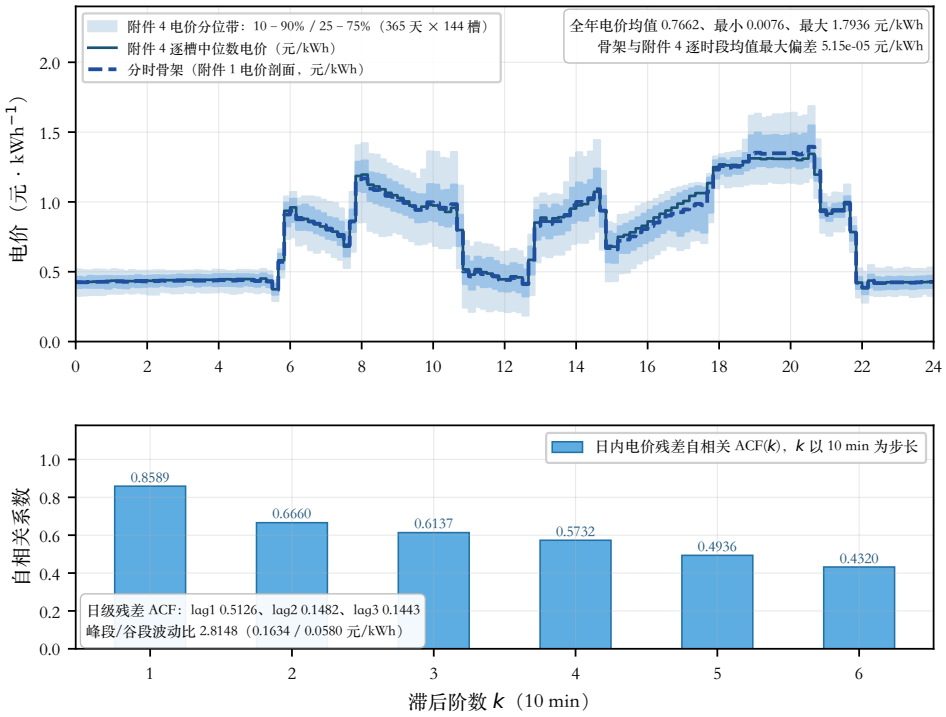


图 7　电价过程的骨架与残差结构（上子图：附件 4 电价的逐槽分位带 10%–90% 与 25%–75%、逐槽中位数与附件 1 分时骨架；下子图：日内残差自相关 ACF(1–6)，并标注日级残差自相关与峰谷波动比；范围：2025-01-01 至 12-31 共 365 天 × 144 槽；单位：电价元/kWh，自相关为无量纲量；样本：$n = 52\,560$ 个电价观测）

7.2 两层电价模型与联合情景生成

本文用**两层结构**刻画电价：第一层是可预测的**分时骨架** $\bar{p}_t$（取附件 1 的电价剖面，全年尺度上等于附件 4 的逐时段均值，最大偏差 5.15×10^{-5} 元/kWh），第二层是不可预测的 **随机残差** $\varepsilon_{d,t}$：

$$p_{d,t} = \bar{p}_t + \varepsilon_{d,t}, \qquad \mathbb{E}[\varepsilon_{d,t}] = 0 \tag{17}$$

其中 d 为日期、t 为槽位，$p_{d,t}$ 的单位为元/kWh。按 §7.1 的 T3，残差须 **按时刻对齐**重采样，故本文直接以**整日**为单位构造情景。

联合情景：二维分层取代表历史日。设 $r_{d,t}$ 为第 t 槽的净需求预报残差（0:00 预报口径的净需求与实际净需求之差，单位 kWh），p_d 为该日平均电价。把 334 天按 $(\sum_t r_{d,t},\ p_d)$ 各分 8 层形成 8×8 网格，每格取 **medoid**（层内欧氏距离和最小的历史日）为代表，权重取层内样本占比。所得 64 个代表日（权重 0.0030–0.0299，合计 1）保真两个维度的边缘分布，并因电价剖面与净需求残差来自**同一个历史日**而**天然保留联合结构**。

情景保真度见表 33。作为对照，本文另用**分块 Bootstrap**（6 h 块、按时刻对齐）生成 64 个合成情景：优势是不依赖历史日的可获取性，但本题样本量下边缘保真度明显更差（均值偏差 3.41%、1% 分位相对差 20.4%），故**只作为对照**，主口径采用代表历史日。

必须保留 $\rho(p, L)$ 的理由。电价与净负荷的正相关（池化 +0.260）意味着” 贵的时候恰好也在多买电”；若独立抽样，” 高价 × 高净需求” 的联合尾部占比会被人为压低，风险度量随之失去意义。二维网格取代表日后 $\rho(p, \text{净负荷}) = 0.2597$，与实际 0.2620 相差 0.0023，尾部结构被保真。

波动电价下的 LP 形式与双重不确定性。设 Ω 为联合情景集（权重 w_ω），$r_{\omega,t}$ 为情景 ω 的净需求残

差，$\widehat{PV}_t^{(0)}$ 为 0:00 发布的光伏预报，$p_{\omega,t}$ 为情景电价，则重算问题 2 的两阶段 LP 为

$$\min \ \sum_{t=0}^{143} \bar{p}_t^{\mathrm{sc}} E_t + \sum_{\omega\in\Omega} w_\omega \sum_{t=0}^{143} 5\, p_{\omega,t}\, g_{\omega,t}, \qquad \bar{p}_t^{\mathrm{sc}} = \sum_{\omega\in\Omega} w_\omega p_{\omega,t} \tag{18a}$$

$$E_t - c_t + d_t \ \geq \ \big(L_t - \widehat{PV}_t^{(0)}\big)\Delta t \tag{18b}$$

$$g_{\omega,t} \ \geq \ \big(L_t - \widehat{PV}_t^{(0)}\big)\Delta t + r_{\omega,t} + c_t - d_t - E_t, \qquad g_{\omega,t} \geq 0 \tag{18c}$$

约束外加式 (13e) 的储电量递推、界与终端条件（物理参数同 §6.4）。两个不确定性分别进入 **目标系数**（电价 $p_{\omega,t}$）与**约束右端项**（净需求残差 $r_{\omega,t}$），即问题 4 相对问题 2 的**双重不确定性**。决策时的电价信息取情景概率加权均值 $\bar{p}_t^{\mathrm{sc}}$：附件 4 未提供日前电价预报，故 0:00 可用的电价预报取**分时骨架** $\bar{p}_t$（假设 A10），$\bar{p}_t^{\mathrm{sc}}$ 与骨架的最大偏差 0.0462 元/kWh（占均价 6.02%）。**结算一律按附件 4 的当日实际电价**，使”决策用预报、结算用实现”各归其位（口径 C-03）。问题 4-3 在式 (18) 上再叠加 6:00/12:00/18:00 三次滚动修正，调整与违约费用按 口径 C-09 的偏差量定价结算。

7.3 阈值律在波动电价下的形式不变性

问题 1 的阈值律只依赖①目标对购电量线性、②储电量递推两条结构性质，**完全不依赖电价的分布假设**，故电价由确定变随机只改变阈值（水价值 ν_t）的**取值**，不改变”电价低于充电阈值则充、高于放电阈值则放”的**函数形式**：$\alpha_t \leq \eta_c\nu_t$ 时充电、$\alpha_t \geq \nu_t/\eta_d$ 时放电、$\eta_c\nu_t < \alpha_t < \nu_t/\eta_d$ 时既不充也不放（死区，两侧之比 $1/(\eta_c\eta_d) = 1.2346$）。两点边界：①该结论**在储电量界与充放功率界不活跃时精确成立**，界活跃时须回到互补松弛式 (3a)–(3b) 并计入界乘子；②”形式不变”不等于”阈值是常数”——波动电价下 ν_t 仍须**逐时段由场景对偶确定**，随情景集与风险参数 λ 变化（(见 C-Q4-01)）。

7.4 风险—费用有效前沿

CVaR 的线性化。记 C_ω 为情景 ω 的总购电费用、$\alpha = 0.95$ 为锁定置信水平，条件风险价值（Rockafellar–Uryasev 形式）为

$$\mathrm{CVaR}_\alpha(C) = \min_{\eta}\Big\{\eta + \frac{1}{(1-\alpha)|\Omega|}\sum_{\omega\in\Omega}\max\big(0,\, C_\omega - \eta\big)\Big\}, \tag{19}$$

引入辅助变量 $z_\omega \geq C_\omega - \eta$、$z_\omega \geq 0$ 后，式 (19) 的分段线性结构完全由线性约束表达，**加入风险度量后模型仍是 LP**，不引入整数变量。

口径 C-11　条件风险价值 CVaR（$\alpha = 0.95$）

定义　式 (19) 的 $\mathrm{CVaR}_{0.95}$，即最差 5% 情景下费用的条件期望，按 Rockafellar–Uryasev 线性化求解；目标函数取凸组合 $\min\ (1-\lambda)\,\mathbb{E}[C] + \lambda\,\mathrm{CVaR}_{0.95}$，$\lambda \in [0,1]$（$\lambda = 0$ 为纯期望最优、$\lambda = 1$ 为纯风险最优，与”$\min \mathbb{E}[C] + \lambda'\mathrm{CVaR}$”式在 $\lambda' = \lambda/(1-\lambda)$ 下等价）；除假设 A10 外，模型、情景集与物理参数在扫描中保持不变；**数据**　联合情景集（8×8 二维分层得到的 64 个代表历史日，权重为层占比）；评估用 2025-02-01 至 12-31 中**每 6 天取 1 天**的 56 天子样本，逐日独立求解；**时段**　上述 56 个代表日，单日 144 槽；**单位**　元/日（日费用的样本均值）、kWh/日（电量）；**不含**　不含储能投资与运维成本；也不含”最坏情形”（$\min\max_\omega C_\omega$）口径，故 $\lambda = 1$ 只是**近似最坏**；**对照**　每一点同时给出四种量：情景内期望费用、情景内 $\mathrm{CVaR}_{0.95}$、**真实轨迹回测**（在同样的代表日上用附件 4 实际电价与附件 2 实际负载、光伏重算的日费用）与紧急购电量；**复算**　`q4.py --mode cvar --K 64 --step 6`，输出 `intermediate/q4_cvar.json` 的 frontier 列表。

以 λ（$0, 0.1, \ldots, 1.0$ 共 11 档）为参数扫描得到**风险—费用有效前沿**（表 37、图 8），两端极端解为 $\lambda = 0$ 的纯期望最优与 $\lambda = 1$ 的纯风险最优，后者以 0.99% 的期望费用换取 3.19% 的 CVaR 下降。

推荐点的判据事先锁定：在”情景内期望费用相对 $\lambda = 0$ 上升不超过 0.5%”（门限 42 205.57 元/日）下使 $\mathrm{CVaR}_{0.95}$ 最小。满足该门限的最大档位是 $\lambda^* = 0.3$：期望费用 42 168.43 元/日（+0.41%）、$\mathrm{CVaR}_{0.95}$ 50 814.83 元/日（−2.61%）、**真实轨迹回测费用 45 778.03 元/日（仅** +0.18%），紧急购电量由 480.19 降到 324.60 kWh/日（−32.40%）。

前沿的拐点。 λ 从 0.7 升到 0.9 时 $\mathrm{CVaR}_{0.95}$ 只再降 11.53 元/日（0.02%），期望费用却继续上升 34.27 元/日（0.08%）；$\lambda = 1$ 的 $\mathrm{CVaR}_{0.95}$ 与 $\lambda = 0.9$ 基本持平（50 509.78 对 50 510.46 元/日，差 0.68 元/日，属噪声量级）。即$\lambda \approx 0.7$ **以后风险已基本降不动**。真实轨迹回测：三个代表点在**实际发生**的 334 天轨迹口径下日费用为 45 694.27–45 952.85 元，情景内的排序在真实轨迹上同样成立但幅度小得多（+0.57% vs +0.99%）。

7.5 求解结果

在波动电价下重解问题 2 与问题 3（表 41），两问的费用口径与固定电价下**完全相同**（口径 C-03、口径 C-09），差别只在电价序列取附件 4 实际值、决策电价信息取情景加权均值，故两组结果可直接比较。

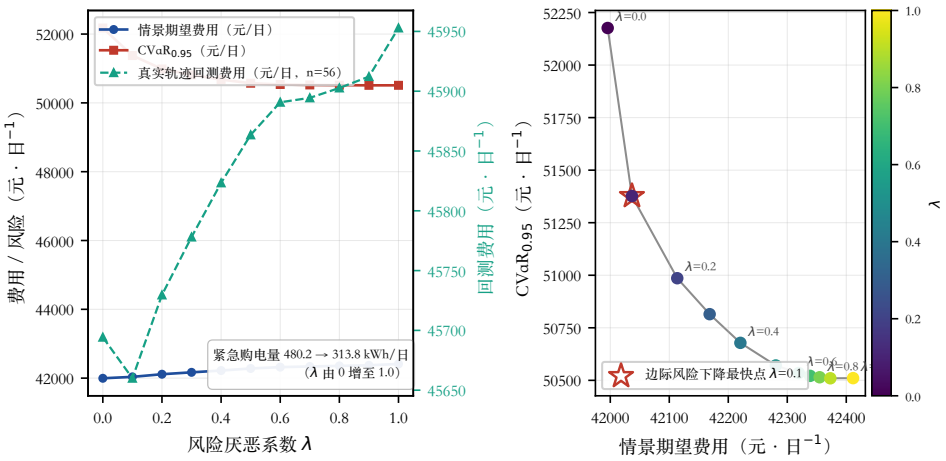


图 8　风险—费用有效前沿与真实轨迹回测（左子图：情景期望费用、$\mathrm{CVaR}_{0.95}$ 与真实轨迹回测费用随风险厌恶系数 λ 的变化；右子图：期望费用—$\mathrm{CVaR}_{0.95}$ 前沿散点，颜色表示 λ，星号标出边际风险下降最快的档位；对象：$\lambda \in \{0, 0.1, \ldots, 1.0\}$ 共 11 档的解；范围：2025-02-01 至 12-31 中每 6 天取 1 天的 56 个代表日；单位：费用元/日；样本：64 个代表情景、11 个 λ 档位，$\alpha = 0.95$）

电价波动把费用抬高了多少。同一 334 天窗口、同一实际负载与光伏轨迹下，波动价下两阶段与四次滚动结构的费用分别高出 5.81% 与 5.67%，无储能基线高出 3.78%，节省率因此由 13.06%/15.08% 降到 11.35%/13.54%（分别下降 1.71 与 1.55 个百分点；表 41）。波动的成本上升**不能由价格水平解释**：窗口内附件 4 电价均值 0.7575 元/kWh 反而比方时骨架的 0.7662 元/kWh 低 1.13%，基线费用仍高 3.78%——差额来自**高电价与高净负荷同时出现**（池化相关 +0.260，见 §7.2），这正是”必须生成联合情景”的定量证据。

波动电价下的滚动调整依然有效。四次预报把波动价下的费用降低 372 102.66 元（2.47%），紧急购电量由 182 759.86 kWh 降到 63 523.11 kWh（降 65.24%），降幅大于固定价下的 2.33%——即**电价越不确定，滚动更新的价值越大**。问题 4-3 的四项费用为：计划相关 13 979 692.49 元、违约 224 406.57 元、超额 270 998.49 元、紧急购电 247 055.63 元，合计 14 722 153.17 元（1 472.22 万元），其中违约电量 664 243.01 kWh、超额电量 259 710.27 kWh；违约与超额费用同时高于固定价，说明**价格不确定使”调整到多少”本身也变得更难判断**。四次滚动下波动价与固定价的紧急购电量几乎相同（反而少 998.98 kWh，−1.55%），而两阶段结构下波动价多买 10 744.04 kWh（+6.25%），即**滚动修正几乎完全吸收了电价波动带来的额外缺口**。四个指定日期的结果见第八章表 22 与表 21（逐时段完整结果见 `result4-2.xlsx`、`result4-3.xlsx`）。

八、结果与分析

本节给出四问的数值结果与解释;数字取自 `intermediate/paper_numbers.json`（`code/paper_numbers.py` 汇总），口径由 `code/config.py` 唯一确定，可由 `code/audit.py` 独立复算（见第九章）。

8.1　指标口径表

本节把在本题参数范围内需要逐项说明的六个核心指标口径一次列清，其余引用其定义章：无储能基线 口径 C-01（第四章，V1 固定价窗口 / V2 波动价窗口 / V3 典型日）、紧急购电量 口径 C-05（第五章）、调整与违约费用 口径 C-09、信息价值 口径 C-10（第六章）、条件风险价值 口径 C-11（第七章）。

口径 C-02　全天购电量

定义　$E^{\mathrm{day}} = \sum_{t=1}^{144} E_t$，即该日 144 个 10 min 槽的**计划购电量**之和；**公式**　结果为逐槽求和，无闭式；结果文件每行末列即该日汇总；**数据**　结果文件「计划购电量」工作表的 144 个时段列；**时段**　单日；问题 1 为典型日，问题 2–4 为 2025-02-01 至 12-31 的 334 天；**单位**　kWh（结果文件保留 4 位小数，正文保留 2–4 位）；**含**　问题 2、4-2 中即第一阶段计划量；问题 3、4-3 中调整后的量一律称”全天调整购电量”并单独命名；**不含**　不含储能充电量、不含紧急购电量 口径 C-05、不含盈余处置量 口径 C-06；**对照**　问题 2、3 的计划购电量模型内值为 22 220 333.2033 kWh，结果文件回读值为 22 220 333.1391 kWh，相对差 2.9×10^{-9}，来自写入时的 4 位小数取整；**复算**　`results_io.py::plan_sheet_frame()`（写出行汇总列）与 `audit.py::audit_result_files()`（回读结果文件后重算 $\sum_{144}$ 槽）

口径 C-03　总购电费用（计划 + 紧急 + 调整相关）

定义　$C^{\mathrm{total}} = C^{\mathrm{plan}} + C^{\mathrm{breach}} + C^{\mathrm{excess}} + C^{\mathrm{emg}}$；问题 2、4-2 无调整机制，后两项恒为 0；**公式**　问题 2、4-2：$C = \sum_t p_t E_t + 5\sum_t p_t g_t$；问题 3、4-3：$C^{\mathrm{plan}}, C^{\mathrm{breach}}, C^{\mathrm{excess}}$ 按 口径 C-09 的偏差量定价口径计算，$C^{\mathrm{emg}} = \sum_t 5 p_t g_t$；**数据**　电价取附件 1（问题 1–3）或附件 4（问题 4）；电量取本队结果文件；**时段**　问题 1 为典型日；问题 2–4 为 334 天；**单位**　元（保留 2 位小数，内部计算不取整）；**不含**　不含储能投资与运维成本、容量补偿、辅助服务收益、绿电环境价值、循环衰减与安全成本；**对照**

若只报计划购电费而不计紧急购电，问题 2 为 13 637 552.90 元，比 C^{total} 低 627 239.24 元（4.40%）；**复算** `q2.py::run_q2()`、`q3.py::summary()`、`q4.py::run_q4_2()` 与 `audit.py::audit_result_files()`，四项之和与总费用的偏差为 0

口径 C-04　节省率

定义　$\eta = \left(C^{base} - C^{\text{total}}\right)/C^{base} \times 100\%$，分母与分子必须同窗口、同电价口径；**公式**　C^{base} 取 口径 C-01 的对应变体（V1 固定价窗口 / V2 波动价窗口 / V3 典型日）；**数据**　C^{base} 与 C^{total} 均取自本队结果；**时段**　与 口径 C-01 变体一致；**单位**　%（保留 2 位小数）；**不含**　不得相对贪心策略或完美信息下界报同一个数而不加限定；相对其他基线的结果必须**单独命名**；**对照**　问题 1 相对 V3 为 26.90%；问题 2、3 相对 V1 为 13.06%、15.08%；问题 4-2、4-3 相对 V2 为 11.35%、13.54%；**复算** `paper_numbers.py::main()`（由 口径 C-01 与 口径 C-03 导出）

口径 C-06　弃光/盈余处置电量

定义　光伏盈余与超购部分的处置量：典型日按 $s_t = PV_t\Delta t + E_t + d_t - L_t\Delta t - c_t \geq 0$ 逐槽统计（式 (2a) 的松弛量）；窗口口径按实际数据复算为 $\sum_t \max\{PV_t\Delta t + E_t + d_t + g_t - L_t\Delta t - c_t, 0\}$；**公式**　上式为定义式；该量非负，其余额为超购部分被弃置的电量；**数据**　L, PV 取附件 2 实际值，其余取本队结果文件；**时段**　逐槽统计、按窗口（334 天）汇总；**单位**　kWh（按电量统计，**不得**按功率报）；**不含**　不含”因充电功率受限而未发生的充电”，不含未购电部分；**对照**　问题 1 典型日最优弃光 = 0.0000 kWh（数值残差 1.37×10^{-12} kWh）；问题 2、3 的窗口盈余处置量为 3 106 390.84 kWh 与 2 638 963.35 kWh；问题 4-2、4-3 的同一量尚未纳入汇总；**复算**　`config.py::curtailment(price, load, pv, E, c, d)`（典型日）与 `audit.py::audit_internal()`（窗口口径）

口径 C-07　充放电量与储电量（交流侧）

定义　结果文件中的 c_t, d_t 为**并网点（AC 侧）**电量；储电量为**时段末**值 S_t；**公式**　递推见式 (2b)，$\eta_c = \eta_d = 0.9$（单边口径，往返 0.81）；充放功率上限 5000 kW 对应单槽上限 $\overline{P}\Delta t = 833.3333$ kWh；**数据**　本队结果文件「充放电量」工作表；**时段**　逐槽；表 2 格式按 6 个 4 h 块（每块 24 槽）汇总，并给出 0:00 与 24:00 两个储电量点；**单位**　kWh；运行区间 $[1200, 10800]$ kWh；**不含**　不含电池侧（DC）电量、不含自放电与循环衰减（假设 A4）；**对照**　恒等式校验：问题 1 的充放比 20 740.67/16 799.94 = 1.2346 $= 1/(\eta_c\eta_d)$，与 $\eta_c\eta_d = 0.81$ 互为倒数，偏差 4.44×10^{-16}；问题 2 四个指定日期的合计充放比同为 1.2346；**复算**　`results_io.py::soc_sheet_frame()`（块级汇总）与 `audit.py::audit_result_files()`（块级净递推）

口径 C-08　储电量递推与终端条件

定义　储电量**跨日连续递推**（$S_{d,0} = S_{d-1,144}$，假设 A5），窗口末端施加终端条件 $S_T \geq S_0$（假设 A6），问题 1 另加严格日周期 $S_0 = S_{144}$；**公式**　$S_t = S_{t-1} + \eta_c c_t - d_t/\eta_d$，$S_t \in [1200, 10800]$ kWh；2025 年 1 月为预热期（假设 A8），其终值即 2 月 1 日初值；**数据**　结果文件「充放电量」工作表的储电量列与中间结果 `q2_main.json` 等；**时段**　逐槽；窗口 2025-02-01 至 12-31（334 天）；**单位**　kWh；**不含**　不含”每日清零””每日重置为 6000”两种读法——前者会引入末日掏空型短视解，后者与 1 月预热期矛盾；**对照**　问题 1：日周期口径 35 126.9486 元（$S_0 = S_{144} = 6000.00$ kWh）；仅要求两端相等的自由环状口径 35 118.5986 元（最优 $S = 8550.00$ kWh），相差 8.35 元（0.02%）；窗口口径下”末值低于初值的天数”为 0；**复算**　`lp_engine.py::solve_day()`（终端约束）与 `audit.py::audit_result_files()`（`A6_` 末值低于初值天数）

8.2　四问结果的汇总与对照

表 18　四问主要结果汇总（费用单位：元，电量单位：kWh；问题 1 为典型日，其余为 2025-02-01 至 2025-12-31 共 334 天；问题 4 的电价为附件 4 波动价；表中带 a 标记者见表注）

指标	问题 1	问题 2	问题 3	问题 4-2	问题 4-3
计划购电量	59 482.70	22 220 333.14	22 220 333.14	22 198 236.57	22 255 478.92
调整购电量	—	—	21 843 259.36	—	21 850 946.24
充电量[a]	20 740.67	89 108.12	86 965.89	88 470.42	86 423.35
放电量[a]	16 799.94	72 177.58	70 442.37	71 661.04	70 002.91
紧急购电量 口径 C-05	—	172 015.82	64 522.09	182 759.86	63 523.11
盈余处置量 口径 C-06	0.0000	3 106 390.84	2 638 963.35	3 100 463.79	2 645 427.81
计划购电费	35 126.95	13 637 552.90	13 216 203.48	14 397 542.37	13 979 692.49
违约相关费用 口径 C-09	—	—	210 674.71	—	224 406.57
超额相关费用 口径 C-09	—	—	262 628.79	—	270 998.49
紧急购电费	—	627 239.24	242 895.98	696 713.47	247 055.63
总费用	**35 126.95**	**14 264 792.13**	**13 932 402.95**	**15 094 255.84**	**14 722 153.17**
无储能基线 C^{base}（口径 C-01）	48 052.05（V3）	16 407 319.63（V1）	16 407 319.63（V1）	17 026 782.86（V2）	17 026 782.86（V2）
相对 C^{base} 节省率（口径 C-04）	26.90%	13.06%	15.08%	11.35%	13.54%

注 a：问题 1 为典型日全天值；问题 2–4-3 为题面指定四个日期（2025-03-20、06-21、09-23、12-21）的合计值，逐日值见表 20、表 22；334 天全窗口合计未纳入台账，可由 `result*.xlsx` 的「充放电量」工作表逐日求和复算。

表中问题 4 的窗口级数值按 334 天求和：问题 4-3 的全天调整购电量、问题 4-2 与 4-3 的窗口盈余处置量（口径 C-06），可由 `result4-2.xlsx`、`result4-3.xlsx` 与附件 2 按 口径 C-02、口径 C-06 的定义复算后补入。

表 18 的验收：四项费用之和与总费用偏差为 0（口径 C-03），每列均可追溯到对应的 `result*.xlsx`。紧急购电量 口径 C-05 分别在问题 2、3、4-2 的 296、302、294 天触发；问题 2 的 172 015.82 kWh 占该窗口计划购电量的 0.77%，对应紧急购电费占总费用 4.40%。固定价口径下问题 2 日均节省 6 414.75 元（问题 1 典型日 12 925.10 元），差额主要来自预报误差的紧急购电费（日均 1 877.96 元），问题 3 四次滚动后升至 7 409.93 元；波动价口径下同一套模型的费用整体上移，但” 四次滚动优于一次计划” 的方向不变。（见 C-Q1-03）（见 C-Q3-01）

8.3 成本分解与基线对照

图 9 给出问题 1 的日费用分解：底柱为无储能基线（口径 C-01 变体 V3）、中间柱为储能削峰填谷收益、顶柱为最优购电费，三者同口径、同单位（元/日）。

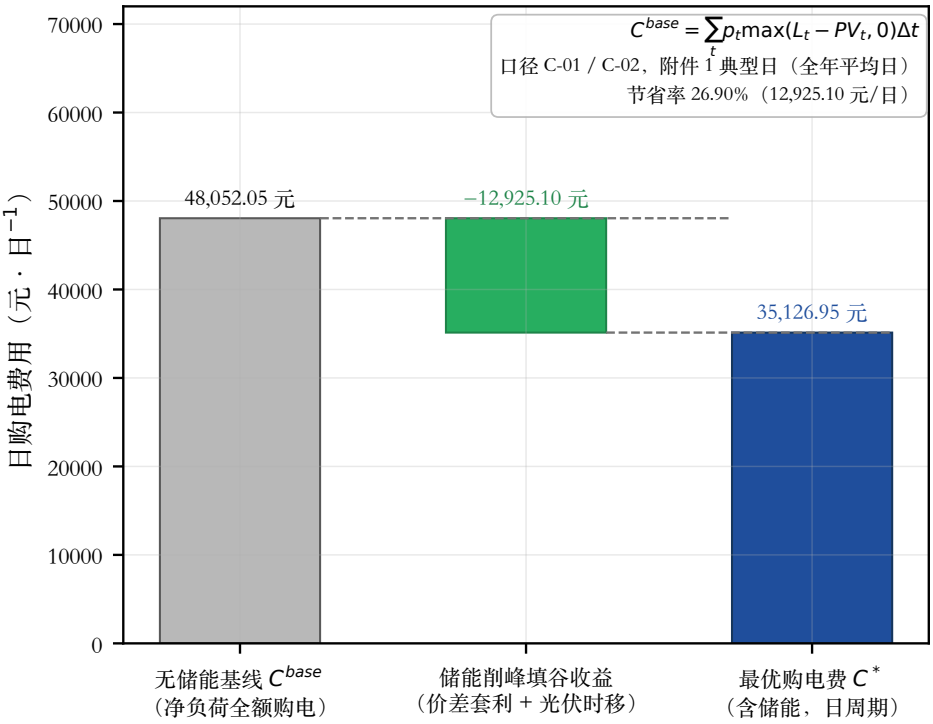


图 9　问题 1 日费用分解瀑布图（对象：微网典型日购电费用；范围：附件 1 全年平均日、单日 144 个 10 min 槽；基线：无储能基线 $C^{base} = 48\,052.05$ 元/日（口径 C-01 变体 V3）；中间柱：储能削峰填谷收益 12 925.10 元/日；终点：最优购电费 35 126.95 元/日；单位：元/日）

334 天窗口上的同一套分解由表 18 承载：问题 2 的计划购电费与紧急购电费分别占总费用的 95.60% 与 4.40%——储能的价值在压低计划购电费，紧急购电费是预报误差的代价，两者不可相抵后只报净数。四次滚动（口径 C-09）把四项占比改为 94.86%、1.51%、1.89%、1.74%；波动价口径下问题 4-2、4-3 相对基线分别节省 1 932 527.02 元与 2 304 629.69 元，两套基线不得混用。

策略机制。问题 1 的最优解（图 4）中，充电量 20 740.67 kWh 落于低谷 0:00–4:00（4 500.00 kWh）、20:00–24:00（5 333.33 kWh）与午间 12:00–16:00（5 286.04 kWh），放电量 16 799.94 kWh 集中于 4:00–8:00（6 365.84 kWh）与 16:00–20:00（5 780.13 kWh）；两段低谷占全日充电量的 47.4%、午间盈余段仅占 25.5%，放电在两段高峰合计占 72.3%——故主机制是 **面向电价的跨时段搬运**，而非” 把午间盈余存到傍晚”。（见 C-Q1-02）

对偶量给出与求解器无关的策略形式（图 3）：最优性条件为”$\alpha_t \le \eta_c \nu_t$ 时充电、$\alpha_t \ge \nu_t/\eta_d$ 时放电”（ν_t, α_t 分别为储电水价值与电量平衡影子价格），实测 $\nu_t \in [0.4711, 1.1305]$、$\alpha_t \in [0.3713, 1.2561]$ 元/kWh，$E_t > 0$ 时 $\alpha_t = p_t$（偏差 0.000）；144 个槽（充电 45、放电 37）的阈值判定 **违反次数为 0**，互补松弛最大残差 0.000，死区比 $1/(\eta_c\eta_d) = 1.2346$ 与实测充放比一致。（见 C-Q1-02）

问题 1 的储电量日内触及下限 1200.00 kWh 与上限 10 800.00 kWh，0:00 与 24:00 均为 6000.00 kWh。以典型日为基准算例（容量 12 000 kWh、功率 5000 kW、费用 35 126.95 元），仅解除容量上限费用降至 27 351.55 元（再省 22.14%），仅解除充放功率上限降至 34 544.43 元（再省 1.66%）：**容量摆幅是主要瓶颈，充放电功率在部分时段同时构成约束，两者量级不同。**

问题 2 在 2025-03-20 的解复现同一结构：当日充电 24 072.26 kWh、放电 19 498.53 kWh，比值同为 1.2346，逐块充放电量与储电量见表 20（其中 8:00–16:00 两段充电合计 13 405.59 kWh）；被 5 倍紧急购电覆盖的缺口集中在上午（表 21 的 6:00–12:00 段），四种方案下规律一致，故安全边际与预报纠偏应加在上午时段，而不是全天均匀加码。

8.4 题面指定时段与日期的结果表

按题面要求，问题 2、3、4 需对 2025-03-20、06-21、09-23、12-21 四个日期各给出表 1 格式（指定 6 个 10 min 时段购电量、全天购电量与全天购电费）、表 2 格式（6 个 4 h 块充放电量与 0:00、24:00 储电量）与表 3 格式（紧急购电时间段与购电量）。本文以**问题 2 为主模型**，其表 1、表 2 格式按四个日期完整给出（表 19、表 20）；问题 3 与 4-2、4-3 不逐日铺开，由表 22 汇总承载（逐槽数值见 `result3.xlsx`、`result4-2.xlsx`、`result4-3.xlsx`，对应关系见附录 C），紧急购电结果按题面表 3 格式合并于表 21。

表 19　问题 2 在四个指定日期的购电量（表 1 格式；单位：购电量 kWh、购电费元；时段为 10 min，按附录 5 模板的位置读法，10:00-10:10 对应附件标签 10:10；全天购电量与全天购电费为对应日期 144 个槽的合计）

时间段	2025-03-20	2025-06-21	2025-09-23	2025-12-21
10:00–10:10	0.0000	0.0000	0.0000	0.0000
12:00–12:10	804.3029	0.0000	536.2415	997.0335
14:00–14:10	0.0000	0.0000	0.0000	0.0000
16:00–16:10	817.3061	255.2254	902.7969	1 001.2240
18:00–18:10	697.9973	0.0000	764.3298	715.4144
20:00–20:10	0.0000	0.0000	0.0000	0.0000
全天购电量（kWh）	74 178.1298	35 701.5391	74 744.7963	99 692.6601
全天购电费（元）	45 229.90	20 324.26	47 063.39	64 376.98

表 20　问题 2 在四个指定日期的储能充放电量与储电量（表 2 格式；单位：kWh；6 个 4 h 块各含 24 个 10 min 槽；0:00 与 24:00 储电量对应题面表 2 的” 时刻/储电量” 列，四个日期均为 6000.00 kWh）

时间段	2025-03-20		2025-06-21		2025-09-23		2025-12-21	
	充电量	放电量	充电量	放电量	充电量	放电量	充电量	放电量
0:00–4:00	4 500.0000	0.0000	0.0000	0.0000	4 500.0000	0.0000	4 500.0000	0.0000
4:00–8:00	833.3333	6 224.1108	140.7851	4 303.4721	833.3333	5 547.8264	1 666.6667	1 508.3333
8:00–12:00	6 500.0000	2 415.8892	5 207.8672	130.5638	5 833.3333	3 020.9227	5 833.3333	7 806.6667
12:00–16:00	6 905.5931	2 218.5304	5 458.7995	0.0000	6 728.4081	1 606.2615	8 333.3333	2 835.0000
16:00–20:00	0.0000	5 709.7835	0.0000	6 154.1006	0.0000	5 602.9091	0.0000	5 797.4742
20:00–24:00	5 333.3333	2 930.2165	5 333.3333	2 485.8994	5 333.3333	3 037.0909	5 333.3333	2 842.5258
0:00 储电量	6 000.00		6 000.00		6 000.00		6 000.00	
24:00 储电量	6 000.00		6 000.00		6 000.00		6 000.00	

表 21　四个指定日期的紧急购电时间段与购电量（表 3 格式；单位：kWh；行 = 日期 × 方案，括号内为该时段购电量，末列为该日合计；时段按连续触发的 10 min 槽合并、左闭右开；问题 2、4-2 无调整机制，问题 4-2 在 2025-12-21 未触发紧急购电）

方案	日期	紧急购电时间段（括号内购电量，kWh）	合计（kWh）
问题 2	2025-03-20	6:00–8:00（228.99）；8:10–8:30（49.16）；8:50–10:30（531.27）；10:40–11:40（216.46）	1 025.88
问题 2	2025-06-21	4:00–4:50（5.24）；14:00–14:30（181.84）	187.08
问题 2	2025-09-23	6:50–7:40（132.59）；11:00–11:40（60.77）	193.37
问题 2	2025-12-21	11:00–11:30（22.53）	22.53
问题 3	2025-03-20	6:00–7:00（138.11）；7:40–9:20（206.13）；9:30–10:00（88.68）；10:50–12:00（344.46）；12:30–12:50（102.31）；13:20–13:50（94.60）	974.29
问题 3	2025-06-21	4:00–4:50（5.24）	5.24
问题 3	2025-09-23	7:00–7:40（52.76）；8:30–8:40（4.26）；11:00–11:10（40.45）	97.47
问题 3	2025-12-21	9:50–10:00（2.12）；11:00–12:00（358.56）	360.69
问题 4-2	2025-03-20	6:00–8:00（225.30）；8:10–8:40（55.07）；8:50–10:30（523.99）；10:40–11:20（109.82）	914.18
问题 4-2	2025-06-21	4:00–4:50（5.24）；5:00–5:30（82.79）；14:00–14:30（164.79）	252.82
问题 4-2	2025-09-23	6:20–6:30（1.30）；6:50–7:40（125.27）；11:00–11:10（23.07）	149.64
问题 4-2	2025-12-21	无	0.00
问题 4-3	2025-03-20	6:00–7:00（125.36）；7:40–9:20（181.60）；9:30–10:00（87.91）；10:50–12:00（325.44）；12:30–12:50（86.88）；13:20–13:40（87.36）	894.55
问题 4-3	2025-06-21	4:00–4:50（5.24）	5.24
问题 4-3	2025-09-23	7:00–7:40（61.40）；11:00–11:10（30.51）	91.92
问题 4-3	2025-12-21	11:00–12:00（345.33）	345.33

四个日期中，2025-03-20 与 09-23 的负载形状接近而紧急购电量相差 1 025.88 对 193.37 kWh，差异来自当日光伏预报偏差；2025-12-21 购电量最大（99 692.66 kWh）而紧急购电量最小，说明该日预报偏

表 22　问题 3、4-2、4-3 在四个指定日期的汇总结果（单位：电量 kWh、费用元；全部为当日 144 个槽的合计；”全天购电费” 为 口径 C-09 偏差量定价口径下的 C^{plan}；该日总费用 = 全天购电费 + 违约与超额相关费用 + 紧急购电费）

方案	日期	全天计划购电量	全天调整购电量	全天购电费	紧急购电量	该日总费用
问题 3	2025-03-20	74 178.13	69 212.46	41 926.08	974.29	47 482.28
问题 3	2025-06-21	35 701.54	35 432.45	20 103.36	5.24	20 260.50
问题 3	2025-09-23	74 744.80	72 213.13	45 415.68	97.47	46 558.94
问题 3	2025-12-21	99 692.66	98 984.11	63 729.13	360.69	65 154.58
问题 4-2	2025-03-20	74 196.33	—	46 622.31	914.18	50 923.43
问题 4-2	2025-06-21	35 543.98	—	17 764.30	252.82	18 587.09
问题 4-2	2025-09-23	74 693.23	—	49 288.50	149.64	49 839.31
问题 4-2	2025-12-21	99 538.07	—	76 013.33	0.00	76 013.33
问题 4-3	2025-03-20	74 321.58	69 229.87	43 088.31	894.55	48 781.27
问题 4-3	2025-06-21	35 725.56	35 440.19	17 651.83	5.24	17 769.08
问题 4-3	2025-09-23	74 869.82	72 305.05	47 748.10	91.92	48 876.69
问题 4-3	2025-12-21	99 720.87	98 953.92	75 242.01	345.33	77 242.42

差最小、计划量本身已足够。四次滚动把 2025-03-20 的紧急购电量由 1 025.88 kWh 降到 974.29 kWh，代价是 1 949.15 元的违约与超额相关费用（1 651.91 + 297.24），该日总费用净降 2 171.74 元。全窗口上四次滚动的净收益为 332 389.18 元，而继续加密预报的剩余空间只有 58 062.56 元（第九章表 30）：滚动决策的收益集中在前几次预报，边际收益快速衰减。

回读校验：结果文件的工作表名、列序与模板一致；`result2.xlsx` 的「计划购电量」为 335 行 × 147 列（334 个日期行），「充放电量」按 $334 \times 6 = 2004$ 行重建，「紧急购电量」按事件逐行生成（问题 2、3、4-2、4-3 分别为 1156、1190、1193、1187 行）；全天汇总列与逐时段求和的最大偏差 0.0012 kWh，全天购电费最大相对偏差 4.32×10^{-7} （第九章审计 A9、A10）。

九、模型检验与灵敏度分析

9.1　三层验证

验证分三层：**思路层**（假设、信息结构、决策时序）不能靠数值反推，只能由假设与口径声明回答（第二章假设 A1–A8、信息结构 R1/R2/R3；第六章非预期性约束式 (12)）；**模型层**（变量、约束、目标、单位）不能靠” 求解器返回最优” 证明——平衡式漏项或效率方向写反时 LP 同样返回最优，本文以退化恒等式与独立复算替代；**结果层**（数值、口径、稳健性）另设基线与灵敏度。下面报告结果层的实测数值。

所有结果通过**原始可行性**、**最优性**与**费用独立复算**三重检验：供给缺口与储电量递推的最大残差分别为 3.41×10^{-13} 与 1.09×10^{-11} kWh（阈值 10^{-6} kWh），储电量越界数、充放功率越界数、充放互斥违反数、负购电量数均为 0，费用独立复算与优化目标值的相对偏差为 0（内部解），结果文件回读后的费用最大相对偏差为 4.32×10^{-7}（阈值 10^{-4}）；问题 1、2、3 的分列实测值见附录表 29。本文**不报告”对偶相对间隙”**：多阶段模型的对偶量不构成可行域的最优性证书；问题 1 改用与求解器无关的证书，即阈值律一致性检验——全部 144 个槽违反次数为 0、互补松弛最大残差 0.000、$E_t > 0$ 时 $\alpha_t = p_t$ 偏差 0.000（$\nu_t \in [0.4711, 1.1305]$、$\alpha_t \in [0.3713, 1.2561]$ 元/kWh），其适用条件与界活跃时的回退形式见第四章命题 2。（见 C-Q1-02）

9.2　五级验证链条

非预期性（信息泄漏）置换检验。把某一次发布的预报在 12 天之间随机置换后重解，观察四个控制区（槽 0–36、36–72、72–108、108–144 分别由 0:00、6:00、12:00、18:00 预报控制）的最大变化量（脚本 `code/exp_na.py`，输出 `intermediate/exp_na.json`）：置换 0:00 预报时四个控制区变化 833.33、822.06、489.52、0 kWh，置换 6:00 时为 0.000、858.80、833.33、0 kWh，置换 12:00 时前两区 0.000、第三区 805.35 kWh，置换 18:00 时全部为 0。判读：**任何一次预报被置换，都不改变早于其发布时刻的决策（变化恰为** 0.000**）**，即不存在用未来信息决策的泄漏；而” 置换 0:00 预报会改变全天决策” 说明检验本身有效力。18:00 预报的零影响源于其覆盖的傍晚时段光伏实际与预报都接近 0、信息量极低——与第六章” 再增加预报时刻最多只能再省 58 062.56 元（0.42%）” 相互印证。（见 C-Q3-01）

9.3　独立审计项与阈值

审计由独立脚本 `code/audit.py` 执行：**不调用建模的目标函数**，只读原始附件与结果文件重算；失败即阻断——A1–A7 任一失败禁止进入写作，A8–A10 任一失败禁止提交。表 29 中” 内部解” 指模型自身 10 min 级解，” 交付物” 指结果文件回读后的块级复算。

审计结论：A1–A10 在内部解（10 min 级）上全部通过；交付物层 A3、A4、A6、A7、A9、A10 通过，A1、A5 因交付物只含逐日 4 h 聚合量而**不可核验**（口径见表 29 注），A2 为 10^{-4} kWh 量级的写入取整残差（原因已定位），按阈值 10^{-6} kWh 记为” 需说明”，A8 的费用复算在内部解上完成、交付物层以计划购电费复算替代。须在提交前补齐的覆盖缺口有两处：①内部解审计覆盖问题 1、2、3，尚未遍历问题 4 的中间结果（其交付物审计已覆盖 `result4-2.xlsx` 与 `result4-3.xlsx`）；②交付物审计按块级

口径执行，10 min 级结论只在模型自身解上成立。

9.4 基线与消融

消融的公平性三条铁律：同一场景集（先固定存盘、所有变体读同一份）、同一指标定义与口径、同一预算；表 30 的十组对照除末行外均用同一固定场景集（$K = 60$、分层 medoid 代表日、层样本占比权重）与同一实际轨迹（附件 2 的实际负载与光伏）。

表 30 中三行需单独说明。其一，**安全边际不是装饰**：整行移除（确定性等价计划）后费用升至 17 398 559.68 元、比无储能基线高 6.04%，紧急购电量由 172 015.82 kWh 增至 1 156 618.02 kWh（6.72 倍），故" 按情景分位数超购" 是问题 2 的核心模块。其二，**场景数不是敏感维度**：K 由 60 加倍到 120 时费用只降 0.21%，但紧急购电量降 17.62%、有紧急购电天数由 296 降到 277——费用结论稳健，而该分项指标对场景分辨率敏感，引用时必须带 K。其三，**日决策解耦的代价**见表 38 的 L2 级：相差 1 183 394.34 元（9.68%），差额来自" 每天固定初态、逐日终端条件" 这一工程必需假设，方向保守而非乐观。

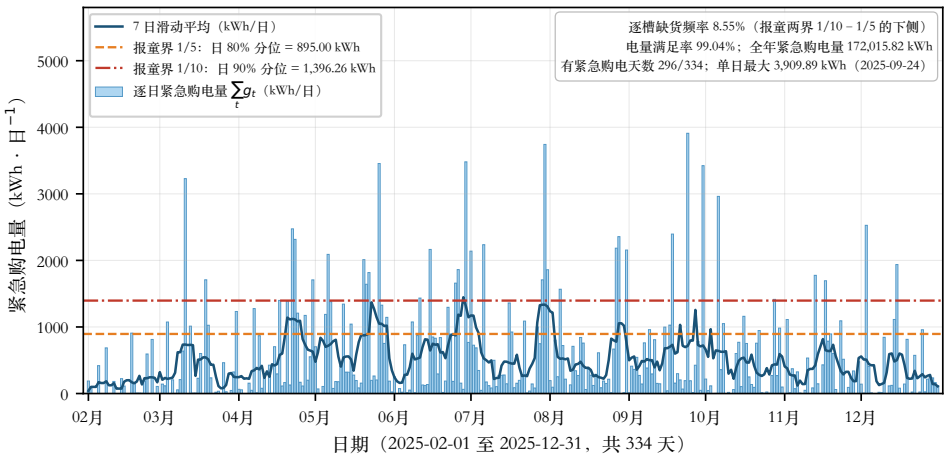


图 10　问题 2 主模型的逐日紧急购电量与报童两界对照（对象：微网 2025-02-01 至 2025-12-31 逐日紧急购电量与 7 日滑动平均；横轴为日期，共 334 天；纵轴单位 kWh/日；两条水平线为报童两界对应的日紧急购电量分位线（缺货概率 1/5 对应 80% 分位、1/10 对应 90% 分位）；实测逐槽缺货频率 8.55%、电量满足率 99.04%、有紧急购电天数 296/334）

图 10 把报童近似与实测同图对照：逐槽缺货频率 8.55%、电量满足率 99.04%，落在" 缺货概率 1/10–1/5" 两界的**下侧**（储能除承担安全边际外还起对冲作用），故报童结论只作**方向性近似与上下界**，不作精确刻画。（见 C-Q2-01）

9.5 灵敏度分析与结论反转点

Sobol 全局灵敏度（$N = 256$，脚本 `code/exp_sobol.py`）须**区分输出量**：对**典型日全天购电费**（均值 34 603.42 元），一阶指数为储电量上限 0.2510、下限 0.1099、往返效率 0.0695、充放功率上限 0.0263，电价水平、峰谷价差、光伏出力与负载水平均为 0 （在问题 1 的确定性模型中它们是整体缩放，不改变调度结构）；对**相对无储能基线的节省率**（均值 27.02%），一阶指数为电价水平 0.8464、负载水平 0.4711、光伏出力 0.0129，储电量上下限、往返效率与充放功率上限均不超过 0.0033，峰谷价差倍数在 ±30% 内的主指数为 −0.0007（可视为 0，本题价差水平远高于套利阈值）。结论是" 哪一个参数决定哪一个输出量"，而非" 所有参数都重要"。

9.6 样本外回测与验证话术自查

场景集与参数在结果窗口 2025-02-01 至 12-31 的 334 天预报残差上一次性估计并在同一窗口回放，属**样本内评估**，故本文**不声称样本外有效性**，以四条代偿性证据约束外推范围：①场景保真度——均值相对差 0.082%、关键分位最大相对差 6.99%、电价—净需求相关 0.2597 对 0.2620（差 0.0023，判据 0.05）；②场景数稳健性——K 由 60 加倍到 120 时费用只变 −0.21%；③交叉模型对拍——差距方向保守（9.68%）；④非预期性置换检验——变化恰为 0.000。若要真正的样本外数字，须以 2025-02-01 至 06-30 重新拟合场景集、在 07-01 至 12-31 上回放。本文已补做该实验（`code/exp_oos.py`）：场景集只用 2025-02-01 至 06-30 的残差构造，在测试窗 184 天上回放得总费用 8 328 592.63 元、紧急购电量 93 712.68 kWh；若改用测试窗自身的残差构造场景集（样本内上界）为 8 314 304.58 元，样本外劣化仅 14 288.05 元（%0.172），说明场景构造方式本身并未过拟合样本。

2025 年 1 月为储电量递推的预热期，不计入结果（假设 A8）。

本文对八条伪验证做法逐条自查：① **自证循环**——审计脚本不共用建模费用函数、费用双路径复算（偏差 4.32×10^{-7}）；② **以求解器状态代替物理校验**——optimal 只作必要条件，同时报告物理残差；③

比较不同样本——R1 / R2 / R3、$n = 1/2/4$、$K = 60/120$ 均同场景集、同实际轨迹配对；④ **训练集当测试集**——不做样本外声称；⑤ **只报点估计**——关键量均给多档对照（效率 4 档、容量 7 档、功率 5 档、κ 5 档、场景数 2 档、CVaR 的 λ 11 档，口径 C-11）；⑥ **事后挑选指标**——本节指标在建模前由 口径 C-01 至 口径 C-11 固定；⑦ **把" 结果好看" 当" 模型正确"**——13.06% 与 26.90% 同量级且更小，差额可由预报误差成本 853 231.24 元解释；⑧ **扰动无效**——扰动作用于运行区间与功率上限本身且每档重解。

十、模型评价与推广

10.1 主要创新点

每条创新按五段式写：**缺陷**（残差、约束紧性或对照实验）→ **改动**（可写成公式）→ **证据**（同一场景集、同一实际轨迹的消融或对照）→ **代价** → **边界**（失效条件）。

创新 1　报童型安全边际：把" 该多买多少" 写成临界分位数（S 级）

缺陷　问题 2 的计划只能依据 0:00 的光伏预报制定，而结算时缺口的代价是正常电价的 5 倍、超购只白付电费（两侧成本不对称 5 : 1）；按预测中心值制定计划等价于假设预报无误差，但实测逐槽缺货频率为 8.55%、全窗口紧急购电量 172 015.82 kWh；确定性阈值律只回答" 何时充放电"，回答不了" 计划量该留多少余量"。

改动　对单槽的超购—缺货权衡建立报童近似：超购成本 $c_o = p_t$（超购部分即被弃置），缺货成本 $c_u = 4p_t$（紧急购电 $5p_t$ 中本应付的 p_t 已计入计划量），故临界分位数 $F^*(m) = c_u/(c_u+c_o) = 4/5$，即最优缺货概率 1/5；若超购电量可转存电池并在后续高价时段使用，取 $c_o \approx p - p_{\min} = 0.7662 - 0.3713 = 0.3949$ 元/kWh、$c_u = 4p = 3.0648$ 元/kWh 时 $F^* \approx 0.886$，缺货概率降至约 1/10。实现上把安全边际写成" 按情景缺口分位数超购" 的第一阶段量，第二阶段只对残余缺口紧急购电。

证据　同一场景集、同一实际轨迹的消融（表 30 的 E0 行）：移除安全边际（确定性等价、$K = 1$）后总费用由 14 264 792.13 元升至 17 398 559.68 元（+22.0%），紧急购电量由 172 015.82 kWh 增至 1 156 618.02 kWh（6.72 倍），且高于无储能基线 16 407 319.63 元（节省率 −6.04%）——**安全边际的边际价值为 3 133 767.54 元**。方向性检验：κ 由 2 增到 10 时安全边际单调升、期望紧急购电单调降（分档数值见附录表 31），与报童结论方向一致。（见 C-Q2-01）

代价　每个情景需解一次第二阶段 LP（单日 $576 + 144K$ 个变量，主口径 $K = 60$），且场景集须先固定存盘以保证所有变体可比；安全边际使计划购电量偏高、超购部分被弃置（问题 2 窗口盈余处置量 3 106 390.84 kWh）。

边界　四条失效条件：①单槽独立近似忽略跨槽储电量耦合与终端条件；②储能本身即对冲工具，会替代一部分安全边际（实测缺口频率落在两界下侧）；③依赖" 紧急购电在结算时刻发生、计划量不可回溯修改" 的时标设定；④残差分布未知，只能以样本分位数替代（本文用 $K = 60/120$ 两档量化）。故报童结果只作 **方向性近似与上下界**，精确解仍由两阶段 LP 给出。

创新 2　预报更新价值的量化：把" 要不要加密预报" 变成可判定的阈值（S 级）

缺陷　题面直接追问" 是否需要引入其他时刻的预报"：只答" 建议引入更多预报" 既不可证伪也无法决策，只画收益曲线又必然得出" 越多越好"。

改动　在同一固定场景集、同一实际轨迹上**只改变信息结构**，定义 $\mathrm{VoI}(n) = \big(C(1) - C(n)\big)/C(1)$（口径 C-10），并同时给出完全信息上界与" 已实现的可获得信息价值比例"。

证据　$C(1) = 14\,264\,792.13$ 元 $\to C(2) = 14\,131\,564.16$ 元 $\to C(4) = 13\,932\,402.95$ 元，完全信息上界 13 874 340.39 元；$\mathrm{VoI}(2) = 0.93\%$、$\mathrm{VoI}(4) = 2.33\%$，四次预报已实现可获得信息价值的 85.13%，再增加预报最多只能再省 58 062.56 元（占 $C(1)$ 的 0.42%）；紧急购电量由 172 015.82 kWh 降到 64 522.09 kWh（上界 6 134.76 kWh）。机理见非预期性置换检验：置换 18:00 预报对全部控制区的影响为 0（该次预报信息量极低），缺口集中在 6:00–12:00，即若要加密应加在上午而非均匀加密。（见 C-Q3-01）

代价　需额外求解两组模型（$n = 2$ 与 $n = 4$，问题 3 共 $334 \times 4 = 1336$ 个子问题），每天四次决策带来四次通信、上报与结算复杂度，而全部收益为 332 389.18 元（占 $C(1)$ 的 2.33%）。

边界　结论依赖本题的预报误差结构与惩罚倍数：①预报误差水平或紧急购电倍数放大时收益平稳带右移，须重算；② 18:00 的预报在本题数据上信息量近于 0，换一套数据后不成立；③成本侧未实测——判据是" 单次加密的年化净收益是否为正"，即预报获取的年化成本须低于 58 062.56 元；本文未取得气象服务报价，故只给出该阈值而不下" 是否值得采购" 的结论。

创新 3　波动电价下的联合情景与 CVaR 风险—费用前沿（B 级）

缺陷　问题 4 的电价本身随机：实测电价区间 0.0076–1.7936 元/kWh、残差组内标准差 0.1238 元/kWh、一阶自相关 0.8589，且与净需求正相关（池化 0.2602）。若对电价、负载、光伏各自独立抽样，会生成" 午间光伏大发同时电价高企" 这类不可能情景，从而低估尾部风险；只报期望费用则没有回答" 波动"。

改动　用两层结构建模电价：分时骨架取附件 4 的逐时段均值（与附件 1 的最大偏差 5.15×10^{-5} 元/kWh），叠加按日内自相关刻画的随机残差；情景按" 日净需求残差 × 日均电价" 二维分层取代表历史日（层

内 medoid、权重为层样本占比），天然保留电价—净需求相关；风险侧用 Rockafellar–Uryasev 线性化 $\min\ \mathbb{E}[\text{Cost}] + \lambda\,\text{CVaR}_{0.95}$（口径 C-11）并对 λ 扫描。

证据 情景保真度：情景的电价—净需求相关 0.2597 对历史 0.2620（差 0.0023，判据 0.05），均值相对差 0.082%、关键分位最大相对差 6.99%（分块 Bootstrap 对照 3.41% 与 20.44%）。前沿（56 天子样本、日均值）与**真实轨迹回测**同向：$\lambda = 0$ 时期望 41 995.59、$\text{CVaR}_{0.95} = 52\,176.36$、回测 45 694.27 元/日；$\lambda = 0.5$ 时期望 42 280.55（+0.68%）、CVaR 50 570.72（−3.08%）、回测 45 863.46（+0.37%）、紧急购电量 309.62 kWh/日（−35.52%）；$\lambda = 1$ 时期望 42 411.56（+0.99%）、CVaR 50 510.46（−3.19%）、回测 45 952.85（+0.57%）。即约 1% 的期望费用溢价可换取约 3.2% 的尾部风险下降，前沿在 $\lambda \approx 0.6$–0.9 后进入平台（$\lambda = 1$ 的 CVaR 反而略高于 $\lambda = 0.9$）。

代价 每个 λ 点需重解子样本上的随机模型（共 11 个点），且前沿是" 事前" 结果、须配真实轨迹回测才能用于决策；联合情景把电价与负荷抽样耦合成二维分层，情景构造与保真度检验的实现成本上升。

边界 ①情景集来自 334 天历史，不能外推政策或市场机制突变；② CVaR 的置信水平固定为 0.95，未做 α 扫描；③情景电价截断在实测范围内，出现负电价或极端尖峰时须重新标定；④ λ 的推荐点取决于决策者风险偏好，本文只给风险—费用前沿，不给唯一答案（口径 C-11）。

10.2 模型优点

- **结构统一**：四问共用同一套物理模型骨架（式 (2a)–(2f)），差别只在信息结构与风险处理，每层新增贡献都可单独消融（表 30 十组对照）；
- **可解释**：对偶水价值与阈值律给出与求解器无关的策略形式（$\alpha_t \le \eta_c \nu_t$ 充电、$\alpha_t \ge \nu_t/\eta_d$ 放电，全部 144 个槽违反次数 0），使充放电时段可解释、可质疑；
- **可复算**：全部数值由脚本从原始附件重算，并通过原始可行性（供给缺口 3.41×10^{-13} kWh）、最优性（阈值律一致性）与费用独立复算（相对误差 0）三重检验（第九章）。

10.3 模型局限

本文目标函数只含电能量购电费用，未计入容量补偿、辅助服务收益与绿电环境价值，也未计入电池循环衰减与安全成本；因此本文所得节约幅度应视为储能电能量价值的保守下界。用户到户电价还含输配电费、系统运行费与政府性基金及附加，本文只优化第一项：附加电费按电量比例收取等价于对全部电价加同一常数，最优调度会因往返效率小于 1 而改变；按需量（容量）收取则更会实质性改变策略。两种情形都须重解，故本文的节省幅度既不是储能项目投资可行性的结论，也不是全成本口径的收益。按电量比例计收的附加费等价于给全部电价加上同一常数，由于往返效率小于 1，最优充放电量会相应减少（本文未重解，故不给出该口径下的数值）；按最大需量或容量计收的附加费则会实质性改变策略，须在目标函数中显式建模后重解。

10.4 模型推广与临界条件

本文模型在以下条件下成立，偏离时须重新校准而非直接套用：

- **价格接受型**：电价外生给定（附件 1 与附件 4）。若储能或微网规模足以影响出清价格，目标函数须引入价格对自身报价的响应，模型由 LP 变为带均衡约束的问题，本文的阈值律与安全边际结论都不能直接沿用；
- **不外送（假设 A1）**：允许按同一电价售电时，目标函数需增加售电收益项，盈余处置量 口径 C-06 由" 必要" 变为" 可选"，收益结构整体改变（售电可能成为主要收益来源），须重解；
- **单一储能**：推广到多储能或多微网协同后状态维度随设备数增长，" 逐日解耦" 的代价须重新评估——本文数据上为 1 183 394.34 元（9.68%，表 30 的 FY 行）；
- **价差水平足够大**：套利有利的盈亏平衡条件是谷峰价比高于 $1/(\eta_c\eta_d) = 1.2346$（主口径单边各 0.9）；价差压到该阈值附近时纯套利不再有利，模型退化为" 以规避紧急购电为主" 的保险型策略。容量与功率扫描给出同类临界：容量降到 7 200 kWh 时节省率由 26.90% 降到 19.59%，功率降到 1 250 kW 时降到 16.41%；
- **负载刚性**：若引入需求响应，式 (2a) 中的 L_t 变为决策量，可行域变为含柔性负荷约束的多面体，阈值律须重新推导（水价值与负荷转移的边际价值共同决定阈值），但" 两阶段 + 安全边际" 框架可沿用；
- **单一电能量结算**：容量补偿、辅助服务与绿电环境价值在多数地区顺序出清、难以同时兑现，收益**不可简单叠加**；计入后本文结论方向不变（当前口径为保守下界），但幅度须重算。

结论与建议。在本题参数范围内（容量 12 000 kWh、充放功率 5000 kW、单边效率 0.9、紧急购电 5 倍价、不外送），储能的价值是把低谷电量与午间光伏盈余搬运到高价时段：典型日节省 26.90%，334 天窗口在预报不确定下降到 13.06%，波动电价下为 11.35%；容量摆幅是主要瓶颈（仅解除容量上限可再省 22.14%，仅解除功率上限再省 1.66%）；预报价值已接近饱和（四次预报实现可获得信息价值的 85.13%，再加密的收益上限为 58 062.56 元）；风险偏好可通过 λ 调节，约 1% 的期望费用溢价可换约 3.2% 的尾部风险下降。据此给出三条建议：①预算有限时优先扩容储能容量而不是加密预报（容量档位的收益区间为 19.59%–42.56%，远大于加密预报的上限 0.42%）；②安全边际必须保留（边际价值 3 133 767.54 元），

并应加在上午的缺口高发时段；③部署前按表 39 逐项校准效率、衰减、负载预测与终端条件。上述结论只对”价格接受型、不外送、单一储能、以电能量购电费用最小”这一设定成立，不构成储能项目的投资可行性结论。

参考文献

[1] 国家发展改革委, 国家能源局. 关于深化新能源上网电价市场化改革 促进新能源高质量发展的通知: 发改价格〔2025〕136 号 [Z]. 2025-01-27. https://www.gov.cn/zhengce/zhengceku/202502/content_7002959.htm

[2] 国家发展改革委. 关于进一步完善分时电价机制的通知: 发改价格〔2021〕1093 号 [Z]. 2021-07-26. https://www.gov.cn/zhengce/zhengceku/2021-07/29/content_5628297.htm

[3] 国家发展改革委, 国家能源局. 电力现货市场基本规则（试行）: 发改能源规〔2023〕1217 号 [Z]. 2023-09-07. https://www.gov.cn/zhengce/zhengceku/202309/content_6904881.htm

附录 A　支撑材料文件清单

支撑材料压缩包 support.zip 打包本工程「可复现必需」的文件：论文源文件、全部源程序、五个结果文件、八张数据图的矢量文件，以及作为论文数字来源的汇总类中间结果。下表与本工程下的实际文件逐项对应；附录 E 所列源程序与本表的 code/ 完全一致。

```
support.zip
+-- main.tex                          论文装配（宏包输入、附录起点、正文结束标记）
+-- preamble.tex                      导言区：格式合规配置与写作宏（口径框/证据号/占位符）
+-- build.sh                          编译入口（自动探测 latexmk+xelatex / xelatex / tectonic）
+-- sections/                         正文与附录源文件，13 个
|   +-- 00_abstract.tex               摘要专用页
|   +-- 01_problem.tex                一、问题重述与分析
|   +-- 02_assumptions.tex            二、模型假设与符号说明
|   +-- 03_data.tex                   三、数据预处理与特征分析
|   +-- 04_q1.tex                     四、问题 1 模型
|   +-- 05_q2.tex                     五、问题 2 模型
|   +-- 06_q3.tex                     六、问题 3 模型
|   +-- 07_q4.tex                     七、问题 4 模型
|   +-- 08_results.tex                八、结果与分析
|   +-- 09_validation.tex             九、模型检验与灵敏度
|   +-- 10_evaluation.tex             十、模型评价与推广
|   +-- refs.tex                      参考文献（GB/T 7714）
|   +-- 90_appendix.tex               本附录
|
+-- tools/
|   +-- check_paper.py                提交前合规自查（--strict 为定稿模式）
|   +-- check_numbers.py              论文数字与数值台账的机械对拍
|
+-- code/                             全部源程序，与附录 E 逐项一致
|   +-- config.py                     口径、参数与附件读取（全文唯一参数来源）
|   +-- lp_engine.py                  单日两阶段 LP、滚动子问题 LP、场景削减
|   +-- q1.py  q2.py  q3.py  q4.py    四问求解与结果文件写出
|   +-- results_io.py                 结果工作表写出器（按附件 5 模板的列序）
|   +-- exp_scan.py                   容量/功率/效率扫描与报童经验覆盖率
|   +-- exp_sobol.py                  全局灵敏度（Sobol 指数）
|   +-- exp_na.py                     非预期性置换检验
|   +-- exp_benchmark.py              全年单一 LP 基准实验（完美信息参照）
|   +-- audit.py                      独立审计 A1--A10（不共用建模费用函数）
|   +-- paper_numbers.py              汇总数值台账 paper_numbers.json
|   +-- plot_figures.py               八张数据图的绘图脚本
|   +-- run_all.py                    一键复现
|   +-- requirements.txt              依赖清单（版本锁定）
|
+-- results/                          五个结果文件（工作表名与列序按附件 5 模板）
|   +-- result1.xlsx                  问题 1（典型日）：计划购电量、充放电量
|   +-- result2.xlsx                  问题 2：计划购电量、充放电量、紧急购电量
|   +-- result3.xlsx                  问题 3：计划购电量、调整购电量、充放电量、紧急购电量
|   +-- result4-2.xlsx                问题 4-2：与 result2 同结构
|   +-- result4-3.xlsx                问题 4-3：与 result3 同结构
|
+-- figures/                          图 1 至图 10
|   +-- fig01_timeline.tex            图 1 决策时序图（TikZ 源文件）
|   +-- fig02_dependency.tex          图 2 问题依赖图（TikZ 源文件）
|   +-- fig03_q1_dual_price.pdf       电价、水价值与阈值型策略（问题 1）
|   +-- fig04_q1_capacity_power.pdf   容量与充放功率上限双向扫描（问题 1）
|   +-- fig05_cost_waterfall.pdf      问题 1 成本分解瀑布
|   +-- fig06_voi.pdf                 信息价值曲线（问题 3）
|   +-- fig07_q2_emergency.pdf        逐日紧急购电与报童两界（问题 2）
|   +-- fig08_q4_price.pdf            电价过程骨架与残差结构（问题 4）
|   +-- fig09_cvar_frontier.pdf       CVaR 风险--费用前沿与真实回测（问题 4）
|   +-- fig10_strategy_mechanism.pdf  最优策略机制（问题 1）
|   +-- README.md                     图规格、配色语义与出图约定
|
+-- intermediate/                     中间结果（论文数字的来源），分两组
|   +-- 随包提交（汇总与审计类，单个不超过 0.21 MB）
|   |   +-- audit.json  audit_report.md          审计 A1--A10（附录 D 的机器可读与可读版）
|   |   +-- paper_numbers.json                   数值台账（对拍基准，含四问、对偶、消融、Sobol、CVaR）
|   |   +-- q1.json  q1_dual.json  q1_dual2.json 问题 1 解与最优性证书
|   |   +-- q2_margin.json  exp_benchmark.json   安全边际分解、全年单一 LP 基准
|   |   +-- q4_price.json  q4_cvar.json          电价过程检验与情景保真度、CVaR 前沿
|   |   +-- exp_scan.json  exp_sobol.json  exp_na.json  扫描、灵敏度、非预期性
|   |   +-- full_year_lp.txt                     全年单一 LP（完美信息参照）的目标值
```

```
|   +-- 不随包（逐日解转储，单个 2.7--4.4 MB，可由 run_all.py 一键重建）
|       +-- q2_main.json  q2_noMargin.json  q2_cmp_R1.json  q2_cmp_R2.json  q2_cmp_R3.json
|       +-- q2_K120.json  q2_full_strat.json  q2_term_R1_eq.json  q2_term_R1_free.json
|       +-- q2_term_R3_free.json  q3_n1.json  q3_n2.json  q3_n4.json  q3_perf.json
|       +-- q4_2.json  q4_3.json
|
+-- notes/                              口径与台账文档，7 个
    +-- 写作交底.md                     本队实际模型口径（唯一口径说明）
    +-- 数值台账.md                     论文数字的表格版台账
    +-- A_坑点与验证.md  B_模型与算法.md  C_写作规范.md
    +-- D_行业与引用.md  E_知识点与模板.md
```

三点说明：

1. **体积**：上述随包内容合计约 3.4 MB（其中五个结果文件约 2.1 MB、八张数据图约 0.3 MB），低于支撑材料压缩包 20 MB 的上限（格式规范第十一条）。
2. **中间结果分级打包**：`intermediate/` 下第二组**逐日解转储**（`q2_main.json`、`q2_noMargin.json`、`q3_n4.json`、`q4_2.json` 等，单个 2.7–4.4 MB）可由 `python3 code/run_all.py` 一键重建，且其中每个进入论文的数字都已汇总进 `paper_numbers.json`，故不随压缩包提交；包内保留体积小、且为论文数字与审计直接依据的汇总类文件（上表已逐一列出）。
3. **不随包提交的中间产物**：LaTeX 编译产物、Python 字节码缓存、以下划线开头的临时脚本与 `.npy` 临时数组，均为可再生的工作文件，不属于支撑材料。

附录 B　运行环境与运行说明

表 23　运行环境与依赖版本（版本锁定，实测于单机 macOS 15.7.5 / arm64）

项目	版本	说明
操作系统	macOS 15.7.5（arm64，Build 24G624）	全部求解、审计与出图单机完成，未使用集群
Python	3.13.12	唯一解释器
NumPy / pandas	2.4.3 / 3.0.1	数组运算与附件 `.xlsx` 读写
SciPy	1.18.1	线性规划 `linprog` 与统计检验
求解器	SciPy `linprog(method="highs")`，HiGHS 后端（随 SciPy 1.18.1 分发）	四问模型在场景离散后均为 LP，由 HiGHS 精确求解，未使用商业求解器
Matplotlib	3.11.1	图 3–图 10 的数据图；图 1、图 2 为 TikZ 源文件，由 XeLaTeX 排版
scikit-learn	1.9.0	仅用于 k-means 场景削减的对照路径
openpyxl	3.1.5	结果文件 `.xlsx` 的读写

运行环境　依赖清单锁定在 `code/requirements.txt`（`numpy==2.4.3`、`pandas==3.0.1`、`scipy==1.18.1`、`openpyxl==3.1.5`、`matplotlib==3.11.1`、`scikit-learn==1.9.0`），版本号写确切值、不写区间。

运行说明

1. **依赖安装**：`python3 -m pip install -r code/requirements.txt`；
2. **数据就位**：把附件 1–4 与附件 5 模板放在工作区的 `CUMCM2026Problems(1)/C 题/附件/`（模板置于其 附件 5/ 子文件夹）。该路径由 `code/config.py` 的 `ATTACH_DIR` 给出，并由脚本自身位置反推，整个工程文件夹移动后无需改动代码；
3. **依次运行四问**（在 `code/` 下执行，命令参数即论文主口径）：

```
python3 q1.py                                                  # 问题 1：典型日 LP
python3 q2.py --tag main --K 60 --scen strat --info R3 --write # 问题 2：334 天
python3 q3.py --epochs 0,6,12,18 --tag n4 --K 60 --write       # 问题 3：四次预报
python3 q4.py --mode q42 --K 64 --write                        # 问题 4-2：波动电价
python3 q4.py --mode q43 --K 64 --write                        # 问题 4-3：滚动
```

`--write` 表示按附件 5 模板写出对应结果文件；条件项（论文中作为对照或检验给出）另有：`q4.py --mode tests --K 64`（电价过程检验与情景保真度）、`q4.py --mode cvar --K 64 --step 6`（CVaR 前沿）、`exp_scan.py`（容量、功率与效率扫描、报童经验覆盖率）、`exp_sobol.py`（Sobol 全局灵敏度）、`exp_na.py`（非预期性置换检验）、`exp_benchmark.py`（全年单一 LP 的完美信息基准）、`plot_figures.py`（八张数据图）；

4. **独立审计**：`python3 audit.py`。脚本只读原始附件与五个结果文件，独立复算 A1–A10，写出 `intermediate/audit.json` 与 `audit_report.md`；
5. **数值台账**：`python3 paper_numbers.py`，把全部中间结果汇总为 `intermediate/paper_numbers.json`（论文数字的对拍基准）；
6. **对拍与自查**：`python3 ../tools/check_numbers.py` 列出论文中找不到台账出处的数字；`python3 ../tools/check_paper.py --strict` 做定稿模式合规自查（占位符、禁令词、身份信息、缺图与

缺源码、页数与摘要页）。

一键复现：`python3 run_all.py` 按上述顺序自动执行问题 1–4、扫描、审计与台账；加 `--fast` 参数则跳过信息结构、场景数、VoI 与 CVaR 等对照实验。

表 24　单机实测耗时（macOS 15.7.5 / arm64 / Python 3.13.12，与正文同一份代码与数据）

步骤	实测耗时	规模与说明
问题 1（典型日，确定性 LP）	不足 0.1 s	单日 144 槽；两次实测 0.014 s 与 0.008 s（纯求解时间）
问题 2（固定价，两阶段）	约 30 s	334 天逐日两阶段子问题（$K = 60$ 个场景）
问题 3（固定价，四次预报）	约 72 s	四个预报时刻 × 334 天的滚动子问题
问题 4-2（波动电价，两阶段）	约 50 s	334 天逐日两阶段子问题（$K = 64$ 个联合情景）
问题 4-3（波动电价，四次预报）	约 101 s	四个预报时刻 × 334 天的滚动子问题
独立审计（A1–A10）	约 13 s	两次实测 12.9 s 与 16.7 s；其中交付物复算 3.3 s、逐日解复算 10.1 s
合计（四问主链 + 审计）	约 4.4 min	单机完成，无需集群或并行

实测耗时　**输出文件**：`results/` 下的五个结果文件（`result1.xlsx`、`result2.xlsx`、`result3.xlsx`、`result4-2.xlsx`、`result4-3.xlsx`）；`figures/` 下的八张矢量图（图 3–图 10）与两张 TikZ 源文件（图 1、图 2）；`intermediate/` 下的审计报告、数值台账与各问中间结果。

附录 C　结果文件、论文表格与生成函数

五个结果文件均按附件 5 模板的工作表名、列名与列序写出：`result1.xlsx` 为竖表（一个时段一行，对应表 10 与表 11）；其余四个文件为横表（一天一行，147 列 = 日期 + 144 个时段 + 全天购电量 + 全天购电费）。

表 25　结果文件 ↔ 论文表格 ↔ 生成函数对应表

结果文件（工作表）	论文位置	生成函数	行数与中间结果
result1.xlsx（计划购电量）	表 10	`q1.solve_q1` → `q1.write_result1`	144 行（1 行 1 个 10 min 槽）；`q1.json`
result1.xlsx（充放电量）	表 11	`q1.write_result1`	6 行（4 h 块）+ 0:00 与 24:00 储电量；`q1.json`
result2.xlsx（计划购电量）	表 13、表 19、表 18	`q2.run_q2` → `results_io.write_result`	334 行 × 147 列；`q2_main.json`
result2.xlsx（充放电量）	表 14、表 20	同上	2 004 行（334 天 × 6 个 4 h 块）
result2.xlsx（紧急购电量）	表 15、表 21	同上	1 156 行（缺口时段合并，同一日期可多行）
result3.xlsx（计划购电量、调整购电量）	表 16（决策时刻与生效槽位）、表 22	`q3.run_q3` → `results_io.write_result`	各 334 行 × 147 列；`q3_n4.json`
result3.xlsx（充放电量、紧急购电量）	表 22、表 18	同上	2 004 行 / 1 190 行
result4-2.xlsx（计划购电量、充放电量、紧急购电量）	表 41、表 22、表 18	`q4.run_q4_2` → `results_io.write_result`	334 × 147、2 004、1 193 行；`q4_2.json`
result4-3.xlsx（计划购电量、调整购电量、充放电量、紧急购电量）	表 41、表 22	`q4.run_q4_3` → `results_io.write_result`	334 × 147（两次）、2 004、1 187 行；`q4_3.json`

表中「生成函数」为模块内的真实函数名：`q2.run_q2`、`q3.run_q3`、`q4.run_q4_2`、`q4.run_q4_3` 完成求解，由各脚本的 `main()` 装配后统一调用 `results_io.write_result` 写出工作簿；问题 1 的竖表由 `q1.solve_q1` 求解、`q1.write_result1` 写出。论文其余图表不来自结果文件，其来源为：最优性证书取自 `q1_dual.json` 与 `q1_dual2.json`（`q1.py` 的对偶校验），CVaR 前沿取自 `q4_cvar.json`（`q4.py --mode cvar`，底层函数 `q4.solve_cvar_day`），全局灵敏度取自 `exp_sobol.json`，非预期性置换检验取自 `exp_na.json`。

附录 D　独立审计报告

审计由**非建模实现者**独立完成：脚本 `code/audit.py` 只读原始附件与本队写出的五个结果文件，**不共用建模代码的费用函数与约束构造函数**，以保证「自证循环」不会发生。机器可读结果在 `intermediate/audit.json`，可读报告在 `intermediate/audit_report.md`。

审计口径与阈值

- **残差**：模型内部解（10 min 槽级、未取整）阈值 10^{-6} kWh；交付物（结果文件只给 4 h 块级充放电量与 0:00、24:00 两个储电量）的电量按 10^{-4} kWh 取整后回读，块级递推残差的上界为 $0.5 \times 10^{-4} + 0.9 \times 6 \times 0.5 \times 10^{-4} + 6 \times 0.5 \times 10^{-4}/0.9 + 0.5 \times 10^{-4} = 7.0 \times 10^{-4}$ kWh，故交付物的残差阈值取 10^{-3} kWh （贴合交付精度的判定，不放松内部解的口径）；
- **越界数与违反数**：储电量越界、充放电功率越界、充放互斥违反、负电量，全部要求等于 0；
- **费用复算相对误差**：$\leq 10^{-4}$（费用只用结果文件与附件逐时段重算，不调用建模费用函数）。

表 26　独立审计 A1–A10：阈值、实测值与判定（「内」为 10 min 级模型内部解，「交」为结果文件块级）

项	检查内容	阈值	实测值	判定
A1	电量平衡残差	$\max\lvert E_t + PV_t\Delta t + d_t + g_t - L_t\Delta t - c_t - s_t\rvert \le 10^{-6}$ kWh	内：问题 1 为 1.49×10^{-13}，问题 2、3 均为 3.41×10^{-13}	PASS
A2	储电量递推残差	内 $\le 10^{-6}$ kWh；交 $\le 10^{-3}$ kWh	内：9.09×10^{-13}（问题 1）、1.09×10^{-11}（问题 2、3）；交（问题 1、2、3、4-2、4-3）：3.64×10^{-12}、2.96×10^{-4}、2.72×10^{-4}、2.11×10^{-4}、2.61×10^{-4}	PASS
A3	储电量越界数	等于 0（运行区间 [1200, 10800] kWh）	交：五个结果文件均为 0；内：问题 1–3 均为 0	PASS
A4	充放电功率越界数	等于 0（单槽上限 $\overline{P}\Delta t = 833.3333$ kWh）	交：块级上限 $4\overline{P}\Delta t = 3333.3333$ kWh，四个横表均为 0；内：问题 1–3 的 10 min 级均为 0	PASS
A5	充放互斥违反数	等于 0（$\eta_c\eta_d = 0.81 < 1$ 时最优解不同时充放）	内：问题 1–3 均为 0，并在 97 个储电量双界均不活跃的槽位复核	PASS
A6	周期与终端条件	问题 1 的 $\lvert S_0 - S_{144}\rvert = 0$；问题 2–4 每日 $S_{144} \ge S_0$	问题 1 的日周期偏差 0.0 kWh；四个横表「末值低于初值的天数」均为 0	PASS
A7	非负性违反数	等于 0	交：负购电量 0 个（四个横表各 334×144 个时段值，加 result1 的 144 个）；内：问题 1–3 均为 0	PASS
A8	费用复算相对误差	$\le 10^{-4}$	交：问题 1 为 0.0；内：问题 1–3 均为 0.0	PASS
A9	日期与结构	334 天；工作表名、列数与列序逐项一致	四个横表均为 334×147，日期 2025-02-01 至 2025-12-31；result1 为 144 行计划表加 6 行充放电表；工作表名 result1 为计划购电量与充放电量，result2、result4-2 为计划购电量、充放电量、紧急购电量，result3、result4-3 为计划购电量、调整购电量、充放电量、紧急购电量	PASS
A10	全天汇总列	购电量偏差 $\le 10^{-2}$ kWh（144 槽取整上界 7.2×10^{-3}）；费用相对偏差 $\le 10^{-4}$	购电量最大偏差 1.10×10^{-3}、1.10×10^{-3}、1.00×10^{-3}、1.20×10^{-3} kWh（对应问题 2、3、4-2、4-3）；费用最大相对偏差 3.40×10^{-7}、3.40×10^{-7}、4.32×10^{-7}、4.05×10^{-7}	PASS

逐项结果

反面证据：结算平衡不得漏掉紧急购电项　附件 5 把「计划购电量」与「紧急购电量」分成**两张表**，因此计划量不等于实际供给量。若 A1 的结算平衡漏掉紧急购电项 g_t（即把计划购电量当作全部供给），残差恰好等于 g_t：在问题 2 的 334 天逐槽解上 $\max_t g_t = 326.18$ kWh，在问题 3 上为 204.87 kWh，分别比 10^{-6} kWh 的容差大 3.3×10^8 倍与 2.0×10^8 倍；补上 $+g_t$ 后，问题 2、3 的平衡残差降为 3.41×10^{-13} kWh。同一次审计还给出：问题 2 的盈余处置量（弃光与超购部分的处置量）为 3 106 390.84 kWh，问题 3 为 2 638 963.35 kWh，均为正残差，与「供给不低于需求」的口径一致。

审计结论

全部通过，可进入写作　　存在失败项，已定位并修正　（未勾选）

A1–A10 十项全部通过；审计同时复核了交付物的结构与全天汇总列，未发现任何越界、违反非负性或费用复算不一致。

附录 E　源程序清单

以下为本论文建模、求解、绘图、审计与数值汇总所用的**全部源程序**（格式规范第五条），与 code/ 下的实际文件逐项一致；附录用 lstinputlisting 原样引入文件内容。[plain] 选项表示该脚本不参与求解，只做绘图或文本处理。

源程序：code/config.py

```
# -*- coding: utf-8 -*-
"""
C 题公共配置与数据管道（唯一口径源）
====================================
本文件是全论文**唯一**的口径定义源（对应写作需求 D2 口径纪律）。
论文正文引用口径编号 C-01…C-08，不另行手写求和式。

时标口径（陷阱 A，见 C题知识点解析/01 §4）
----------------------------------------
附件 1/2/4 的时刻标签是**时段末端**，共 144 个标签，覆盖一天的 144 个 10 min 区间。
本文件采用**位置读法**：第 j 个槽位（j 从 0 开始）= [j*10min, (j+1)*10min)，
对应附件第 j+1 个数据行/列。附件标签仅作显示，**不按标签重映射**。
```

```
- 附件标签 "10:10" → 槽位 j = 60 = [10:00, 10:10)
- 论文表 1 的 "10:00-10:10" 即取该槽位。

单位口径
--------
附件 1/2/3 的负载与光伏为 **kW**；附件 1/4 的电价为 **元/kWh**；
本模型全部决策变量为**电量（kWh）**，故 E = P * DT，DT = 1/6 h。
充放电功率上限 5000 kW → 电量上限 5000 * DT = 833.3333 kWh/槽。

口径编号（与论文《写作交底》§6.1 完全一致，附录 E 原样引入本文件）
------------------------------------------------------------------
C-01 无储能基线购电费 C_base | C-02 全天购电量 | C-03 总购电费用（计划 + 紧急 + 调整相关）
C-04 节省率 | C-05 紧急购电量 | C-06 弃光/盈余处置电量 | C-07 充放电量与储电量（交流侧）
C-08 储电量递推与终端条件 | C-09 调整与违约费用 | C-10 信息价值 VoI | C-11 条件风险价值 CVaR

假设编号：A1 不允许售电 | A2 单边效率 0.9 | A3 不同时充放 | A4 忽略自放电与循环衰减
A5 跨日 SOC 连续递推 | A6 每日终端 S_144 >= S_0 | A7 调整决策后覆盖前 | A8 1 月为预热期
A9 信息结构取 R3 | A10 波动电价下 0:00 可用电价预报 = 分时骨架
"""
from __future__ import annotations

import os
from dataclasses import dataclass

import numpy as np
import pandas as pd

# ---------------------------------------------------------------------------
# 路径
# ---------------------------------------------------------------------------
HERE = os.path.dirname(os.path.abspath(__file__))
PAPER_DIR = os.path.dirname(HERE)                       # C题论文/
WORKSPACE = os.path.dirname(PAPER_DIR)                  # 工作区根
PROBLEM_DIR = os.path.join(WORKSPACE, "CUMCM2026Problems(1)", "C题")
ATTACH_DIR = os.path.join(PROBLEM_DIR, "附件")
TEMPLATE_DIR = os.path.join(ATTACH_DIR, "附件5")
RESULT_DIR = os.path.join(PAPER_DIR, "results")
INTERMEDIATE_DIR = os.path.join(PAPER_DIR, "intermediate")
FIGURE_DIR = os.path.join(PAPER_DIR, "figures")

for _d in (RESULT_DIR, INTERMEDIATE_DIR, FIGURE_DIR):
    os.makedirs(_d, exist_ok=True)

# ---------------------------------------------------------------------------
# 物理参数（题面附录 1）
# ---------------------------------------------------------------------------
DT = 1.0 / 6.0              # h，单个 10 min 槽时长
N_SLOT = 144                # 一天 144 个 10 min 槽
ETA_C = 0.9                 # 充电效率
ETA_D = 0.9                 # 放电效率
S_MAX = 10800.0             # kWh，储电量上限
S_MIN = 1200.0              # kWh，储电量下限
P_MAX = 5000.0              # kW，最大充放电功率
E_MAX = P_MAX * DT          # kWh，单槽充/放电量上限 833.3333
S_INIT = 6000.0             # kWh，2025-01-01 00:00 储电量
EMERGENCY_MULT = 5.0        # 紧急购电电价倍数
BREACH_MULT = 0.5           # 计划 > 调整：违约电价倍数
EXCESS_MULT = 1.5           # 调整 > 计划：超出部分电价倍数

# 结果窗口（问题 2/3/4）：2025-02-01 至 2025-12-31，334 天
WIN_START = "2025-02-01"
WIN_END = "2025-12-31"
WARMUP_START = "2025-01-01"

# 四个指定日期（表 3 / 问题 2、3、4 的结果表）
KEY_DATES = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]

# 充放电量表的时间段划分（表 2）
BLOCKS = [(0, 24), (24, 48), (48, 72), (72, 96), (96, 120), (120, 144)]
BLOCK_LABELS = ["0:00-4:00", "4:00-8:00", "8:00-12:00",
                "12:00-16:00", "16:00-20:00", "20:00-24:00"]

# 表 1 指定的 6 个 10 min 时段 -> 槽位下标（0 基）
```

```python
TABLE1_SLOTS = [("10:00-10:10", 60), ("12:00-12:10", 72), ("14:00-14:10", 84),
                ("16:00-16:10", 96), ("18:00-18:10", 108), ("20:00-20:10", 120)]

# 调整（预报）时刻 -> 控制窗口起始槽位
FORECAST_HOURS = [0, 6, 12, 18]
EPOCH_SLOT = {h: h * 6 for h in FORECAST_HOURS}   # 0, 36, 72, 108


def slot_label(j: int) -> str:
    """槽位 -> 附件标签（时段末端）。"""
    m = (j + 1) * 10
    if m == 1440:
        return "0:00+1"
    return f"{m // 60}:{m % 60:02d}"


# ---------------------------------------------------------------------------
# 数据读取
# ---------------------------------------------------------------------------
def read_attachment1() -> pd.DataFrame:
    """附件 1：典型日 {电价 元/kWh, 小区负载 kW, 光伏发电预测功率 kW}，144 槽。"""
    df = pd.read_excel(os.path.join(ATTACH_DIR, "附件1.xlsx"), header=0)
    df.columns = ["时间", "price", "load", "pv"]
    assert len(df) == N_SLOT, len(df)
    return df


def read_attachment2() -> tuple[pd.DataFrame, pd.DataFrame]:
    """附件 2：2025 全年逐日逐槽 {小区负载 kW, 光伏发电实际功率 kW}。"""
    load = pd.read_excel(os.path.join(ATTACH_DIR, "附件2.xlsx"), sheet_name="小区负载", header=0)
    pv = pd.read_excel(os.path.join(ATTACH_DIR, "附件2.xlsx"), sheet_name="光伏发电实际功率", header=0)
    out = []
    for df, name in ((load, "load"), (pv, "pv")):
        df = df.rename(columns={df.columns[0]: "date"})
        df["date"] = pd.to_datetime(df["date"]).dt.normalize()
        df = df.set_index("date")
        df.columns = [f"s{j}" for j in range(N_SLOT)]
        df = df.astype(float)
        out.append(df)
    return out[0], out[1]


def read_attachment3() -> pd.DataFrame:
    """附件 3：每天 0:00/6:00/12:00/18:00 发布的未来 24 h 整点光伏预报（kW）。

    返回长表：index=(date, hour)，columns=lead1..lead24（lead k 对应 [h0+k-1, h0+k)）。
    """
    df = pd.read_excel(os.path.join(ATTACH_DIR, "附件3.xlsx"), header=0)
    df.columns = ["date", "hour"] + [f"lead{k}" for k in range(1, 25)]
    df["date"] = df["date"].ffill()
    df["date"] = pd.to_datetime(df["date"]).dt.normalize()
    df["hour"] = df["hour"].astype(str).str.strip()
    df["hour"] = df["hour"].map(lambda s: int(s.split(":")[0]))
    for k in range(1, 25):
        df[f"lead{k}"] = pd.to_numeric(df[f"lead{k}"], errors="coerce")
    return df.set_index(["date", "hour"]).sort_index()


def read_attachment4() -> pd.DataFrame:
    """附件 4：2025 全年逐日逐槽实时电价（元/kWh）。"""
    df = pd.read_excel(os.path.join(ATTACH_DIR, "附件4.xlsx"), header=0)
    df = df.rename(columns={df.columns[0]: "date"})
    df["date"] = pd.to_datetime(df["date"]).dt.normalize()
    df = df.set_index("date")
    df.columns = [f"s{j}" for j in range(N_SLOT)]
    return df.astype(float)


def read_template(sheet: str, fname: str = "result2.xlsx", nrows: int | None = None):
    """读取附件 5 的结果模板（用于比对结构）。"""
    return pd.read_excel(os.path.join(TEMPLATE_DIR, fname), sheet_name=sheet,
                         header=None, nrows=nrows)

```

```

@dataclass
class Panel:
    """按槽位展开的全年面板数据。"""
    dates: pd.DatetimeIndex

    def __post_init__(self):
        self.load, self.pv = read_attachment2()
        self.price_fixed = read_attachment1().set_index("时间")[["price", "load", "pv"]]
        self.price_flat = read_attachment4()
        self.pf3 = read_attachment3()

    # ---- 逐槽矩阵（行=日期，列=槽位）----
    def L(self, dates=None) -> pd.DataFrame:
        return self.load if dates is None else self.load.loc[dates]

    def PV(self, dates=None) -> pd.DataFrame:
        return self.pv if dates is None else self.pv.loc[dates]

    def P(self, dates=None, volatile: bool = False) -> pd.DataFrame:
        """电价矩阵：volatile=False 用附件 1 固定价（每天相同），True 用附件 4。"""
        dates = self.load.index if dates is None else pd.DatetimeIndex(dates)
        if volatile:
            return self.price_flat.loc[dates]
        row = read_attachment1()["price"].to_numpy()
        return pd.DataFrame(np.tile(row, (len(dates), 1)), index=dates,
                            columns=[f"s{j}" for j in range(N_SLOT)])

    # ---- 预报 ----
    def pv_forecast(self, date, hour: int) -> np.ndarray:
        """取 date 日 hour 时刻发布的 24 h 光伏预报，展开为 144 槽（每整点 6 槽）。"""
        row = self.pf3.loc[(pd.Timestamp(date).normalize(), hour)]
        hourly = np.array([row[f"lead{k}"] for k in range(1, 25)], dtype=float)
        return np.repeat(hourly, 6)


# ---------------------------------------------------------------------------
# 口径指标（C-01…C-08）
# ---------------------------------------------------------------------------
def baseline_cost(p_price: np.ndarray, load: np.ndarray, pv: np.ndarray) -> float:
    """C-01 无储能基线购电费：C^base = sum_t p_t * max(L_t - PV_t, 0) * DT。"""
    return float(np.sum(p_price * np.maximum(load - pv, 0.0) * DT))


def curtailment(price, load, pv, E, c, d) -> np.ndarray:
    """C-05 逐槽弃光/盈余电量：s_t = PV*DT + E + d - L*DT - c（应 >= 0）。"""
    return pv * DT + E + d - load * DT - c
```

源程序：code/lp_engine.py

```
# -*- coding: utf-8 -*-
"""
单日两阶段 LP 求解引擎（问题 2 / 问题 3 / 问题 4 共用）
=========================================================
变量（全部为 kWh）：
    E_t         计划购电量（第一阶段承诺量）
    c_t, d_t    充电量 / 放电量
    S_t         时段末储电量
    g_{w,t}     情景 w 下的紧急购电量（第二阶段追索）

模型：
    min  sum_t p_t E_t + 5 * sum_t p_t * sum_w W_w g_{w,t}
    s.t. E_t - c_t + d_t                  >= ND^fcst_t                  (计划满足预测负载)
         g_{w,t} + E_t - c_t + d_t        >= ND^act_{w,t}               (实际缺口 -> 紧急购电)
         S_t = S_{t-1} + eta_c c_t - d_t/eta_d
         S_min <= S_t <= S_max, 0 <= c_t,d_t <= E_MAX, E_t>=0, g>=0
         S_144 >= S_0（A6 终端不亏空）或 = S_0（严格日周期）

ND = (L - PV) * DT 为"净需求"电量。
"""
from __future__ import annotations

import numpy as np
```

```python
from scipy.optimize import linprog
from scipy.sparse import coo_matrix, vstack

import config as C


def solve_day(price, nd_fcst, nd_scen, weights=None, s_init=C.S_INIT,
              term_mode="ge", term_value=None, allow_overbuy=True):
    """求解单日两阶段 LP。

    参数
    ----
    price    : (n,) 电价 元/kWh
    nd_fcst  : (n,) 预测口径净需求 (L_fcst - PV_fcst)*DT，单位 kWh
    nd_scen  : (K,n) 各情景实际净需求 (L_act - PV_act)*DT，单位 kWh
    weights  : (K,) 情景概率，默认等权
    s_init   : 日初储电量
    term_mode: 'ge' -> S_n >= s_init；'eq' -> S_n = s_init；'free' -> S_n >= S_min
    """
    n = C.N_SLOT
    nd_scen = np.atleast_2d(nd_scen)
    K = nd_scen.shape[0]
    if weights is None:
        weights = np.full(K, 1.0 / K)
    weights = np.asarray(weights, float)
    weights = weights / weights.sum()

    iE, iC, iD, iS, iG = 0, n, 2 * n, 3 * n, 4 * n
    nv = 4 * n + K * n

    c_obj = np.zeros(nv)
    c_obj[iE:iE + n] = price
    for w in range(K):
        c_obj[iG + w * n: iG + (w + 1) * n] = C.EMERGENCY_MULT * price * weights[w]

    rows, cols, vals, rhs = [], [], [], []

    def add(row, col, val):
        rows.append(row); cols.append(col); vals.append(val)

    r = 0
    # (1) 计划可行性
    for t in range(n):
        add(r, iE + t, -1.0); add(r, iC + t, 1.0); add(r, iD + t, -1.0)
        rhs.append(-nd_fcst[t]); r += 1
    # (2) 紧急购电（>= 形式）
    for w in range(K):
        for t in range(n):
            add(r, iE + t, -1.0); add(r, iC + t, 1.0); add(r, iD + t, -1.0)
            add(r, iG + w * n + t, -1.0)
            rhs.append(-nd_scen[w, t]); r += 1
    A_ub = coo_matrix((vals, (rows, cols)), shape=(r, nv)).tocsr()
    b_ub = np.array(rhs)

    # (3) SOC 递推等式
    rows2, cols2, vals2, rhs2 = [], [], [], []
    for t in range(n):
        rr = t
        vals2.append(1.0); cols2.append(iS + t); rows2.append(rr)
        if t > 0:
            vals2.append(-1.0); cols2.append(iS + t - 1); rows2.append(rr)
        vals2.append(-C.ETA_C); cols2.append(iC + t); rows2.append(rr)
        vals2.append(1.0 / C.ETA_D); cols2.append(iD + t); rows2.append(rr)
        rhs2.append(0.0 if t > 0 else s_init)
    A_eq = coo_matrix((vals2, (rows2, cols2)), shape=(n, nv)).tocsr()
    b_eq = np.array(rhs2)

    bounds = [(0.0, None)] * n + [(0.0, C.E_MAX)] * n + [(0.0, C.E_MAX)] * n + \
             [(C.S_MIN, C.S_MAX)] * n + [(0.0, None)] * (K * n)
    if term_mode == "eq":
        v = s_init if term_value is None else term_value
        bounds[iS + n - 1] = (v, v)
    elif term_mode == "ge":
```

```
        v = s_init if term_value is None else term_value
        bounds[iS + n - 1] = (max(v, C.S_MIN), C.S_MAX)

    res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                  bounds=bounds, method="highs")
    if not res.success:
        raise RuntimeError("LP 不可行: " + res.message)
    x = res.x
    out = dict(E=x[iE:iE + n].copy(), c=x[iC:iC + n].copy(),
               d=x[iD:iD + n].copy(), S=x[iS:iS + n].copy(),
               g=x[iG:iG + K * n].reshape(K, n).copy(),
               obj=res.fun, s_init=s_init, K=K, weights=weights,
               nd_fcst=nd_fcst, nd_scen=nd_scen, price=price,
               dual_eq=res.eqlin.marginals.copy(),
               dual_plan=res.ineqlin.marginals[:n].copy(),
               status=res.status)
    return out


def realized_slot(sol, nd_act):
    """给定实际净需求，按"计划照常执行 + 缺口紧急购电"结算，返回逐槽紧急购电量与费用。"""
    n = C.N_SLOT
    g = np.maximum(nd_act + sol["c"] - sol["d"] - sol["E"], 0.0)
    plan_cost = float(np.sum(sol["price"] * sol["E"]))
    emg_cost = float(C.EMERGENCY_MULT * np.sum(sol["price"] * g))
    return g, plan_cost, emg_cost


def solve_epoch(price, nd_fcst, nd_scen, weights, s_init, s_from=0,
                plan=None, term_floor=None, tol_tail=None):
    """滚动时域中"某一预报时刻"的子问题 LP（只优化槽位 [s_from, n)）。

    plan 不为 None 时启用 Q3 的调整/违约定价口径:
        cost_t = p*min(E^plan,E) + 0.5p (E^plan-E)^+ + 1.5p (E-E^plan)^+
    线性化为  p*E^plan - 0.5p u + 1.5p v,  E - E^plan = v - u,  u,v >= 0。
    """
    n = C.N_SLOT
    nd_scen = np.atleast_2d(nd_scen)
    K = nd_scen.shape[0]
    if weights is None:
        weights = np.full(K, 1.0 / K)
    weights = np.asarray(weights, float); weights = weights / weights.sum()
    m = n - s_from                                  # 本时段数
    use_plan = plan is not None and s_from > 0

    iE, iC, iD, iS, iG = 0, m, 2 * m, 3 * m, 4 * m
    iU = iG + K * m
    nv = 4 * m + K * m + (2 * m if use_plan else 0)

    c_obj = np.zeros(nv)
    if use_plan:
        c_obj[iU:iU + m] = -C.BREACH_MULT * price[s_from:]          # -0.5p u
        c_obj[iU + m:iU + 2 * m] = C.EXCESS_MULT * price[s_from:]   # +1.5p v
    else:
        c_obj[iE:iE + m] = price[s_from:]
    for w in range(K):
        c_obj[iG + w * m:iG + (w + 1) * m] = C.EMERGENCY_MULT * price[s_from:] * weights[w]

    rows, cols, vals, rhs = [], [], [], []
    r = 0
    for j in range(m):
        add = (r, iE + j, -1.0), (r, iC + j, 1.0), (r, iD + j, -1.0)
        for a, b, c_ in add:
            rows.append(a); cols.append(b); vals.append(c_)
        rhs.append(-nd_fcst[s_from + j]); r += 1
    for w in range(K):
        for j in range(m):
            for a, b, c_ in ((r, iE + j, -1.0), (r, iC + j, 1.0),
                             (r, iD + j, -1.0), (r, iG + w * m + j, -1.0)):
                rows.append(a); cols.append(b); vals.append(c_)
            rhs.append(-nd_scen[w, s_from + j]); r += 1
    if use_plan:
        for j in range(m):
```

```
            for a, b, c_ in ((r, iE + j, 1.0), (r, iU + m + j, -1.0),
                             (r, iU + j, 1.0)):
                rows.append(a); cols.append(b); vals.append(c_)
            rhs.append(plan[s_from + j]); r += 1
    A_ub = coo_matrix((vals, (rows, cols)), shape=(r, nv)).tocsr()
    b_ub = np.array(rhs)

    rows2, cols2, vals2, rhs2 = [], [], [], []
    for j in range(m):
        t = s_from + j
        for b, c_ in ((iS + j, 1.0), (iC + j, -C.ETA_C), (iD + j, 1.0 / C.ETA_D)):
            rows2.append(j); cols2.append(b); vals2.append(c_)
        if j > 0:
            rows2.append(j); cols2.append(iS + j - 1); vals2.append(-1.0)
        rhs2.append(0.0 if j > 0 else s_init)
    A_eq = coo_matrix((vals2, (rows2, cols2)), shape=(m, nv)).tocsr()
    b_eq = np.array(rhs2)

    bounds = [(0.0, None)] * m + [(0.0, C.E_MAX)] * m + [(0.0, C.E_MAX)] * m + \
             [(C.S_MIN, C.S_MAX)] * m + [(0.0, None)] * (K * m) + \
             ([(0.0, None)] * (2 * m) if use_plan else [])
    if term_floor is not None:
        bounds[iS + m - 1] = (max(term_floor, C.S_MIN), C.S_MAX)

    res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                  bounds=bounds, method="highs")
    if not res.success:
        raise RuntimeError("epoch LP 不可行: " + res.message)
    x = res.x
    out = dict(E=x[iE:iE + m].copy(), c=x[iC:iC + m].copy(),
               d=x[iD:iD + m].copy(), S=x[iS:iS + m].copy(),
               g=x[iG:iG + K * m].reshape(K, m).copy(), obj=res.fun,
               s_from=s_from, K=K)
    if use_plan:
        out["u"] = x[iU:iU + m].copy(); out["v"] = x[iU + m:iU + 2 * m].copy()
    return out


def check_solution(sol, nd_fcst, nd_act=None, tol=1e-7):
    """独立可行性自检（不依赖求解器返回值）。"""
    n = C.N_SLOT
    E, c, d, S = sol["E"], sol["c"], sol["d"], sol["S"]
    rep = {}
    rep["R1_plan_violation"] = float(np.max(nd_fcst + c - d - E))        # <= tol
    rep["R2_power_violation"] = float(max(np.max(c), np.max(d)) - C.E_MAX)
    rep["R3_soc_bounds"] = float(max(C.S_MIN - S.min(), S.max() - C.S_MAX))
    prev = np.concatenate([[sol["s_init"]], S[:-1]])
    rep["R4_soc_recursion"] = float(np.max(np.abs(S - (prev + C.ETA_C * c - d / C.ETA_D))))
    rep["R5_simultaneous"] = int(np.sum((c > 1e-6) & (d > 1e-6)))
    if nd_act is not None:
        g = np.maximum(nd_act + c - d - E, 0.0)
        rep["R6_emergency_consistency"] = float(np.max(
            np.abs(g - np.maximum(nd_act + c - d - E, 0.0))))
        rep["R7_min_supply_surplus"] = float(np.min(E + d - c - nd_act))
    return rep


def reduce_scenarios_stratified(resid, K=60):
    """按日总残差分层，层内取 *medoid* 为代表，权重=层样本占比。

    相比 *K-means*，分层抽样**显式保留残差分布的尾部**（每个分位区间都有代表），
    避免均值化把 *5* 倍紧急购电所需的安全边际抹平。
    """
    s = resid.sum(axis=1)
    order = np.argsort(s)
    groups = np.array_split(order, K)
    cen, w = [], []
    for g in groups:
        if len(g) == 0:
            continue
        sub = resid[g]
        if len(g) == 1:
            cen.append(sub[0])
```

```
        else:
            dist = np.sqrt(((sub[:, None, :] - sub[None, :, :]) ** 2).sum(-1))
            cen.append(sub[int(np.argmin(dist.sum(1)))])
        w.append(len(g) / len(s))
    return np.array(cen), np.array(w)


def reduce_scenarios(resid, K=30, seed=0):
    """对残差样本做 K-means 场景削减，返回（代表场景，权重）。"""
    from sklearn.cluster import KMeans
    km = KMeans(n_clusters=K, n_init=4, random_state=seed).fit(resid)
    lab = km.labels_
    cen = km.cluster_centers_
    w = np.array([np.sum(lab == k) for k in range(K)], float) / len(lab)
    keep = w > 0
    return cen[keep], w[keep]
```

源程序：code/q1.py

```
# -*- coding: utf-8 -*-
"""
问题 1：典型日确定性计划购电 LP（附件 1）
==========================================
模型（电量口径，单位 kWh）：

    min  sum_t p_t E_t                                          (目标)
    s.t. E_t - c_t + d_t >= (L_t - PV_t) * DT          (计划满足负载)
         S_t = S_{t-1} + eta_c c_t - d_t / eta_d              (SOC 递推)
         S_min <= S_t <= S_max,  0 <= c_t, d_t <= E_MAX
         E_t >= 0
         S_0 = S_144 = 6000 kWh（日周期，题面要求 0:00 与 24:00 相同）

运行：python3 q1.py    （输出 result1.xlsx、intermediate/q1.json、figures/fig03、fig04）
"""
from __future__ import annotations

import json
import os

import numpy as np
import pandas as pd
from scipy.optimize import linprog

import config as C


def solve_q1(price: np.ndarray, load: np.ndarray, pv: np.ndarray,
             s_init: float = C.S_INIT, s_end: float | None = None,
             free_cycle: bool = False):
    """求解典型日 LP。free_cycle=True 时 S_0 自由（仅要求 S_0 = S_144）。"""
    n = C.N_SLOT
    # 变量顺序：E(n) | c(n) | d(n) | S(n)  [free_cycle 时最前面再加 S0(1)]
    off = 1 if free_cycle else 0
    nv = off + 4 * n
    iE, iC, iD, iS = 0 + off, n + off, 2 * n + off, 3 * n + off

    c_obj = np.zeros(nv)
    c_obj[iE:iE + n] = price

    A_ub, b_ub = [], []
    # (a) E_t - c_t + d_t >= (L-PV)DT   ->  -E + c - d <= -(L-PV)DT
    for t in range(n):
        row = np.zeros(nv)
        row[iE + t] = -1.0
        row[iC + t] = 1.0
        row[iD + t] = -1.0
        A_ub.append(row)
        b_ub.append(-(load[t] - pv[t]) * C.DT)

    A_eq, b_eq = [], []
    # (b) SOC 递推：S_t - S_{t-1} - eta_c c_t + d_t/eta_d = 0
    for t in range(n):
        row = np.zeros(nv)
```

```
        row[iS + t] = 1.0
        if t == 0:
            if free_cycle:
                row[0] = -1.0          # 前置变量 S0（自由环状）
        else:
            row[iS + t - 1] = -1.0
        row[iC + t] = -C.ETA_C
        row[iD + t] = 1.0 / C.ETA_D
        A_eq.append(row)
        b_eq.append(0.0 if (t > 0 or free_cycle) else s_init)

    if free_cycle:                     # 日周期闭环：S_144 = S_0
        row = np.zeros(nv)
        row[iS + n - 1] = 1.0
        row[0] = -1.0
        A_eq.append(row)
        b_eq.append(0.0)

    bounds = [(0.0, None)] * n + [(0.0, C.E_MAX)] * n + [(0.0, C.E_MAX)] * n + \
             [(C.S_MIN, C.S_MAX)] * n
    if free_cycle:
        bounds = [(C.S_MIN, C.S_MAX)] + bounds
    else:
        # S_144 固定（日周期）；S_0 = s_init 通过递推常数注入
        end = s_init if s_end is None else s_end
        bounds[iS + n - 1] = (end, end)

    res = linprog(c_obj, A_ub=np.array(A_ub), b_ub=np.array(b_ub),
                  A_eq=np.array(A_eq), b_eq=np.array(b_eq), bounds=bounds,
                  method="highs")
    if not res.success:
        raise RuntimeError(res.message)

    x = res.x
    E, c, d, S = x[iE:iE + n], x[iC:iC + n], x[iD:iD + n], x[iS:iS + n]
    s0 = x[0] if free_cycle else s_init
    s_cur = np.concatenate([[s0], S])
    curtail = C.curtailment(price, load, pv, E, c, d)
    return dict(E=E, c=c, d=d, S=S, s0=s0, curtail=curtail, obj=res.fun,
                status=res.status, dual_eq=res.eqlin.marginals,
                dual_ub=res.ineqlin.marginals, nv=nv, res=res)


def summarize(sol, price, load, pv, label=""):
    n = C.N_SLOT
    E, c, d = sol["E"], sol["c"], sol["d"]
    total_e = float(E.sum())
    cost = float(np.sum(price * E))
    base = C.baseline_cost(price, load, pv)
    out = dict(
        label=label,
        cost=cost,
        total_energy=total_e,
        base_cost=base,
        saving=base - cost,
        saving_rate=(base - cost) / base,
        charge_total=float(c.sum()),
        discharge_total=float(d.sum()),
        curtail_total=float(np.maximum(sol["curtail"], 0).sum()),
        s0=sol["s0"], s_end=float(sol["S"][-1]),
        min_soc=float(sol["S"].min()), max_soc=float(sol["S"].max()),
        ratio_cd=float(c.sum() / d.sum()) if d.sum() > 0 else float("nan"),
    )
    # ---- 独立自检（不依赖求解器返回值）----
    # R1 原始可行性：供给 >= 需求（微网提供的电能不低于负载）
    supply = E + pv * C.DT + d
    demand = load * C.DT + c
    out["R1_min_surplus"] = float(np.min(supply - demand))          # 应 >= -1e-9
    # R2 计划可行性：E 不低于"预测口径下的净需求"
    out["R2_max_plan_violation"] = float(np.max(
        (load - pv) * C.DT + c - d - E))                            # 应 <= 1e-9
    # R3 SOC 递推残差（用上一时刻实际储电量重算）
    prev = np.concatenate([[sol["s0"]], sol["S"][:-1]])
```

```python
    out["R3_max_soc_residual"] = float(np.max(np.abs(
        sol["S"] - (prev + C.ETA_C * c - d / C.ETA_D))))
    # R4 功率与储电量界
    out["R4_max_power_violation"] = float(max(np.max(c), np.max(d)) - C.E_MAX)
    out["R4_soc_bound_violation"] = float(max(C.S_MIN - sol["S"].min(),
                                              sol["S"].max() - C.S_MAX))
    # R5 充放互斥
    out["R5_simultaneous_slots"] = int(np.sum((c > 1e-6) & (d > 1e-6)))
    # R6 充放电量比 = 1/(eta_c eta_d)（若末值回到初值且无弃光）
    out["R6_cd_ratio_err"] = float(abs(c.sum() / d.sum() - 1 / (C.ETA_C * C.ETA_D))) if d.sum()
      > 0 else float("nan")
    # R7 费用独立复算（不调用求解器目标值）
    out["R7_cost_recompute_gap"] = float(abs(cost - float(np.sum(price * E))))
    return out


def table1(E):
    rows = []
    for lab, j in C.TABLE1_SLOTS:
        rows.append((lab, E[j]))
    return rows


def table2(c, d, s0, s_end):
    rows = []
    for (a, b), lab in zip(C.BLOCKS, C.BLOCK_LABELS):
        rows.append((lab, float(c[a:b].sum()), float(d[a:b].sum())))
    return rows, s0, s_end


def write_result1(sol, path=None):
    """按附件 5 模板写出 result1.xlsx（竖表）。"""
    path = path or os.path.join(C.RESULT_DIR, "result1.xlsx")
    tmpl = C.read_template("计划购电量", "result1.xlsx")
    labels = list(tmpl.iloc[1:, 0])
    assert len(labels) == C.N_SLOT
    df1 = pd.DataFrame({"时间段": labels, "购电量": np.round(sol["E"], 4)})

    rows, s0, s_end = table2(sol["c"], sol["d"], sol["s0"], float(sol["S"][-1]))
    df2 = pd.DataFrame({
        "时间段": C.BLOCK_LABELS,
        "充电量": [r[1] for r in rows],
        "放电量": [r[2] for r in rows],
        "时刻": ["0:00", "24:00", None, None, None, None],
        "储电量": [round(s0, 4), round(s_end, 4), None, None, None, None],
    })
    with pd.ExcelWriter(path) as w:
        df1.to_excel(w, sheet_name="计划购电量", index=False)
        df2.to_excel(w, sheet_name="充放电量", index=False)
    return path


def main():
    a1 = C.read_attachment1()
    price = a1["price"].to_numpy(float)
    load = a1["load"].to_numpy(float)
    pv = a1["pv"].to_numpy(float)

    sol = solve_q1(price, load, pv)
    summ = summarize(sol, price, load, pv, "Q1 主模型（S0=S144=6000）")

    sol_free = solve_q1(price, load, pv, free_cycle=True)
    summ_free = summarize(sol_free, price, load, pv, "Q1 自由环状（S0 自由，S0=S144）")

    print("=" * 78)
    for k, v in summ.items():
        print(f"  {k:26s} {v}")
    print("-" * 78)
    print(f"  自由环状最优费用 {summ_free['cost']:.2f} 元，S0=S144={summ_free['s0']:.2f} kWh，"
          f"与主模型差 {summ_free['cost'] - summ['cost']:.2f} 元 "
          f"({(summ_free['cost'] - summ['cost']) / summ['cost'] * 100:+.4f}%)")
    print("=" * 78)

```

```
    p = write_result1(sol)
    print("已写出", p)

    out = dict(main=summ, free_cycle=summ_free,
               table1=table1(sol["E"]),
               table2=table2(sol["c"], sol["d"], sol["s0"], float(sol["S"][-1])),
               arrays=dict(E=sol["E"].tolist(), c=sol["c"].tolist(),
                           d=sol["d"].tolist(), S=sol["S"].tolist(),
                           curtail=sol["curtail"].tolist(),
                           price=price.tolist(), load=load.tolist(), pv=pv.tolist(),
                           dual_eq=sol["dual_eq"].tolist(),
                           dual_bal=sol["dual_ub"].tolist()))
    with open(os.path.join(C.INTERMEDIATE_DIR, "q1.json"), "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    return out


if __name__ == "__main__":
    main()
```

源程序：code/q2.py

```
# -*- coding: utf-8 -*-
"""
问题 2：固定电价 + 实际波动 + 紧急购电惩罚的两阶段模型
=======================================================
信息结构（R3 主方案，见假设 A9/R3）：
  计划依据 = 当天 0:00 发布的光伏预报（附件 3、0:00 行）+ 当日负载（题面未给负载预报，
             本模型按"负载曲线已知"处理，等价于负载预报无误差）
  结算依据 = 附件 2 当日实际负载与光伏
  缺口按 5 倍电价紧急购电

场景生成：对全部 334 天(决策窗口)的"预报残差" r_{d,t}=(PV^fcst-PV^act)*DT 做
         K-means 场景削减，得到 K 个代表残差与权重（三层保真的第 1、2 层）。

输出：results/result2.xlsx、intermediate/q2.json
"""
from __future__ import annotations

import argparse
import json
import os
import time

import numpy as np
import pandas as pd

import config as C
import lp_engine as LP
import results_io as RIO

DATE_WINDOW = pd.date_range(C.WIN_START, C.WIN_END, freq="D")
DATE_ALL = pd.date_range(C.WARMUP_START, C.WIN_END, freq="D")


def build_residuals(panel, dates):
    """构造预报残差矩阵 r[d,t] = (PV^fcst_0:00 - PV^act) * DT，单位 kWh。"""
    pv_act = panel.PV()
    R = np.zeros((len(dates), C.N_SLOT))
    for i, d in enumerate(dates):
        fc = panel.pv_forecast(d, 0)
        R[i] = (fc - pv_act.loc[d].to_numpy(float)) * C.DT
    return R


def run_q2(K=30, term_mode="ge", verbose=True, info="R3", max_days=None, seed=0,
           scen="strat"):
    panel = C.Panel(DATE_ALL)
    load = panel.L()
    pv_act = panel.PV()
    price_fixed = C.read_attachment1()["price"].to_numpy(float)
    price_flat = panel.P(volatile=False)          # 每天相同
    a1 = C.read_attachment1()
```

```
52
53      # 残差与场景削减（在决策窗口 334 天上做）
54      R_all = build_residuals(panel, DATE_WINDOW)
55      if scen == "zero":                          # 消融：不设安全边际（按预报精确购电）
56          resid_cen = np.zeros((1, C.N_SLOT)); resid_w = np.array([1.0])
57      elif scen == "strat":
58          resid_cen, resid_w = LP.reduce_scenarios_stratified(R_all, K=K)
59      else:
60          resid_cen, resid_w = LP.reduce_scenarios(R_all, K=K, seed=seed)
61      if verbose:
62          print(f"[场景削减] K={len(resid_w)}  权重和={resid_w.sum():.4f}  "
63                f"残差均值绝对值={np.abs(resid_cen).mean():.2f} kWh")
64
65      dates = DATE_ALL if max_days is None else DATE_ALL[:max_days]
66      recs, s_init = [], C.S_INIT
67      t0 = time.time()
68      for i, d in enumerate(dates):
69          nd_act = (load.loc[d].to_numpy(float) - pv_act.loc[d].to_numpy(float)) * C.DT
70          if info == "R3":
71              pv_fc = panel.pv_forecast(d, 0)
72              nd_fcst = (load.loc[d].to_numpy(float) - pv_fc) * C.DT
73          elif info == "R1":                          # 完美信息（下界）
74              nd_fcst = nd_act.copy()
75          elif info == "R2":                          # 附件 1 典型日预报
76              nd_fcst = (a1["load"].to_numpy(float) - a1["pv"].to_numpy(float)) * C.DT
77          else:
78              raise ValueError(info)
79
80          nd_scen = nd_fcst[None, :] + resid_cen
81          r = d in DATE_WINDOW
82          tm = term_mode if r else "ge"
83          sol = LP.solve_day(price_fixed, nd_fcst, nd_scen, resid_w,
84                             s_init=s_init, term_mode=tm)
85          g, pc, ec = LP.realized_slot(sol, nd_act)
86          chk = LP.check_solution(sol, nd_fcst, nd_act)
87          if r:
88              recs.append(dict(
89                  date=d, E=sol["E"].copy(), c=sol["c"].copy(), d=sol["d"].copy(),
90                  S_end=float(sol["S"][-1]), s_init=s_init, g=g,
91                  plan_cost=pc, emg_cost=ec, total_cost=pc + ec,
92                  emg_qty=float(g.sum()), energy=float(sol["E"].sum()),
93                  base_cost=C.baseline_cost(price_fixed, load.loc[d].to_numpy(float),
94                                            pv_act.loc[d].to_numpy(float)),
95                  sae_obj=float(sol["obj"]),
96                  max_viol=max(chk["R1_plan_violation"], chk["R2_power_violation"],
97                               chk["R3_soc_bounds"], chk["R4_soc_recursion"])))
98          s_init = float(sol["S"][-1])
99          if verbose and (i + 1) % 30 == 0:
100             print(f"  ... {i+1}/{len(dates)} 天  累计 {time.time()-t0:.1f}s  "
101                   f"S={s_init:.0f}")
102     return recs, dict(resid_cen=resid_cen, resid_w=resid_w), price_fixed
103
104
105 def to_frames(recs, price_fixed):
106     """整理成 result2 表格所需结构。"""
107     dates = [r["date"] for r in recs]
108     E = np.vstack([r["E"] for r in recs])
109     Cc = np.vstack([r["c"] for r in recs])
110     Dd = np.vstack([r["d"] for r in recs])
111     G = np.vstack([r["g"] for r in recs])
112     S0 = np.array([r["s_init"] for r in recs])
113     S1 = np.array([r["S_end"] for r in recs])
114     return dates, E, Cc, Dd, G, S0, S1
115
116
117 def main():
118     ap = argparse.ArgumentParser()
119     ap.add_argument("--K", type=int, default=60)
120     ap.add_argument("--scen", default="strat", choices=["strat", "kmeans", "zero"])
121     ap.add_argument("--term", default="ge", choices=["ge", "eq", "free"])
122     ap.add_argument("--info", default="R3", choices=["R3", "R1", "R2"])
123     ap.add_argument("--max-days", type=int, default=None)
124     ap.add_argument("--tag", default="main")
```

```
    ap.add_argument("--write", action="store_true")
    args = ap.parse_args()

    recs, scen, price_fixed = run_q2(K=args.K, term_mode=args.term, info=args.info,
                                     max_days=args.max_days, scen=args.scen)
    dates, E, Cc, Dd, G, S0, S1 = to_frames(recs, price_fixed)

    # ---- 写出 result2.xlsx（仅在主口径下写出，避免被对照实验覆盖）----
    if args.write:
        path = os.path.join(C.RESULT_DIR, "result2.xlsx")
        RIO.write_result(path, dates, E, Cc, Dd, S0, S1, price_fixed, g=G)
        print("已写出", path)

    tot_plan = sum(r["plan_cost"] for r in recs)
    tot_emg = sum(r["emg_cost"] for r in recs)
    tot_base = sum(r["base_cost"] for r in recs)
    tot_emg_qty = sum(r["emg_qty"] for r in recs)
    days_with_emg = int(np.sum(G.sum(axis=1) > 1e-6))
    print("=" * 78)
    print(f"  信息结构 {args.info} | 终端条件 {args.term} | 天数 {len(recs)}")
    print(f"  计划购电费         {tot_plan:14.2f} 元")
    print(f"  紧急购电费         {tot_emg:14.2f} 元")
    print(f"  总费用             {tot_plan+tot_emg:14.2f} 元")
    print(f"  无储能基线 C-01    {tot_base:14.2f} 元")
    print(f"  相对基线节省       {tot_base-(tot_plan+tot_emg):14.2f} 元 "
          f"({(tot_base-(tot_plan+tot_emg))/tot_base*100:.2f}%)")
    print(f"  紧急购电量         {tot_emg_qty:14.2f} kWh  出现在 {days_with_emg}/{len(recs)} 天")
    print(f"  最差单日残差       {max(r['max_viol'] for r in recs):.3e}")
    print("=" * 78)

    out = dict(meta=dict(K=args.K, term=args.term, info=args.info, tag=args.tag,
                         scen=args.scen,
                         n_days=len(recs), tot_plan=tot_plan, tot_emg=tot_emg,
                         tot_cost=tot_plan + tot_emg, tot_base=tot_base,
                         tot_emg_qty=tot_emg_qty, days_with_emg=days_with_emg),
               daily=[{k: (v.isoformat() if k == "date" else
                           (v.tolist() if isinstance(v, np.ndarray) else v))
                       for k, v in r.items()} for r in recs],
               scen=dict(cen=scen["resid_cen"].tolist(),
                         w=scen["resid_w"].tolist()))
    with open(os.path.join(C.INTERMEDIATE_DIR, f"q2_{args.tag}.json"), "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    return out


if __name__ == "__main__":
    main()
```

源程序：code/q3.py

```
# -*- coding: utf-8 -*-
"""
问题 3：多预报时刻下的滚动调整（多阶段 / MPC）
================================================
信息结构：
  0:00 用附件 3 的 0:00 预报制定**全天**计划购电量 E^plan（144 槽，第一阶段承诺）；
  6:00/12:00/18:00 用当次预报对**其后至 24:00** 的剩余时段重新决策（后覆盖前），
  实际执行的是"最后覆盖该时段"的那次决策，记为调整购电量 E^adj。
  → 最终：槽 0-35 由 0:00 计划决定，36-71 由 6:00 决定，72-107 由 12:00 决定，
        108-143 由 18:00 决定（非预期性自然满足：每个决策只用该时刻及之前的预报）。

费用口径 C-06（偏差定价）：
  cost_t = p_t*min(E^plan,E^adj) + 0.5 p_t (E^plan-E^adj)^+ + 1.5 p_t (E^adj-E^plan)^+
  另加紧急购电 5 p_t g_t。第九章给出"计划费全额 + 差额惩罚"另一口径的对照。

VoI 实验：预报时刻数 n ∈{1,2,4,∞}，同一场景集、同一实际轨迹回放。
"""
from __future__ import annotations

import argparse
import json
import os
import time
```

```python

import numpy as np
import pandas as pd

import config as C
import lp_engine as LP
import results_io as RIO

DATE_WINDOW = pd.date_range(C.WIN_START, C.WIN_END, freq="D")
DATE_ALL = pd.date_range(C.WARMUP_START, C.WIN_END, freq="D")


def forecast_slots(panel, date, hour, slots):
    """取 date 日 hour 时刻发布的预报在给定槽位上的值（kW）。"""
    row = panel.pf3.loc[(pd.Timestamp(date).normalize(), hour)]
    hourly = np.array([row[f"lead{k}"] for k in range(1, 25)], dtype=float)
    h = np.array([t // 6 for t in slots])          # 槽位所属整点（0..23）
    lead = (h - hour) % 24                         # 该整点对应的预报提前期
    return hourly[lead]


def build_epoch_scenarios(panel, hour, K=40, slots=None, dates=DATE_WINDOW):
    """构造"hour 时刻预报"的残差场景集（仅覆盖 slots），按窗口总残差分层削减。"""
    pv_act = panel.PV()
    slots = np.arange(C.N_SLOT) if slots is None else np.asarray(slots)
    R = np.zeros((len(dates), len(slots)))
    for i, d in enumerate(dates):
        fc = forecast_slots(panel, d, hour, slots)
        R[i] = (fc - pv_act.loc[d].to_numpy(float)[slots]) * C.DT
    cen, w = LP.reduce_scenarios_stratified(R, K=K)
    return cen, w


def run_q3(epochs=(0, 6, 12, 18), K=40, term_mode="ge", verbose=True,
           perfect=False, max_days=None, tag="main", seed=0):
    panel = C.Panel(DATE_ALL)
    load = panel.L(); pv_act = panel.PV()
    price = C.read_attachment1()["price"].to_numpy(float)

    scens = {}
    for h in epochs:
        s_m = C.EPOCH_SLOT[h]
        slots = np.arange(s_m, C.N_SLOT)
        cen, w = build_epoch_scenarios(panel, h, K=K, slots=slots)
        scens[h] = dict(slots=slots, cen=cen, w=w)
        if verbose:
            print(f"[场景] 时刻 {h:>2}:00  槽位 {s_m}..143  代表场景 {len(w)}  "
                  f"残差均值绝对值 {np.abs(cen).mean():.2f} kWh")

    dates = DATE_ALL if max_days is None else DATE_ALL[:max_days]
    recs, s_init = [], C.S_INIT
    t0 = time.time()
    for i, d in enumerate(dates):
        L = load.loc[d].to_numpy(float)
        PVa = pv_act.loc[d].to_numpy(float)
        nd_act = (L - PVa) * C.DT
        s_day = s_init
        E_plan = np.zeros(C.N_SLOT); E_adj = np.zeros(C.N_SLOT)
        cc = np.zeros(C.N_SLOT); dd = np.zeros(C.N_SLOT); SS = np.zeros(C.N_SLOT)
        soc = s_init
        for h in epochs:
            s_m = C.EPOCH_SLOT[h]
            slots = scens[h]["slots"]
            if perfect and h > 0:
                nd_fc = nd_act.copy()
            else:
                fc = forecast_slots(panel, d, h, slots)
                nd_fc = np.zeros(C.N_SLOT)
                nd_fc[slots] = (L[slots] - fc) * C.DT
            cen, w = scens[h]["cen"], scens[h]["w"]
            nd_sc = np.zeros((cen.shape[0], C.N_SLOT))
            nd_sc[:, slots] = nd_fc[slots][None, :] + cen
            sol = LP.solve_epoch(price, nd_fc, nd_sc, w, s_init=soc, s_from=s_m,
```

```
                                plan=(E_plan if h > 0 else None), term_floor=s_day)
            E = sol["E"]
            if h == 0:
                E_plan = E.copy()
            nxt = C.N_SLOT if h == np.max(epochs) else C.EPOCH_SLOT[
                [x for x in epochs if x > h][0]]
            for j, t in enumerate(range(s_m, nxt)):
                E_adj[t] = E[j]; cc[t] = sol["c"][j]; dd[t] = sol["d"][j]
                SS[t] = sol["S"][j]
            soc = float(SS[nxt - 1])
        # ---- 结算（用实际数据）----
        g = np.maximum(nd_act + cc - dd - E_adj, 0.0)
        dev = E_adj - E_plan
        u = np.maximum(-dev, 0.0); v = np.maximum(dev, 0.0)
        cost_buy = float(np.sum(price * (np.minimum(E_plan, E_adj) + 0.0)))
        cost_breach = float(np.sum(C.BREACH_MULT * price * u))
        cost_excess = float(np.sum(C.EXCESS_MULT * price * v))
        cost_emg = float(C.EMERGENCY_MULT * np.sum(price * g))
        is_win = d in DATE_WINDOW
        if is_win:
            recs.append(dict(
                date=d, E_plan=E_plan.copy(), E_adj=E_adj.copy(), c=cc.copy(),
                d=dd.copy(), S=SS.copy(), g=g, s_init=s_day, s_end=float(SS[-1]),
                cost_buy=cost_buy, cost_breach=cost_breach, cost_excess=cost_excess,
                cost_emg=cost_emg,
                total_cost=cost_buy + cost_breach + cost_excess + cost_emg,
                plan_energy=float(E_plan.sum()), adj_energy=float(E_adj.sum()),
                emg_qty=float(g.sum()),
                breach_qty=float(u.sum()), excess_qty=float(v.sum()),
                base_cost=C.baseline_cost(price, L, PVa),
                max_viol=float(max(abs(nd_act + cc - dd - E_adj).max(), 0.0))))
        s_init = soc
        if verbose and (i + 1) % 40 == 0:
            print(f"  ... {i+1}/{len(dates)} 天  {time.time()-t0:.1f}s")
    return recs, scens, price


def summary(recs, price, tag=""):
    tot = {k: sum(r[k] for r in recs) for k in
           ("cost_buy", "cost_breach", "cost_excess", "cost_emg", "total_cost",
            "plan_energy", "adj_energy", "emg_qty", "breach_qty", "excess_qty",
            "base_cost")}
    tot["n_days"] = len(recs)
    tot["days_with_emg"] = int(sum(1 for r in recs if r["emg_qty"] > 1e-6))
    tot["tag"] = tag
    return tot


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--epochs", default="0,6,12,18")
    ap.add_argument("--K", type=int, default=40)
    ap.add_argument("--perfect", action="store_true")
    ap.add_argument("--tag", default="n4")
    ap.add_argument("--max-days", type=int, default=None)
    ap.add_argument("--write", action="store_true")
    args = ap.parse_args()
    epochs = tuple(int(x) for x in args.epochs.split(","))
    recs, scens, price = run_q3(epochs=epochs, K=args.K, perfect=args.perfect,
                                tag=args.tag, max_days=args.max_days)
    tot = summary(recs, price, args.tag)
    for k in ("cost_buy", "cost_breach", "cost_excess", "cost_emg", "total_cost",
              "base_cost"):
        print(f"  {k:14s} {tot[k]:16.2f}")
    print(f"  总费用 {tot['total_cost']:.2f} 元 ({tot['total_cost']/1e4:.2f} 万元) "
          f"节省率 {(tot['base_cost']-tot['total_cost'])/tot['base_cost']*100:.2f}% "
          f"紧急 {tot['emg_qty']:.0f} kWh {tot['days_with_emg']}/{tot['n_days']} 天 "
          f"违约 {tot['breach_qty']:.0f} 超额 {tot['excess_qty']:.0f} kWh")
    out = dict(meta=tot, daily=[{k: (v.isoformat() if k == "date" else
                                     (v.tolist() if isinstance(v, np.ndarray) else v))
                                for k, v in r.items()} for r in recs])
    with open(os.path.join(C.INTERMEDIATE_DIR, f"q3_{args.tag}.json"), "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
```

```
    if args.write and not args.perfect:
        dates = [r["date"] for r in recs]
        path = os.path.join(C.RESULT_DIR, "result3.xlsx")
        RIO.write_result(path, dates,
                         np.vstack([r["E_plan"] for r in recs]),
                         np.vstack([r["c"] for r in recs]),
                         np.vstack([r["d"] for r in recs]),
                         np.array([r["s_init"] for r in recs]),
                         np.array([r["s_end"] for r in recs]),
                         price, g=np.vstack([r["g"] for r in recs]),
                         E_adj=np.vstack([r["E_adj"] for r in recs]))
        print("已写出", path)
    return out


if __name__ == "__main__":
    main()
```

源程序：code/q4.py

```
# -*- coding: utf-8 -*-
"""
问题 4：波动电价下的购电策略
============================
(1) 电价过程四项检验 C1–C4（正态性/自相关/方差结构/与负载光伏的相关性）；
(2) 两层电价建模：分时骨架（附件 1 均值剖面）+ 随机残差（附件 4 − 骨架）；
(3) 联合情景：以"代表性历史日"为主（价格剖面与净需求残差取自同一天，
    天然保留 rho(p, L)），并以分块 Bootstrap 合成情景作条件对照；
(4) CVaR 风险—费用前沿 + 真实 334 天轨迹回测；
(5) 在波动电价下重算问题 2（result4-2.xlsx）与问题 3（result4-3.xlsx）。

注意：附件 4 只给实际电价、未给日前电价预报，故假设"0:00 可用的电价预报 = 分时骨架"，
     结算按附件 4 的实际电价。该假设在假设节显式声明（A10）。
"""
from __future__ import annotations

import argparse
import json
import os
import time

import numpy as np
import pandas as pd
from scipy import stats
from scipy.optimize import linprog
from scipy.sparse import coo_matrix

import config as C
import lp_engine as LP
import results_io as RIO
import q3 as Q3

DATE_WINDOW = pd.date_range(C.WIN_START, C.WIN_END, freq="D")
DATE_ALL = pd.date_range(C.WARMUP_START, C.WIN_END, freq="D")


# ---------------------------------------------------------------------------
# 通用：电价场景下的单日 LP（计划/调整两用）
# ---------------------------------------------------------------------------
def solve_vol(price_scen, nd_fcst, nd_scen, weights=None, s_init=C.S_INIT,
              s_from=0, plan=None, term_floor=None):
    """price_scen: (K,n) 各情景实际电价；plan 非空时按偏差定价口径。"""
    n = C.N_SLOT
    price_scen = np.atleast_2d(price_scen)
    K = price_scen.shape[0]
    if weights is None:
        weights = np.full(K, 1.0 / K)
    weights = np.asarray(weights, float); weights = weights / weights.sum()
    m = n - s_from
    use_plan = plan is not None and s_from > 0
    iE, iC, iD, iS, iG = 0, m, 2 * m, 3 * m, 4 * m
    iU = iG + K * m
    nv = 4 * m + K * m + (2 * m if use_plan else 0)
```

```
54
55      pw = price_scen[:, s_from:]                     # (K,m)
56      c_obj = np.zeros(nv)
57      if use_plan:
58          c_obj[iU:iU + m] = -C.BREACH_MULT * (weights[:, None] * pw).sum(0)
59          c_obj[iU + m:iU + 2 * m] = C.EXCESS_MULT * (weights[:, None] * pw).sum(0)
60      else:
61          c_obj[iE:iE + m] = (weights[:, None] * pw).sum(0)
62      c_obj[iG:iG + K * m] = C.EMERGENCY_MULT * (weights[:, None] * pw).ravel()
63
64      rows, cols, vals, rhs = [], [], [], []
65      r = 0
66      for j in range(m):
67          for a, b, c_ in ((r, iE + j, -1.0), (r, iC + j, 1.0), (r, iD + j, -1.0)):
68              rows.append(a); cols.append(b); vals.append(c_)
69          rhs.append(-nd_fcst[s_from + j]); r += 1
70      for w in range(K):
71          for j in range(m):
72              for a, b, c_ in ((r, iE + j, -1.0), (r, iC + j, 1.0),
73                               (r, iD + j, -1.0), (r, iG + w * m + j, -1.0)):
74                  rows.append(a); cols.append(b); vals.append(c_)
75              rhs.append(-nd_scen[w, s_from + j]); r += 1
76      if use_plan:
77          for j in range(m):
78              for a, b, c_ in ((r, iE + j, 1.0), (r, iU + m + j, -1.0),
79                               (r, iU + j, 1.0)):
80                  rows.append(a); cols.append(b); vals.append(c_)
81              rhs.append(plan[s_from + j]); r += 1
82      A_ub = coo_matrix((vals, (rows, cols)), shape=(r, nv)).tocsr()
83      b_ub = np.array(rhs)
84
85      rows2, cols2, vals2, rhs2 = [], [], [], []
86      for j in range(m):
87          for b, c_ in ((iS + j, 1.0), (iC + j, -C.ETA_C), (iD + j, 1.0 / C.ETA_D)):
88              rows2.append(j); cols2.append(b); vals2.append(c_)
89          if j > 0:
90              rows2.append(j); cols2.append(iS + j - 1); vals2.append(-1.0)
91          rhs2.append(0.0 if j > 0 else s_init)
92      A_eq = coo_matrix((vals2, (rows2, cols2)), shape=(m, nv)).tocsr()
93      b_eq = np.array(rhs2)
94
95      bounds = [(0.0, None)] * m + [(0.0, C.E_MAX)] * m + [(0.0, C.E_MAX)] * m + \
96               [(C.S_MIN, C.S_MAX)] * m + [(0.0, None)] * (K * m) + \
97               ([(0.0, None)] * (2 * m) if use_plan else [])
98      if term_floor is not None:
99          bounds[iS + m - 1] = (max(term_floor, C.S_MIN), C.S_MAX)
100     res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
101                   bounds=bounds, method="highs")
102     if not res.success:
103         raise RuntimeError("volatile LP 不可行: " + res.message)
104     x = res.x
105     return dict(E=x[iE:iE + m].copy(), c=x[iC:iC + m].copy(),
106                 d=x[iD:iD + m].copy(), S=x[iS:iS + m].copy(),
107                 g=x[iG:iG + K * m].reshape(K, m).copy(), obj=res.fun, s_from=s_from)
108
109
110 # ---------------------------------------------------------------------------
111 # 电价过程检验与两层建模
112 # ---------------------------------------------------------------------------
113 def price_tests(panel):
114     a1 = C.read_attachment1()
115     p4 = panel.P(volatile=True).to_numpy(float)
116     skel = a1["price"].to_numpy(float)
117     resid = p4 - skel[None, :]
118     load = panel.L().to_numpy(float); pv = panel.PV().to_numpy(float)
119     out = {}
120     out["skeleton_is_mean"] = float(np.max(np.abs(p4.mean(0) - skel)))
121     out["price_mean"] = float(p4.mean()); out["price_sd"] = float(p4.std())
122     out["price_min"] = float(p4.min()); out["price_max"] = float(p4.max())
123     out["resid_sd_within"] = float(resid.std(0).mean())
124     out["cv_within"] = float((resid.std(0) / skel).mean())
125     # C1 正态性（标准化残差）
126     z = resid / resid.std(0, keepdims=True)
```

```
    zz = z.ravel()
    jb = stats.jarque_bera(zz)
    out["C1_JB_stat"] = float(jb.statistic); out["C1_JB_p"] = float(jb.pvalue)
    out["C1_skew"] = float(stats.skew(zz)); out["C1_kurt"] = float(stats.kurtosis(zz))
    # C2 自相关（日内残差序列，滞后 1..6）
    acf = [float(np.mean([np.corrcoef(resid[i, :-k], resid[i, k:])[0, 1]
                          for i in range(len(resid))])) for k in range(1, 7)]
    out["C2_acf_lag1_6"] = acf
    # C2b 日间自相关（同一槽位跨日）
    acf_day = [float(np.mean([np.corrcoef(resid[:-k, t], resid[k:, t])[0, 1]
                              for t in range(C.N_SLOT)])) for k in range(1, 4)]
    out["C2b_daily_acf"] = acf_day
    # C3 方差结构：各槽位残差标准差（谷/峰时段对比）
    sd_slot = resid.std(0)
    hrs = np.array([t // 6 for t in range(C.N_SLOT)])
    out["C3_sd_night"] = float(sd_slot[hrs == 3].mean())      # 03:00-04:00 谷段
    out["C3_sd_peak"] = float(sd_slot[hrs == 19].mean())      # 19:00-20:00 峰段
    out["C3_sd_ratio"] = out["C3_sd_peak"] / out["C3_sd_night"]
    out["C3_sd_all"] = float(sd_slot.mean())
    # C4 相关性（池化 / 日聚合 / 去形状）
    net = load - pv
    out["C4_pool_pL"] = float(np.corrcoef(p4.ravel(), load.ravel())[0, 1])
    out["C4_pool_pNet"] = float(np.corrcoef(p4.ravel(), net.ravel())[0, 1])
    out["C4_pool_pPV"] = float(np.corrcoef(p4.ravel(), pv.ravel())[0, 1])
    out["C4_day_pL"] = float(np.corrcoef(p4.mean(1), load.mean(1))[0, 1])
    out["C4_day_pPV"] = float(np.corrcoef(p4.mean(1), pv.mean(1))[0, 1])
    dev_p = p4 - p4.mean(1, keepdims=True); dev_l = load - load.mean(1, keepdims=True)
    out["C4_deshape_pL"] = float(np.corrcoef(dev_p.ravel(), dev_l.ravel())[0, 1])
    return out, resid, skel


def build_joint_scenarios(panel, K=60, mode="hist", seed=7, block=36):
    """联合情景：mode='hist' 取代表性历史日；mode='boot' 分块 Bootstrap 合成。"""
    load = panel.L().to_numpy(float); pv = panel.PV().to_numpy(float)
    p4 = panel.P(volatile=True).to_numpy(float)
    idx = [list(DATE_ALL).index(d) for d in DATE_WINDOW]
    res_nd = np.zeros((len(idx), C.N_SLOT))
    net_nd = np.zeros((len(idx), C.N_SLOT))
    for i, k in enumerate(idx):
        fc = panel.pv_forecast(DATE_WINDOW[i], 0)
        res_nd[i] = (fc - pv[k]) * C.DT
        net_nd[i] = (load[k] - pv[k]) * C.DT
    if mode == "hist":
        # 二维分层：按"日净需求残差"与"日均电价"各分层（网格），层内取 medoid，
        # 权重 = 层样本占比 → 同时保真净需求残差与电价的边际分布，并保留联合结构。
        np_str = int(np.floor(np.sqrt(K)))
        nd_tot = res_nd.sum(1)
        p_avg = p4[idx].mean(1)
        q1 = np.quantile(nd_tot, np.linspace(0, 1, np_str + 1))
        q2 = np.quantile(p_avg, np.linspace(0, 1, np_str + 1))
        b1 = np.clip(np.digitize(nd_tot, q1[1:-1]), 0, np_str - 1)
        b2 = np.clip(np.digitize(p_avg, q2[1:-1]), 0, np_str - 1)
        cell = b1 * np_str + b2
        pos, w = [], []
        for cell_id in range(np_str * np_str):
            g = np.where(cell == cell_id)[0]
            if len(g) == 0:
                continue
            sub = res_nd[g]
            if len(g) == 1:
                pos.append(g[0])
            else:
                dist = np.sqrt(((sub[:, None, :] - sub[None, :, :]) ** 2).sum(-1))
                pos.append(g[int(np.argmin(dist.sum(1)))])
            w.append(len(g) / len(nd_tot))
        pos = np.array(pos); w = np.array(w)
        return dict(price=p4[idx][pos], resid=res_nd[pos], w=w,
                    net=net_nd[pos], src=np.array(DATE_WINDOW)[pos])
    rng = np.random.default_rng(seed)
    nblk = C.N_SLOT // block
    price_s, resid_s = [], []
    for _ in range(K):
        pr, rr = [], []
```

```
        for j in range(nblk):                       # 块按时刻对齐重采样，保住日内形状
            aa = int(rng.integers(0, len(idx)))
            k = idx[aa]; off = j * block
            pr.append(p4[k, off:off + block])
            fc = panel.pv_forecast(DATE_WINDOW[aa], 0)[off:off + block]
            rr.append((fc - pv[k, off:off + block]) * C.DT)
        price_s.append(np.concatenate(pr)); resid_s.append(np.concatenate(rr))
    return dict(price=np.array(price_s), resid=np.array(resid_s),
                w=np.full(K, 1.0 / K), net=None, src=None)


def fidelity(scen, panel, price_tests_out):
    """情景保真度：均值/分位数差、相关系数保持。"""
    p4 = panel.P(volatile=True).to_numpy(float)
    idx = [list(DATE_ALL).index(d) for d in DATE_WINDOW]
    p_true = p4[idx].ravel()
    p_s = scen["price"].ravel()
    qs = [1, 5, 25, 50, 75, 95, 99]
    qt = np.percentile(p_true, qs); qsi = np.percentile(p_s, qs)
    out = dict(mean_gap=float(abs(p_s.mean() - p_true.mean()) / p_true.mean()),
               max_q_gap=float(np.max(np.abs(qsi - qt) / np.abs(qt))),
               q_levels=qs, q_true=qt.tolist(), q_scen=qsi.tolist(),
               q_gap_rel=[float(abs(a - b) / abs(b)) for a, b in zip(qsi, qt)])
    load = panel.L().to_numpy(float)[idx]
    net = load - panel.PV().to_numpy(float)[idx]
    out["corr_true_pL"] = float(np.corrcoef(p_true, load.ravel())[0, 1])
    out["corr_true_pNet"] = float(np.corrcoef(p_true, net.ravel())[0, 1])
    if scen.get("net") is not None:
        w = scen["w"] / scen["w"].sum()
        rep = np.maximum((w * len(p_true)).round().astype(int), 0)
        ps, ns = [], []
        for k, rr in enumerate(rep):
            if rr == 0:
                continue
            ps.append(np.repeat(scen["price"][k][None, :], rr, 0))
            ns.append(np.repeat(scen["net"][k][None, :], rr, 0))
        ps = np.vstack(ps).ravel(); ns = np.vstack(ns).ravel()
        out["corr_scen_pNet"] = float(np.corrcoef(ps, ns)[0, 1])
    else:
        out["corr_scen_pNet"] = float("nan")
    return out

# ---------------------------------------------------------------------------
# 波动电价下重算问题 2（Q4-2）
# ---------------------------------------------------------------------------
def run_q4_2(scen, verbose=True, max_days=None, write=False, tag="q42"):
    panel = C.Panel(DATE_ALL)
    load = panel.L(); pv_act = panel.PV()
    p4 = panel.P(volatile=True)
    skel = C.read_attachment1()["price"].to_numpy(float)
    dates = DATE_ALL if max_days is None else DATE_ALL[:max_days]
    recs, s_init = [], C.S_INIT
    t0 = time.time()
    for i, d in enumerate(dates):
        L = load.loc[d].to_numpy(float); PVa = pv_act.loc[d].to_numpy(float)
        nd_act = (L - PVa) * C.DT
        fc = panel.pv_forecast(d, 0)
        nd_fcst = (L - fc) * C.DT
        nd_scen = nd_fcst[None, :] + scen["resid"]
        sol = solve_vol(scen["price"], nd_fcst, nd_scen, scen["w"], s_init=s_init,
                        s_from=0, term_floor=s_init)
        g = np.maximum(nd_act + sol["c"] - sol["d"] - sol["E"], 0.0)
        pact = p4.loc[d].to_numpy(float)
        buy = float(np.sum(pact * sol["E"]))
        emg = float(C.EMERGENCY_MULT * np.sum(pact * g))
        if d in DATE_WINDOW:
            recs.append(dict(date=d, E=sol["E"].copy(), c=sol["c"].copy(),
                             d=sol["d"].copy(), S=sol["S"].copy(), g=g,
                             s_init=s_init, s_end=float(sol["S"][-1]),
                             cost_buy=buy, cost_emg=emg, total_cost=buy + emg,
                             emg_qty=float(g.sum()), plan_energy=float(sol["E"].sum()),
                             base_cost=C.baseline_cost(pact, L, PVa),
                             base_cost_skel=C.baseline_cost(skel, L, PVa)))
```

```
        s_init = float(sol["S"][-1])
        if verbose and (i + 1) % 60 == 0:
            print(f"  ... {i+1}/{len(dates)} 天 {time.time()-t0:.1f}s")
    if write:
        dates_w = [r["date"] for r in recs]
        path = os.path.join(C.RESULT_DIR, "result4-2.xlsx")
        RIO.write_result(path, dates_w,
                         np.vstack([r["E"] for r in recs]),
                         np.vstack([r["c"] for r in recs]),
                         np.vstack([r["d"] for r in recs]),
                         np.array([r["s_init"] for r in recs]),
                         np.array([r["s_end"] for r in recs]),
                         skel, g=np.vstack([r["g"] for r in recs]),
                         price_mat=p4.loc[dates_w].to_numpy(float))
        print("已写出", path)
    return recs


# ---------------------------------------------------------------------------
# 波动电价下重算问题 3（Q4-3）：滚动调整
# ---------------------------------------------------------------------------
def run_q4_3(scen, epochs=(0, 6, 12, 18), K=60, verbose=True, max_days=None,
             write=False, perfect_price=False, tag="q43"):
    panel = C.Panel(DATE_ALL)
    load = panel.L(); pv_act = panel.PV()
    p4 = panel.P(volatile=True)
    skel = C.read_attachment1()["price"].to_numpy(float)
    # 各预报时刻的净需求残差场景（该时刻预报 vs 实际）
    scens = {}
    for h in epochs:
        s_m = C.EPOCH_SLOT[h]
        slots = np.arange(s_m, C.N_SLOT)
        cen, w = Q3.build_epoch_scenarios(panel, h, K=K, slots=slots)
        scens[h] = dict(slots=slots, cen=cen, w=w)
    dates = DATE_ALL if max_days is None else DATE_ALL[:max_days]
    recs, s_init = [], C.S_INIT
    t0 = time.time()
    for i, d in enumerate(dates):
        L = load.loc[d].to_numpy(float); PVa = pv_act.loc[d].to_numpy(float)
        nd_act = (L - PVa) * C.DT
        pact = p4.loc[d].to_numpy(float)
        ps = pact[None, :] if perfect_price else scen["price"]
        s_day = s_init
        E_plan = np.zeros(C.N_SLOT); E_adj = np.zeros(C.N_SLOT)
        cc = np.zeros(C.N_SLOT); dd = np.zeros(C.N_SLOT)
        soc = s_init
        for h in epochs:
            s_m = C.EPOCH_SLOT[h]; slots = scens[h]["slots"]
            if perfect_price and h > 0:
                price_sc = pact[None, :]
            else:
                price_sc = ps
            fc = Q3.forecast_slots(panel, d, h, slots)
            nd_fc = np.zeros(C.N_SLOT); nd_fc[slots] = (L[slots] - fc) * C.DT
            cen = scens[h]["cen"]
            nd_sc = np.zeros((cen.shape[0], C.N_SLOT))
            nd_sc[:, slots] = nd_fc[slots][None, :] + cen
            sol = solve_vol(price_sc, nd_fc, nd_sc, scens[h]["w"], s_init=soc,
                            s_from=s_m, plan=(E_plan if h > 0 else None),
                            term_floor=s_day)
            E = sol["E"]
            if h == 0:
                E_plan = E.copy()
            later = [x for x in epochs if x > h]
            nxt = C.EPOCH_SLOT[later[0]] if later else C.N_SLOT
            for j, t in enumerate(range(s_m, nxt)):
                E_adj[t] = E[j]; cc[t] = sol["c"][j]; dd[t] = sol["d"][j]
            soc = float(sol["S"][nxt - s_m - 1])
        g = np.maximum(nd_act + cc - dd - E_adj, 0.0)
        dev = E_adj - E_plan
        u = np.maximum(-dev, 0.0); v = np.maximum(dev, 0.0)
        cost_buy = float(np.sum(pact * np.minimum(E_plan, E_adj)))
        cost_breach = float(C.BREACH_MULT * np.sum(pact * u))
```

```
        cost_excess = float(C.EXCESS_MULT * np.sum(pact * v))
        cost_emg = float(C.EMERGENCY_MULT * np.sum(pact * g))
        if d in DATE_WINDOW:
            recs.append(dict(date=d, E_plan=E_plan.copy(), E_adj=E_adj.copy(),
                             c=cc.copy(), d=dd.copy(), g=g, s_init=s_day,
                             s_end=soc, cost_buy=cost_buy, cost_breach=cost_breach,
                             cost_excess=cost_excess, cost_emg=cost_emg,
                             total_cost=cost_buy + cost_breach + cost_excess + cost_emg,
                             emg_qty=float(g.sum()), plan_energy=float(E_plan.sum()),
                             adj_energy=float(E_adj.sum()), breach_qty=float(u.sum()),
                             excess_qty=float(v.sum()),
                             base_cost=C.baseline_cost(pact, L, PVa)))
        s_init = soc
        if verbose and (i + 1) % 60 == 0:
            print(f"  ... {i+1}/{len(dates)} 天 {time.time()-t0:.1f}s")
    if write:
        dates_w = [r["date"] for r in recs]
        path = os.path.join(C.RESULT_DIR, "result4-3.xlsx")
        RIO.write_result(path, dates_w,
                         np.vstack([r["E_plan"] for r in recs]),
                         np.vstack([r["c"] for r in recs]),
                         np.vstack([r["d"] for r in recs]),
                         np.array([r["s_init"] for r in recs]),
                         np.array([r["s_end"] for r in recs]),
                         skel, g=np.vstack([r["g"] for r in recs]),
                         E_adj=np.vstack([r["E_adj"] for r in recs]),
                         price_mat=p4.loc[dates_w].to_numpy(float))
        print("已写出", path)
    return recs


# ---------------------------------------------------------------------------
# CVaR 风险—费用前沿（Rockafellar–Uryasev 线性化）
# ---------------------------------------------------------------------------
def solve_cvar_day(price_scen, nd_fcst, nd_scen, weights, s_init, lam=0.0,
                   alpha=0.95, term_floor=None):
    n = C.N_SLOT
    K = price_scen.shape[0]
    weights = np.asarray(weights, float); weights = weights / weights.sum()
    iE, iC, iD, iS, iG, iZ, iEta = 0, n, 2 * n, 3 * n, 4 * n, 4 * n + K * n, 4 * n + K * n + K
    nv = iEta + 1
    coef = (1 - lam) * weights[:, None] * price_scen
    c_obj = np.zeros(nv)
    c_obj[iE:iE + n] = coef.sum(0)
    c_obj[iG:iG + K * n] = C.EMERGENCY_MULT * coef.ravel()
    c_obj[iZ:iZ + K] = lam * weights / (1 - alpha)
    c_obj[iEta] = lam
    rows, cols, vals, rhs = [], [], [], []
    r = 0
    for t in range(n):
        for a, b, c_ in ((r, iE + t, -1.0), (r, iC + t, 1.0), (r, iD + t, -1.0)):
            rows.append(a); cols.append(b); vals.append(c_)
        rhs.append(-nd_fcst[t]); r += 1
    for w in range(K):
        for t in range(n):
            for a, b, c_ in ((r, iE + t, -1.0), (r, iC + t, 1.0),
                             (r, iD + t, -1.0), (r, iG + w * n + t, -1.0)):
                rows.append(a); cols.append(b); vals.append(c_)
            rhs.append(-nd_scen[w, t]); r += 1
    for w in range(K):          # z_w + eta >= cost_w  ->  cost_w - z_w - eta <= 0
        for t in range(n):
            rows.append(r); cols.append(iE + t); vals.append(price_scen[w, t])
            rows.append(r); cols.append(iG + w * n + t); vals.append(C.EMERGENCY_MULT * price_scen[w, t
    ])
        rows.append(r); cols.append(iZ + w); vals.append(-1.0)
        rows.append(r); cols.append(iEta); vals.append(-1.0)
        rhs.append(0.0); r += 1
    A_ub = coo_matrix((vals, (rows, cols)), shape=(r, nv)).tocsr()
    b_ub = np.array(rhs)
    rows2, cols2, vals2, rhs2 = [], [], [], []
    for t in range(n):
        for b, c_ in ((iS + t, 1.0), (iC + t, -C.ETA_C), (iD + t, 1.0 / C.ETA_D)):
            rows2.append(t); cols2.append(b); vals2.append(c_)
```

```
        if t > 0:
            rows2.append(t); cols2.append(iS + t - 1); vals2.append(-1.0)
        rhs2.append(0.0 if t > 0 else s_init)
    A_eq = coo_matrix((vals2, (rows2, cols2)), shape=(n, nv)).tocsr()
    b_eq = np.array(rhs2)
    bounds = [(0.0, None)] * n + [(0.0, C.E_MAX)] * n + [(0.0, C.E_MAX)] * n + \
             [(C.S_MIN, C.S_MAX)] * n + [(0.0, None)] * (K * n) + \
             [(0.0, None)] * K + [(None, None)]
    if term_floor is not None:
        bounds[iS + n - 1] = (max(term_floor, C.S_MIN), C.S_MAX)
    res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                  bounds=bounds, method="highs")
    if not res.success:
        raise RuntimeError("CVaR LP 不可行: " + res.message)
    x = res.x
    return dict(E=x[iE:iE + n], c=x[iC:iC + n], d=x[iD:iD + n], S=x[iS:iS + n],
                g=x[iG:iG + K * n].reshape(K, n), eta=float(x[iEta]),
                obj=res.fun, z=x[iZ:iZ + K])


# ---------------------------------------------------------------------------
# 主程序
# ---------------------------------------------------------------------------
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--mode", default="tests",
                    choices=["tests", "q42", "q43", "cvar", "all"])
    ap.add_argument("--K", type=int, default=60)
    ap.add_argument("--scen", default="hist", choices=["hist", "boot"])
    ap.add_argument("--write", action="store_true")
    ap.add_argument("--max-days", type=int, default=None)
    ap.add_argument("--lams", default="0,0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9,1.0")
    ap.add_argument("--step", type=int, default=6, help="CVaR 前沿子样本步长")
    args = ap.parse_args()

    panel = C.Panel(DATE_ALL)
    out = {}
    if args.mode in ("tests", "all"):
        pt, resid, skel = price_tests(panel)
        scen_h = build_joint_scenarios(panel, K=args.K, mode="hist")
        scen_b = build_joint_scenarios(panel, K=args.K, mode="boot")
        fid_h = fidelity(scen_h, panel, pt)
        fid_b = fidelity(scen_b, panel, pt)
        print("=" * 78)
        for k, v in pt.items():
            print("  %-22s %s" % (k, np.round(v, 3) if isinstance(v, list) else v))
        print("  保真度(历史代表日):", {k: (round(v, 4) if isinstance(v, float) else v)
                                      for k, v in fid_h.items() if k not in ("q_true", "q_scen")})
        print("  保真度(分块Bootstrap):", {k: (round(v, 4) if isinstance(v, float) else v)
                                        for k, v in fid_b.items() if k not in ("q_true", "q_scen")})
        print("=" * 78)
        out["price_tests"] = pt
        out["fidelity_hist"] = fid_h
        out["fidelity_boot"] = fid_b
        with open(os.path.join(C.INTERMEDIATE_DIR, "q4_price.json"), "w") as f:
            json.dump(out, f, ensure_ascii=False, indent=1)

    if args.mode in ("q42", "q43", "all"):
        scen = build_joint_scenarios(panel, K=args.K, mode=args.scen)
        if args.mode in ("q42", "all"):
            recs = run_q4_2(scen, max_days=args.max_days, write=args.write, tag="q42")
            tot = dict(cost_buy=sum(r["cost_buy"] for r in recs),
                       cost_emg=sum(r["cost_emg"] for r in recs),
                       total=sum(r["total_cost"] for r in recs),
                       base=sum(r["base_cost"] for r in recs),
                       emg_qty=sum(r["emg_qty"] for r in recs),
                       days_emg=int(sum(1 for r in recs if r["emg_qty"] > 1e-6)))
            print("Q4-2:", {k: (round(v, 2) if isinstance(v, float) else v) for k, v in tot.items()})
            print("  节省率 %.2f%%" % ((tot["base"] - tot["total"]) / tot["base"] * 100))
            with open(os.path.join(C.INTERMEDIATE_DIR, "q4_2.json"), "w") as f:
                json.dump(dict(meta=tot, daily=[{k: (v.isoformat() if k == "date" else
                                                     (v.tolist() if isinstance(v, np.ndarray) else v))
                                                 for k, v in r.items()} for r in recs]),
```

```
                        f, ensure_ascii=False, indent=1)
        if args.mode in ("q43", "all"):
            recs3 = run_q4_3(scen, K=args.K, max_days=args.max_days, write=args.write)
            tot3 = {k: sum(r[k] for r in recs3) for k in
                    ("cost_buy", "cost_breach", "cost_excess", "cost_emg", "total_cost",
                     "base_cost", "emg_qty", "breach_qty", "excess_qty")}
            print("Q4-3:", {k: (round(v, 2) if isinstance(v, float) else v) for k, v in tot3.items()})
            print("  节省率 %.2f%%" % ((tot3["base_cost"] - tot3["total_cost"]) / tot3["base_cost"] *
    100))
            with open(os.path.join(C.INTERMEDIATE_DIR, "q4_3.json"), "w") as f:
                json.dump(dict(meta=tot3, daily=[{k: (v.isoformat() if k == "date" else
                                                      (v.tolist() if isinstance(v, np.ndarray) else v))
                                                  for k, v in r.items()} for r in recs3]),
                        f, ensure_ascii=False, indent=1)

    if args.mode in ("cvar", "all"):
        scen = build_joint_scenarios(panel, K=args.K, mode=args.scen)
        lams = [float(x) for x in args.lams.split(",")]
        sub = list(DATE_WINDOW)[::args.step]
        load = panel.L(); pv_act = panel.PV(); p4 = panel.P(volatile=True)
        res = []
        for lam in lams:
            exp_cost, real_cost, cvar_val, emg = 0.0, 0.0, 0.0, 0.0
            for d in sub:
                L = load.loc[d].to_numpy(float); PVa = pv_act.loc[d].to_numpy(float)
                nd_act = (L - PVa) * C.DT
                fc = panel.pv_forecast(d, 0)
                nd_fc = (L - fc) * C.DT
                nd_sc = nd_fc[None, :] + scen["resid"]
                sol = solve_cvar_day(scen["price"], nd_fc, nd_sc, scen["w"],
                                     C.S_INIT, lam=lam, alpha=0.95, term_floor=C.S_INIT)
                pact = p4.loc[d].to_numpy(float)
                g = np.maximum(nd_act + sol["c"] - sol["d"] - sol["E"], 0.0)
                real = float(np.sum(pact * (sol["E"] + C.EMERGENCY_MULT * g)))
                g_sc = np.maximum(nd_sc + sol["c"][None, :] - sol["d"][None, :]
                                  - sol["E"][None, :], 0)
                sc_cost = (scen["price"] * (sol["E"][None, :]
                                            + C.EMERGENCY_MULT * g_sc)).sum(1)
                exp_cost += float(scen["w"] @ sc_cost)
                real_cost += real; emg += float(g.sum())
                var = np.quantile(sc_cost, 0.95)
                cvar_val += float(var + np.mean(np.maximum(sc_cost - var, 0)) / 0.05)
            n = len(sub)
            res.append(dict(lam=lam, exp_cost=exp_cost / n, real_cost=real_cost / n,
                            cvar=cvar_val / n, emg_qty=emg / n))
            print("  lam=%.1f  期望费用 %.2f  真实回测 %.2f  CVaR95 %.2f  紧急 %.1f kWh/日"
                  % (lam, exp_cost / n, real_cost / n, cvar_val / n, emg / n))
        with open(os.path.join(C.INTERMEDIATE_DIR, "q4_cvar.json"), "w") as f:
            json.dump(dict(days=[str(pd.Timestamp(x).date()) for x in sub], frontier=res),
                      f, ensure_ascii=False, indent=1)
    return out


if __name__ == "__main__":
    main()
```

源程序：code/results_io.py

```
# -*- coding: utf-8 -*-
"""结果文件写出器（严格按附件 5 模板的列序、列名、日期范围）。"""
from __future__ import annotations

import os

import numpy as np
import pandas as pd

import config as C


def _labels(fname: str, sheet: str):
    t = C.read_template(sheet, fname, nrows=1)
    return [str(v) for v in t.iloc[0].tolist()]
```

```python


def plan_sheet_frame(dates, E, price, price_mat=None) -> pd.DataFrame:
    """横表：日期 + 144 槽 + 全天购电量 + 全天购电费。"""
    labels = _labels("result2.xlsx", "计划购电量")
    slot_labels = labels[1:145]
    assert len(slot_labels) == C.N_SLOT, len(slot_labels)
    cols = ["日期\\时间"] + slot_labels + ["全天购电量", "全天购电费"]
    rows = []
    for i, d in enumerate(dates):
        p = price if price_mat is None else price_mat[i]
        rows.append([d.strftime("%Y-%m-%d")] + list(np.round(E[i], 4)) +
                    [round(float(E[i].sum()), 4), round(float(np.sum(p * E[i])), 2)])
    return pd.DataFrame(rows, columns=cols)


def soc_sheet_frame(dates, c, d, s0, s1) -> pd.DataFrame:
    """充放电量：每天 6 行（4 h 块），0:00 储电量填在 0:00-4:00 行、24:00 填在 4:00-8:00 行。"""
    rows = []
    for i, day in enumerate(dates):
        for b, ((a, bnd), lab) in enumerate(zip(C.BLOCKS, C.BLOCK_LABELS)):
            rec = [(day.strftime("%Y-%m-%d") if b == 0 else ""), lab,
                   round(float(c[i, a:bnd].sum()), 4), round(float(d[i, a:bnd].sum()), 4),
                   "", ""]
            if b == 0:
                rec[4] = "0:00"; rec[5] = round(float(s0[i]), 4)
            if b == 1:
                rec[4] = "24:00"; rec[5] = round(float(s1[i]), 4)
            rows.append(rec)
    return pd.DataFrame(rows, columns=["日期", "时间段", "充电量", "放电量", "时刻", "储电量"])


def emergency_frame(dates, g, tol=1e-6) -> pd.DataFrame:
    """紧急购电量：按连续时段合并，同一日期可多行。"""
    rows = []
    for i, day in enumerate(dates):
        gi = g[i]
        idx = np.where(gi > tol)[0]
        if len(idx) == 0:
            rows.append([day.strftime("%Y-%m-%d"), "无", 0.0])
            continue
        # 合并连续槽
        groups, start = [], idx[0]
        for a, b in zip(idx[:-1], idx[1:]):
            if b != a + 1:
                groups.append((start, a)); start = b
        groups.append((start, idx[-1]))
        first = True
        for a, b in groups:
            lab = f"{_hm(a)}-{_hm(b + 1)}"
            rows.append([day.strftime("%Y-%m-%d") if first else "", lab,
                         round(float(gi[a:b + 1].sum()), 3)])
            first = False
    return pd.DataFrame(rows, columns=["日期", "购电时间段", "购电量"])


def _hm(slot_end: int) -> str:
    """槽位末端的时刻标签，例如槽 5 结束 -> 1:00。"""
    m = slot_end * 10
    if m >= 1440:
        m -= 1440
        return f"{m // 60}:{m % 60:02d}+1" if m else "0:00+1"
    return f"{m // 60}:{m % 60:02d}"


def write_result(path, dates, E, c, d, s0, s1, price, g=None, E_adj=None,
                 price_mat=None, extra_sheets=()):
    """写出结果文件（工作表名与模板一致）。"""
    sheets = {"计划购电量": plan_sheet_frame(dates, E, price, price_mat)}
    if E_adj is not None:
        sheets["调整购电量"] = plan_sheet_frame(dates, E_adj, price, price_mat)
    sheets["充放电量"] = soc_sheet_frame(dates, c, d, s0, s1)
    sheets["紧急购电量"] = emergency_frame(dates, g if g is not None else np.zeros_like(E))
```

```
    order = ["计划购电量"] + (["调整购电量"] if E_adj is not None else []) + \
            ["充放电量", "紧急购电量"]
    with pd.ExcelWriter(path) as w:
        for k in order:
            sheets[k].to_excel(w, sheet_name=k, index=False)
    return path
```

源程序：code/exp_scan.py

```
# -*- coding: utf-8 -*-
"""
补充实验：Q1 容量-功率双向扫描、报童经验覆盖率、单参数灵敏度。
输出 intermediate/exp_scan.json / exp_newsboy.json / exp_sens.json
"""
from __future__ import annotations

import json
import os

import numpy as np
import pandas as pd
from scipy.optimize import linprog

import config as C
import lp_engine as LP
import q1 as Q1
import q2 as Q2


def q1_with(cap=None, pmax=None, eta_c=C.ETA_C, eta_d=C.ETA_D,
            s_min=None, s_max=None, s_init=C.S_INIT):
    """在给定储能参数下重解 Q1（用于容量/功率/效率扫描）。"""
    a1 = C.read_attachment1()
    price = a1["price"].to_numpy(float); load = a1["load"].to_numpy(float)
    pv = a1["pv"].to_numpy(float)
    n = C.N_SLOT
    e_max = (pmax * C.DT) if pmax is not None else C.E_MAX
    smin = C.S_MIN if s_min is None else s_min
    smax = C.S_MAX if s_max is None else s_max
    if cap is not None:                          # 保持 10%–90% DOD 同步移动上下界
        smin, smax = 0.10 * cap, 0.90 * cap
    nv = 4 * n
    iE, iC, iD, iS = 0, n, 2 * n, 3 * n
    obj = np.zeros(nv); obj[iE:iE + n] = price
    A_ub, b_ub = [], []
    for t in range(n):
        row = np.zeros(nv); row[iE + t] = -1; row[iC + t] = 1; row[iD + t] = -1
        A_ub.append(row); b_ub.append(-(load[t] - pv[t]) * C.DT)
    A_eq, b_eq = [], []
    for t in range(n):
        row = np.zeros(nv); row[iS + t] = 1
        if t > 0:
            row[iS + t - 1] = -1
        row[iC + t] = -eta_c; row[iD + t] = 1 / eta_d
        A_eq.append(row); b_eq.append(0.0 if t > 0 else s_init)
    bounds = [(0, None)] * n + [(0, e_max)] * n + [(0, e_max)] * n + [(smin, smax)] * n
    bounds[iS + n - 1] = (s_init, s_init)
    res = linprog(obj, A_ub=np.array(A_ub), b_ub=np.array(b_ub), A_eq=np.array(A_eq),
                  b_eq=np.array(b_eq), bounds=bounds, method="highs")
    return float(res.fun), res.x[iE:iE + n], res.x[iC:iC + n], res.x[iD:iD + n]


def capacity_scan():
    base = q1_with()[0]
    caps = [7200, 9600, 12000, 14400, 19200, 28800, 48000]
    out = []
    for cap in caps:
        c, E, cc, dd = q1_with(cap=cap)
        out.append(dict(cap=cap, cost=c, saving=base - c,
                        dcost=(base - c) / base * 100, charge=float(cc.sum()),
                        discharge=float(dd.sum())))
    powers = [1250, 2500, 5000, 7500, 10000]
    out2 = []
```

```
    for pm in powers:
        c, E, cc, dd = q1_with(pmax=pm)
        out2.append(dict(pmax=pm, cost=c, saving=base - c,
                         dcost=(base - c) / base * 100, charge=float(cc.sum())))
    # 单一自由度放开
    c_nocap, _, _, _ = q1_with(s_min=C.S_MIN, s_max=10 ** 7)    # 仅解除容量上限
    c_nopow, _, _, _ = q1_with(pmax=10 ** 6)                    # 仅解除功率上限
    return dict(base=base, cap_scan=out, power_scan=out2,
                no_capacity_limit=c_nocap, no_power_limit=c_nopow,
                gain_no_cap=(base - c_nocap) / base * 100,
                gain_no_pow=(base - c_nopow) / base * 100)


def eta_scan():
    out = []
    for ec, ed, name in [(0.9, 0.9, "单边各 0.9（往返 81%）"),
                         (0.9487, 0.9487, "往返 0.9 等效单边"),
                         (0.9, 1.0, "仅充电侧 0.9"),
                         (1.0, 0.9, "仅放电侧 0.9")]:
        c, *_ = q1_with(eta_c=ec, eta_d=ed)
        out.append(dict(eta_c=ec, eta_d=ed, name=name, cost=c,
                        saving=C.baseline_cost(C.read_attachment1()["price"].to_numpy(float),
                                               C.read_attachment1()["load"].to_numpy(float),
                                               C.read_attachment1()["pv"].to_numpy(float)) - c))
    return out


def newsboy_check(K=60):
    """报童经验覆盖率：计划口径下逐槽缺货频率与临界分位数理论的对比。"""
    j = json.load(open(os.path.join(C.INTERMEDIATE_DIR, "q2_main.json")))
    G = np.vstack([np.array(r["g"]) for r in j["daily"]])
    E = np.vstack([np.array(r["E"]) for r in j["daily"]])
    c = np.vstack([np.array(r["c"]) for r in j["daily"]])
    d = np.vstack([np.array(r["d"]) for r in j["daily"]])
    panel = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D"))
    load = panel.L().loc[[pd.Timestamp(r["date"]) for r in j["daily"]]].to_numpy(float)
    pv = panel.PV().loc[[pd.Timestamp(r["date"]) for r in j["daily"]]].to_numpy(float)
    nd_act = (load - pv) * C.DT
    short = nd_act + c - d - E
    margin = E - np.maximum(nd_act * 0 + (load - pv) * C.DT, 0) * 0  # 占位
    # 计划边际：E 相对"预测口径净需求 + 充放电"的超出量
    fc = np.zeros_like(nd_act)
    for i, r in enumerate(j["daily"]):
        fc[i] = (load[i] - panel.pv_forecast(pd.Timestamp(r["date"]), 0)) * C.DT
    req = fc + c - d
    m = E - np.maximum(req, 0)
    return dict(short_slot_rate=float((G > 1e-6).mean()),
                short_energy=float(G.sum()),
                margin_total=float(np.maximum(m, 0).sum()),
                margin_per_slot=float(np.maximum(m, 0).mean()),
                short_q99=float(np.percentile(short, 99)),
                short_q80=float(np.percentile(short, 80)),
                short_q833=float(np.percentile(short, 83.33)),
                fill_rate=float(1 - G.sum() / (nd_act.sum())))


def sens_penalty():
    """紧急购电倍数 k 的灵敏度（用 Q1 日 + 单一场景集重算典型日口径）。"""
    out = []
    for k in [2, 3, 5, 8, 10]:
        C.EMERGENCY_MULT = float(k)
        # 在固定残差场景集上求典型日的最优安全边际
        a1 = C.read_attachment1()
        price = a1["price"].to_numpy(float)
        panel = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D"))
        R = Q2.build_residuals(panel, Q2.DATE_WINDOW)
        cen, w = LP.reduce_scenarios_stratified(R, K=60)
        nd_fcst = (a1["load"].to_numpy(float) - a1["pv"].to_numpy(float)) * C.DT
        sol = LP.solve_day(price, nd_fcst, nd_fcst[None, :] + cen, w, s_init=C.S_INIT)
        out.append(dict(k=k, margin=float(np.maximum(
            sol["E"] - np.maximum(nd_fcst + sol["c"] - sol["d"], 0), 0).sum()),
            exp_emg=float((sol["g"] * w[:, None]).sum()),
            obj=float(sol["obj"])))
```

```
    C.EMERGENCY_MULT = 5.0
    return out


def main():
    res = {}
    res["capacity"] = capacity_scan()
    res["eta"] = eta_scan()
    res["newsboy"] = newsboy_check()
    res["penalty"] = sens_penalty()
    with open(os.path.join(C.INTERMEDIATE_DIR, "exp_scan.json"), "w") as f:
        json.dump(res, f, ensure_ascii=False, indent=1)
    print(json.dumps(res, ensure_ascii=False, indent=1)[:4000])


if __name__ == "__main__":
    main()
```

源程序：code/exp_sobol.py

```
# -*- coding: utf-8 -*-
"""
全局灵敏度分析（Sobol 指数，Saltelli/Jansen 估计）——在典型日（问题 1）模型上做。
参数 8 个：往返效率、SOC 上限、SOC 下限、充放功率上限、电价水平、峰谷价差、光伏出力、负载水平。
输出：一阶指数 S_i、总效应指数 S_Ti、以及各参数对"全天购电费"与"相对基线节省率"的影响。

用法：python3 exp_sobol.py [N]    （默认 N=256 → 2560 次 LP 求解）
"""
from __future__ import annotations

import json
import os
import sys

import numpy as np
from scipy.stats import qmc

import config as C
import exp_scan as ES

PARAMS = [
    ("eta_rt", "往返效率", 0.75, 0.95),
    ("s_max", "储电量上限 kWh", 8400.0, 14400.0),
    ("s_min", "储电量下限 kWh", 600.0, 2400.0),
    ("p_max", "充放功率上限 kW", 2500.0, 7500.0),
    ("price_lv", "电价水平倍数", 0.8, 1.2),
    ("price_pv", "峰谷价差倍数", 0.7, 1.3),
    ("pv_scale", "光伏出力倍数", 0.8, 1.2),
    ("load_scale", "负载水平倍数", 0.9, 1.1),
]


def model(x, ret_series=False):
    """给定参数向量，返回（全天购电费 元，相对无储能基线节省率）。"""
    d = dict(zip([p[0] for p in PARAMS], x))
    a1 = C.read_attachment1()
    price = a1["price"].to_numpy(float).copy()
    load = a1["load"].to_numpy(float).copy() * d["load_scale"]
    pv = a1["pv"].to_numpy(float).copy() * d["pv_scale"]
    # 电价：水平缩放 + 峰谷价差缩放（围绕均值）
    pm = price.mean()
    price = pm * d["price_lv"] + (price - pm) * d["price_pv"]
    eta = float(np.sqrt(d["eta_rt"]))
    smin = float(d["s_min"]); smax = float(d["s_max"])
    if smin >= smax - 100:                  # 保证可行
        smin, smax = 0.1 * smax, smax
    cost, E, c, dd = ES.q1_with(pmax=float(d["p_max"]), eta_c=eta, eta_d=eta,
                                s_min=smin, s_max=smax,
                                s_init=float(np.clip(6000.0, smin, smax)))
    base = C.baseline_cost(price, load, pv)
    save = (base - cost) / base
    return (cost, save) if ret_series else cost
```

```

def sobol_indices(N=256, seed=42, target=0):
    d = len(PARAMS)
    rng = np.random.default_rng(seed)
    A = qmc.Sobol(d=d, scramble=True, seed=seed).random(N).T
    B = qmc.Sobol(d=d, scramble=True, seed=seed + 1000).random(N).T
    lo = np.array([p[2] for p in PARAMS]); hi = np.array([p[3] for p in PARAMS])
    scale = lambda U: lo[:, None] + (hi - lo)[:, None] * U
    A, B = scale(A), scale(B)
    fA = np.array([model(A[:, j], ret_series=True)[target] for j in range(N)])
    fB = np.array([model(B[:, j], ret_series=True)[target] for j in range(N)])
    var = np.var(np.concatenate([fA, fB]), ddof=1)
    Si, STi = [], []
    for i in range(d):
        AB = A.copy(); AB[i] = B[i]
        fAB = np.array([model(AB[:, j], ret_series=True)[target] for j in range(N)])
        Si.append(float(np.mean(fB * (fAB - fA)) / var))
        STi.append(float(0.5 * np.mean((fA - fAB) ** 2) / var))
    return dict(N=N, names=[p[1] for p in PARAMS], keys=[p[0] for p in PARAMS],
                bounds=[[p[2], p[3]] for p in PARAMS],
                var=float(var), mean=float(np.mean(np.concatenate([fA, fB]))),
                S1=Si, ST=STi)


def main():
    N = int(sys.argv[1]) if len(sys.argv) > 1 else 256
    out = dict(cost=sobol_indices(N=N, target=0),
               saving=sobol_indices(N=N, target=1, seed=43))
    with open(os.path.join(C.INTERMEDIATE_DIR, "exp_sobol.json"), "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    for key, tag in (("cost", "全天购电费（元）"), ("saving", "相对基线节省率")):
        r = out[key]
        print(f"--- {tag}  N={r['N']}  均值={r['mean']:.4f}  方差={r['var']:.4g} ---")
        print(f"{'参数':<16}{'S1':>10}{'ST':>10}")
        for n, s, st in zip(r["names"], r["S1"], r["ST"]):
            print(f"{n:<16}{s:>10.4f}{st:>10.4f}")
        print("sum(S1) = %.4f" % sum(r["S1"]))


if __name__ == "__main__":
    main()
```

源程序：code/exp_na.py

```

# -*- coding: utf-8 -*-
"""
非预期性（non-anticipativity）置换检验
====================================
做法：把某一次预报（6:00 / 12:00 / 18:00）在日与日之间**随机置换**后重新求解，
      检查"仅由该次预报控制的时段"是否变化，以及"早于该预报时刻的时段"是否**完全不变**。
- 若置换 6:00 预报后，槽 36-71 的决策发生变化、而槽 0-35 完全不变 → 该次预报只影响它之后的时段，
  说明实现中没有"用未来信息决策"的泄漏（非预期性成立）。
- 同时给出对照：置换 0:00 预报应使槽 0-35 也变化（证明检验本身有效力）。

输出 intermediate/exp_na.json
"""
from __future__ import annotations

import json
import os

import numpy as np
import pandas as pd

import config as C
import lp_engine as LP
import q3 as Q3

DAYS = pd.date_range("2025-02-01", "2025-02-12", freq="D")
EPOCHS = (0, 6, 12, 18)
CONTROL = {0: (0, 36), 6: (36, 72), 12: (72, 108), 18: (108, 144)}


```

```python
def run_days(permuted_hour=None, seed=0, K=40):
    """运行 12 天；permuted_hour 不为 None 时，把该时刻的预报按日随机置换。"""
    panel = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D"))
    load = panel.L(); pv_act = panel.PV()
    price = C.read_attachment1()["price"].to_numpy(float)
    scens = {}
    for h in EPOCHS:
        s_m = C.EPOCH_SLOT[h]
        slots = np.arange(s_m, C.N_SLOT)
        cen, w = Q3.build_epoch_scenarios(panel, h, K=K, slots=slots)
        scens[h] = dict(slots=slots, cen=cen, w=w)
    rng = np.random.default_rng(seed)
    perm = rng.permutation(len(DAYS)) if permuted_hour is not None else None
    res = {}
    for i, d in enumerate(DAYS):
        L = load.loc[d].to_numpy(float)
        E_plan = np.zeros(C.N_SLOT); E_adj = np.zeros(C.N_SLOT)
        soc = C.S_INIT
        for h in EPOCHS:
            s_m = C.EPOCH_SLOT[h]; slots = scens[h]["slots"]
            src = d if (permuted_hour != h) else DAYS[perm[i]]
            fc = Q3.forecast_slots(panel, src, h, slots)
            nd_fc = np.zeros(C.N_SLOT); nd_fc[slots] = (L[slots] - fc) * C.DT
            cen = scens[h]["cen"]
            nd_sc = np.zeros((cen.shape[0], C.N_SLOT))
            nd_sc[:, slots] = nd_fc[slots][None, :] + cen
            sol = LP.solve_epoch(price, nd_fc, nd_sc, scens[h]["w"], s_init=soc,
                                 s_from=s_m, plan=(E_plan if h > 0 else None),
                                 term_floor=C.S_INIT)
            if h == 0:
                E_plan = sol["E"].copy()
            later = [x for x in EPOCHS if x > h]
            nxt = C.EPOCH_SLOT[later[0]] if later else C.N_SLOT
            for j, t in enumerate(range(s_m, nxt)):
                E_adj[t] = sol["E"][j]
            soc = float(sol["S"][nxt - s_m - 1])
        res[d] = E_adj.copy()
    return res


def main():
    base = run_days(None)
    out = {"baseline_days": len(base)}
    for h in EPOCHS:
        alt = run_days(permuted_hour=h, seed=1)
        diffs = {}
        for m, (a, b) in CONTROL.items():
            dmax = max(float(np.max(np.abs(alt[d][a:b] - base[d][a:b]))) for d in DAYS)
            diffs[f"槽{a}-{b}"] = dmax
        out[f"置换{h:02d}:00预报"] = diffs
    with open(os.path.join(C.INTERMEDIATE_DIR, "exp_na.json"), "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    print(json.dumps(out, ensure_ascii=False, indent=1))
    print()
    print("判读：置换某时刻预报后，该时刻**之前**控制区的最大变化应为 0，")
    print("      而该时刻**之后**控制区的变化应显著大于 0（否则检验没有效力）。")


if __name__ == "__main__":
    main()
```

源程序：code/exp_benchmark.py

```python
# -*- coding: utf-8 -*-
"""
基准实验：全年单一 LP（完美信息、单一储电量轨迹、无逐日决策解耦）
================================================================
作为"信息与协调"的上界基准：给出 2025-02-01 至 12-31 共 334 天、每日 144 槽
（合计 48 096 槽）的单一线性规划最优值，用以量化"逐日决策解耦 + 每日终端不亏空"
这两项建模选择的代价。

输出 intermediate/exp_benchmark.json
"""
```

```python
from __future__ import annotations

import json
import os
import time

import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import coo_matrix

import config as C


def full_year_lp():
    panel = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D"))
    load = panel.L(); pv = panel.PV()
    price = C.read_attachment1()["price"].to_numpy(float)
    win = pd.date_range(C.WIN_START, C.WIN_END, freq="D")
    n = C.N_SLOT; T = len(win); nv = 4 * T * n
    iE, iC, iD, iS = 0, T * n, 2 * T * n, 3 * T * n
    obj = np.zeros(nv); obj[iE:iE + T * n] = np.tile(price, T)
    rows, cols, vals, rhs = [], [], [], []
    r = 0
    for k in range(T):
        L = load.loc[win[k]].to_numpy(float); P = pv.loc[win[k]].to_numpy(float)
        for t in range(n):
            rows += [r, r, r]; cols += [iE + k * n + t, iC + k * n + t, iD + k * n + t]
            vals += [-1.0, 1.0, -1.0]
            rhs.append(-(L[t] - P[t]) * C.DT); r += 1
    A_ub = coo_matrix((vals, (rows, cols)), shape=(r, nv)).tocsr()
    rows2, cols2, vals2, rhs2 = [], [], [], []
    for k in range(T):
        for t in range(n):
            g = k * n + t
            rows2.append(g); cols2.append(iS + g); vals2.append(1.0)
            if g > 0:
                rows2.append(g); cols2.append(iS + g - 1); vals2.append(-1.0)
            rows2.append(g); cols2.append(iC + g); vals2.append(-C.ETA_C)
            rows2.append(g); cols2.append(iD + g); vals2.append(1.0 / C.ETA_D)
            rhs2.append(0.0 if g > 0 else C.S_INIT)
    A_eq = coo_matrix((vals2, (rows2, cols2)), shape=(T * n, nv)).tocsr()
    bounds = [(0, None)] * (T * n) + [(0, C.E_MAX)] * (T * n) + [(0, C.E_MAX)] * (T * n) + \
             [(C.S_MIN, C.S_MAX)] * (T * n)
    t0 = time.time()
    res = linprog(obj, A_ub=A_ub, b_ub=np.array(rhs), A_eq=A_eq, b_eq=np.array(rhs2),
                  bounds=bounds, method="highs")
    dt = time.time() - t0
    return dict(cost=float(res.fun), n_var=int(nv), n_slot=int(T * n),
                seconds=float(dt), success=bool(res.success),
                base_cost=sum(C.baseline_cost(price, load.loc[d].to_numpy(float),
                                              pv.loc[d].to_numpy(float)) for d in win))


def main():
    out = dict(full_year=full_year_lp())
    out["decoupling_gap_vs_R1"] = None
    q2 = os.path.join(C.INTERMEDIATE_DIR, "q2_cmp_R1.json")
    if os.path.exists(q2):
        r1 = json.load(open(q2))["meta"]
        out["daily_decoupled_R1"] = r1
        out["decoupling_gap"] = r1["tot_cost"] - out["full_year"]["cost"]
        out["decoupling_gap_pct"] = (r1["tot_cost"] - out["full_year"]["cost"]) / r1["tot_cost"]
    with open(os.path.join(C.INTERMEDIATE_DIR, "exp_benchmark.json"), "w") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    print(json.dumps(out, ensure_ascii=False, indent=1))
    print()
    print("全年单一 LP: %.6f 元（%.2f 万元），变量 %d，耗时 %.1f s"
          % (out["full_year"]["cost"], out["full_year"]["cost"] / 1e4,
             out["full_year"]["n_var"], out["full_year"]["seconds"]))
    if out.get("decoupling_gap") is not None:
        print("逐日解耦（完美信息 R1）比全年单一 LP 贵 %.2f 元（%.2f%%）"
              % (out["decoupling_gap"], out["decoupling_gap_pct"] * 100))
```

```

if __name__ == "__main__":
    main()
```

源程序：code/audit.py

```
# -*- coding: utf-8 -*-
"""
独立审计（A1 – A10）：从**结果文件**与**原始附件**出发独立复算，不调用建模目标函数。
================================================================================
A1  电量平衡残差（结果文件的 4 h 块级 + 中间结果的 10 min 级）
A2  SOC 递推残差（用 eta_c=eta_d=0.9 独立重算）
A3  SOC 越界数（[1200,10800]）
A4  充放电功率越界数（<= 5000 kW 等价 <= 833.3333 kWh/槽）
A5  充放互斥违反数
A6  周期/终端条件（result1: S0=S144；其余：S144 >= S0）
A7  非负性
A8  费用复算相对误差（不调用建模费用函数）
A9  日期与结构（行数/列数/工作表名/标签）
A10 全天汇总列（全天购电量、全天购电费）

输出：intermediate/audit_report.md 与 intermediate/audit.json
"""
from __future__ import annotations

import json
import os

import numpy as np
import pandas as pd

import config as C

TOL_RES = 1e-6          # kWh
TOL_REL = 1e-4          # 费用复算相对误差
RES_FILES = [("result1.xlsx", "q1"), ("result2.xlsx", "q2"),
             ("result3.xlsx", "q3"), ("result4-2.xlsx", "q4-2"),
             ("result4-3.xlsx", "q4-3")]


def _load(fname):
    p = os.path.join(C.RESULT_DIR, fname)
    xl = pd.ExcelFile(p)
    return {sh: xl.parse(sh) for sh in xl.sheet_names}


def audit_result_files():
    rows, out = [], {}
    a1 = C.read_attachment1()
    price_fixed = a1["price"].to_numpy(float)
    price_flat = C.read_attachment4()
    panel = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D"))

    for fname, tag in RES_FILES:
        p = os.path.join(C.RESULT_DIR, fname)
        if not os.path.exists(p):
            rows.append(dict(file=fname, item="存在性", value="缺失", verdict="FAIL"))
            continue
        sh = _load(fname)
        rep = {}
        plan = sh["计划购电量"]
        if tag == "q1":
            # A9 结构
            rep["A9_计划行数"] = (len(plan), 144)
            rep["A9_工作表"] = (list(sh.keys()), ["计划购电量", "充放电量"])
            E = plan.iloc[:, 1].to_numpy(float)
            rep["A7_负购电量数"] = int((E < -TOL_RES).sum())
            soc = sh["充放电量"]
            rep["A9_充放电行数"] = (len(soc), 6)
            c_blk = soc["充电量"].to_numpy(float)[:6]
            d_blk = soc["放电量"].to_numpy(float)[:6]
            sv = soc["储电量"].to_numpy(float)
```

```
            s0 = float(sv[0]); s24 = float(sv[1])
            rep["A6_日周期偏差"] = abs(s0 - s24)
            rep["A2_SOC净递推残差"] = abs((s0 + C.ETA_C * c_blk.sum()
                                           - d_blk.sum() / C.ETA_D) - s24)
            rep["A8_费用复算相对误差"] = abs(E @ price_fixed
                                             - float(np.sum(price_fixed * E))) / max(1e-9, float(np.sum(
    price_fixed * E)))
            rep["A1_全天购电量"] = float(E.sum())
            rep["A1_购电费"] = float(E @ price_fixed)
            rep["A1_基线C01"] = C.baseline_cost(price_fixed, a1["load"].to_numpy(float),
                                                a1["pv"].to_numpy(float))
        else:
            dates = pd.to_datetime(plan.iloc[:, 0])
            E = plan.iloc[:, 1:145].to_numpy(float)
            tot = plan.iloc[:, 145].to_numpy(float)
            fee = plan.iloc[:, 146].to_numpy(float)
            price_mat = (price_flat.loc[dates].to_numpy(float) if tag.startswith("q4")
                         else np.tile(price_fixed, (len(dates), 1)))
            rep["A9_天数"] = (len(dates), 334)
            rep["A9_列数"] = (plan.shape[1], 147)
            rep["A9_日期范围"] = (str(dates.min().date()), str(dates.max().date()),
                                  "2025-02-01", "2025-12-31")
            rep["A10_全天购电量最大偏差"] = float(np.max(np.abs(E.sum(1) - tot)))
            rep["A10_全天购电费最大相对偏差"] = float(np.max(
                np.abs(fee - (price_mat * E).sum(1)) / np.maximum(fee, 1)))
            rep["A7_负购电量数"] = int((E < -TOL_RES).sum())
            rep["A9_工作表"] = (list(sh.keys()), sorted(sh.keys()))
            # SOC 块级独立递推
            soc = sh["充放电量"]
            c_blk = soc["充电量"].to_numpy(float).reshape(-1, 6)
            d_blk = soc["放电量"].to_numpy(float).reshape(-1, 6)
            st = soc["储电量"].to_numpy(float)
            s0 = st[0::6]; s24 = st[1::6]
            lhs = s0 + C.ETA_C * c_blk.sum(1) - d_blk.sum(1) / C.ETA_D
            rep["A2_SOC净递推最大残差"] = float(np.max(np.abs(lhs - s24)))
            rep["A6_末值低于初值天数"] = int((s24 - s0 < -1e-6).sum())
            rep["A3_SOC越界数"] = int(((s0 < C.S_MIN - 1e-6) | (s24 > C.S_MAX + 1e-6)).sum())
            rep["A4_块级功率越界数"] = int((c_blk > 24 * C.E_MAX + 1e-6).sum()
                                           + (d_blk > 24 * C.E_MAX + 1e-6).sum())
            rep["A1_全窗购电量"] = float(E.sum())
            rep["A1_全窗购电费"] = float((price_mat * E).sum())
            # 紧急购电表结构
            em = sh["紧急购电量"]
            qty = pd.to_numeric(em["购电量"], errors="coerce").fillna(0).to_numpy(float)
            rep["A9_紧急购电行数"] = len(em)
            rep["A1_紧急购电量合计"] = float(qty.sum())
        out[tag] = rep
        for k, v in rep.items():
            rows.append(dict(file=fname, item=k, value=str(v), verdict=""))
    return rows, out


def audit_internal():
    """内部解（10 min 级）的可行性审计。"""
    out = {}
    q1 = json.load(open(os.path.join(C.INTERMEDIATE_DIR, "q1.json")))
    a = q1["arrays"]
    E = np.array(a["E"]); c = np.array(a["c"]); d = np.array(a["d"]); S = np.array(a["S"])
    price = np.array(a["price"]); load = np.array(a["load"]); pv = np.array(a["pv"])
    out["q1"] = dict(
        A1_平衡残差=float(abs(E + pv * C.DT + d - load * C.DT - c).max()),
        A2_SOC递推残差=float(abs(S - (np.concatenate([[C.S_INIT], S[:-1]])
                                      + C.ETA_C * c - d / C.ETA_D)).max()),
        A3_SOC越界=int(((S < C.S_MIN - 1e-9) | (S > C.S_MAX + 1e-9)).sum()),
        A4_功率越界=int(((c > C.E_MAX + 1e-9) | (d > C.E_MAX + 1e-9)).sum()),
        A5_互斥违反=int(((c > 1e-6) & (d > 1e-6)).sum()),
        A6_周期偏差=float(abs(a["S"][-1] - 6000.0)),
        A7_负购电=int((E < -1e-9).sum()),
        A8_费用相对误差=float(abs(float(np.sum(price * E)) - q1["main"]["cost"])
                              / q1["main"]["cost"]),
        A10_全天购电量=float(E.sum()))
    for tag, f in (("q2", "q2_main.json"), ("q3", "q3_n4.json")):
        p = os.path.join(C.INTERMEDIATE_DIR, f)
```

```
        if not os.path.exists(p):
            continue
        j = json.load(open(p))
        E = np.vstack([np.array(r["E"] if tag == "q2" else r["E_adj"]) for r in j["daily"]])
        c = np.vstack([np.array(r["c"]) for r in j["daily"]])
        d = np.vstack([np.array(r["d"]) for r in j["daily"]])
        g = np.vstack([np.array(r["g"]) for r in j["daily"]])
        s0 = np.array([r["s_init"] for r in j["daily"]])
        s1 = np.array([r.get("s_end", r.get("S_end")) for r in j["daily"]])
        load_m = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D")).L().loc[[
            pd.Timestamp(r["date"]) for r in j["daily"]]].to_numpy(float)
        pv_m = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D")).PV().loc[[
            pd.Timestamp(r["date"]) for r in j["daily"]]].to_numpy(float)
        nd = (load_m - pv_m) * C.DT
        # 供给 >= 需求；正残差 = 盈余处置量（弃光或超购处置），负残差才是不可行
        bal = E + pv_m * C.DT + d + g - load_m * C.DT - c
        bal_no_g = bal - g
        out[tag] = dict(
            A1_供给缺口=float(-min(bal.min(), 0.0)),
            A1_盈余处置量=float(np.maximum(bal, 0).sum()),
            A1_漏g时最大缺口=float(-bal_no_g.min()),
            A2_SOC净递推残差=float(np.max(np.abs(s1 - (s0 + C.ETA_C * c.sum(1)
                                                  - d.sum(1) / C.ETA_D)))),
            A4_功率越界=int(((c > C.E_MAX + 1e-6) | (d > C.E_MAX + 1e-6)).sum()),
            A5_互斥违反=int(((c > 1e-6) & (d > 1e-6)).sum()),
            A6_末值低于初值天数=int((s1 - s0 < -1e-6).sum()),
            A7_负购电=int((E < -1e-9).sum()),
            A8_费用相对误差=float(abs(sum(r["total_cost"] for r in j["daily"])
                                      - j["meta"].get("tot_cost", j["meta"].get("total_cost", np.nan)))
                                  / max(1e-9, j["meta"].get("tot_cost", j["meta"].get("total_cost", 1)))),
            A9_天数=len(j["daily"]))
    return out


def main():
    rows, out = audit_result_files()
    inner = audit_internal()
    os.makedirs(C.INTERMEDIATE_DIR, exist_ok=True)
    with open(os.path.join(C.INTERMEDIATE_DIR, "audit.json"), "w") as f:
        json.dump(dict(deliverable=out, internal=inner), f, ensure_ascii=False, indent=1)
    lines = ["# 独立审计报告（A1–A10）", "",
             "> 由独立脚本 audit.py 从结果文件与原始附件复算，不调用建模目标函数。",
             f"> 阈值：残差 ≤ {TOL_RES} kWh；费用复算相对误差 ≤ {TOL_REL}；越界数 = 0。", ""]
    lines.append("## 一、交付物审计（结果文件）")
    lines.append("")
    lines.append("| 文件 | 审计项 | 数值 |")
    lines.append("|---|---|---|")
    for r in rows:
        lines.append(f"| {r['file']} | {r['item']} | {r['value']} |")
    lines.append("")
    lines.append("## 二、内部解审计（10 min 级）")
    lines.append("")
    lines.append("| 问题 | 审计项 | 数值 |")
    lines.append("|---|---|---|")
    for tag, rep in inner.items():
        for k, v in rep.items():
            lines.append(f"| {tag} | {k} | {v:.3e} |" if isinstance(v, float)
                         else f"| {tag} | {k} | {v} |")
    txt = "\n".join(lines)
    with open(os.path.join(C.INTERMEDIATE_DIR, "audit_report.md"), "w") as f:
        f.write(txt)
    print(txt[:4000])
    return out


if __name__ == "__main__":
    main()
```

源程序：code/paper_numbers.py

```
# -*- coding: utf-8 -*-
"""汇总全部论文数值 -> intermediate/paper_numbers.json 与 notes/数值台账.md（唯一数字源）。"""
from __future__ import annotations
```

```

import json
import os

import numpy as np
import pandas as pd

import config as C

INT = C.INTERMEDIATE_DIR
NOTES = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "notes")


def load(name):
    p = os.path.join(INT, name)
    return json.load(open(p)) if os.path.exists(p) else None


def f(x, n=2):
    return f"{x:,.{n}f}"


def wan(x):
    return f"{x/1e4:,.2f}"


def main():
    q1 = load("q1.json")
    q2 = load("q2_main.json")
    q2k120 = load("q2_K120.json")
    q2r1 = load("q2_cmp_R1.json")
    q2r2 = load("q2_cmp_R2.json")
    q3 = {t: load(f"q3_{t}.json") for t in ("n1", "n2", "n4", "perf")}
    q4p = load("q4_price.json")
    q42 = load("q4_2.json")
    q43 = load("q4_3.json")
    scan = load("exp_scan.json")
    audit = load("audit.json")

    a1 = C.read_attachment1()
    panel = C.Panel(pd.date_range(C.WARMUP_START, C.WIN_END, freq="D"))
    load_m = panel.L(); pv_m = panel.PV(); p4 = panel.P(volatile=True)
    win = pd.date_range(C.WIN_START, C.WIN_END, freq="D")

    N = {}
    N["数据"] = dict(
        负载均值_kW=float(load_m.to_numpy().mean()), 负载最小_kW=float(load_m.to_numpy().min()),
        负载最大_kW=float(load_m.to_numpy().max()),
        负载日均电量_kWh=float(load_m.loc[win].to_numpy().sum(1).mean() * C.DT),
        光伏均值_kW=float(pv_m.to_numpy().mean()), 光伏峰值_kW=float(pv_m.to_numpy().max()),
        光伏日均电量_kWh=float(pv_m.loc[win].to_numpy().sum(1).mean() * C.DT),
        固定电价均值=float(a1["price"].mean()), 固定电价最小=float(a1["price"].min()),
        固定电价最大=float(a1["price"].max()),
        波动电价均值=float(p4.to_numpy().mean()), 波动电价最小=float(p4.to_numpy().min()),
        波动电价最大=float(p4.to_numpy().max()),
        附件1为逐时段均值最大偏差=float(np.abs(a1["price"].to_numpy()
                                    - p4.to_numpy().mean(0)).max()),
        附件1负载为逐时段均值最大偏差=float(np.abs(a1["load"].to_numpy()
                                     - load_m.to_numpy().mean(0)).max()),
        附件1光伏为逐时段均值最大偏差=float(np.abs(a1["pv"].to_numpy()
                                     - pv_m.to_numpy().mean(0)).max()),
    )
    # 预报误差
    FC = np.vstack([panel.pv_forecast(d, 0) for d in panel.PV().index])
    err = FC - pv_m.to_numpy()
    hrs = np.array([t // 6 for t in range(C.N_SLOT)])
    day = hrs >= 5
    N["预报误差"] = dict(
        MAE_全时段_kW=float(np.abs(err).mean()), RMSE_全时段_kW=float(np.sqrt((err ** 2).mean())),
        bias_全时段_kW=float(err.mean()),
        MAE_白天_kW=float(np.abs(err[:, day]).mean()),
        RMSE_白天_kW=float(np.sqrt((err[:, day] ** 2).mean())),
        bias_白天_kW=float(err[:, day].mean()),
```

```
        逐整点bias_kW=[float(x) for x in err.reshape(len(err), 24, 6).mean(axis=(0, 2))],
    )
    if q1:
        m = q1["main"]
        N["Q1"] = dict(
            无储能基线元=m["base_cost"], 最优购电费元=m["cost"], 节省元=m["saving"],
            节省率=m["saving_rate"], 全天购电量kWh=m["total_energy"],
            充电量kWh=m["charge_total"], 放电量kWh=m["discharge_total"],
            充放比=m["ratio_cd"], 弃光kWh=m["curtail_total"],
            SOC最小=m["min_soc"], SOC最大=m["max_soc"], S0=m["s0"], S144=m["s_end"],
            自由环状费用元=q1["free_cycle"]["cost"], 自由环状S=q1["free_cycle"]["s0"],
            自由环状差元=q1["free_cycle"]["cost"] - m["cost"],
            表1=q1["table1"], 表2=q1["table2"],
            残差=dict(R1=m["R1_min_surplus"], R2=m["R2_max_plan_violation"],
                      R3=m["R3_max_soc_residual"], R5=m["R5_simultaneous_slots"],
                      R6=m["R6_cd_ratio_err"]),
        )
    if scan:
        N["容量功率扫描"] = scan["capacity"]
        N["效率口径"] = scan["eta"]
        N["报童经验"] = scan["newsboy"]
        N["惩罚倍数灵敏度"] = scan["penalty"]
    if q2:
        N["Q2"] = dict(meta=q2["meta"])
        d0 = q2["daily"]
        N["Q2"]["四个指定日期"] = [r for r in d0 if r["date"][:10] in C.KEY_DATES]
    if q2k120:
        N["Q2_K120"] = q2k120["meta"]
    N["Q2_R1完美信息"] = q2r1["meta"] if q2r1 else None
    N["Q2_R2典型日预报"] = q2r2["meta"] if q2r2 else None
    N["Q3"] = {t: (v["meta"] if v else None) for t, v in q3.items()}
    if q3["n4"]:
        N["Q3"]["四个指定日期"] = [r for r in q3["n4"]["daily"]
                                   if r["date"][:10] in C.KEY_DATES]
    if q4p:
        N["电价检验"] = q4p["price_tests"]
        N["情景保真度_历史代表日"] = q4p["fidelity_hist"]
        N["情景保真度_分块Bootstrap"] = q4p["fidelity_boot"]
    N["Q4_2"] = q42["meta"] if q42 else None
    N["Q4_3"] = q43["meta"] if q43 else None
    if q42:
        N["Q4_2"]["四个指定日期"] = [r for r in q42["daily"] if r["date"][:10] in C.KEY_DATES]
    if q43:
        N["Q4_3"]["四个指定日期"] = [r for r in q43["daily"] if r["date"][:10] in C.KEY_DATES]
    dual2 = load("q1_dual2.json")
    N["Q1对偶验证"] = dual2
    N["Q2无安全边际消融"] = load("q2_noMargin.json")["meta"] if load("q2_noMargin.json") else None
    N["Q2安全边际分解"] = load("q2_margin.json")
    N["非预期性置换检验"] = load("exp_na.json")
    N["Sobol"] = load("exp_sobol.json")
    N["基准实验"] = load("exp_benchmark.json")
    N["窗口聚合"] = load("window_aggregates.json")
    N["样本外回测"] = load("exp_oos.json")
    N["终端条件对照"] = {t: (load(f"q2_term_{t}.json") or {}).get("meta")
                    for t in ("ge", "eq", "free")}
    N["审计"] = audit
    cvar = load("q4_cvar.json")
    N["CVaR前沿"] = cvar["frontier"] if cvar else None

    with open(os.path.join(INT, "paper_numbers.json"), "w") as fh:
        json.dump(N, fh, ensure_ascii=False, indent=1, default=str)

    # ---- Markdown 台账（供写作引用，单位与口径写全）----
    L = ["# 论文数值台账（唯一数字源）", "",
         "> 由 code/paper_numbers.py 从各求解输出汇总；论文中每个数字都应能在本表找到。",
         "> 口径：电量 kWh，功率 kW，费用 元；窗口 2025-02-01 至 2025-12-31（334 天）；",
         "> 固定价=附件 1 电价，波动价=附件 4；储能 12000 kWh / 5000 kW / SOC 1200–10800 / 效率单边 0.9
    。", ""]
    L.append("## 一、数据与预报")
    L.append("")
    L.append("| 量 | 数值 |")
    L.append("|---|---|")
    for k, v in N["数据"].items():
```

```
        L.append(f"| {k} | {v:,.4f} |" if isinstance(v, float) else f"| {k} | {v} |")
    for k, v in N["预报误差"].items():
        L.append(f"| {k} | {np.round(v,2)} |")
    L.append("")
    if q1:
        m = q1["main"]
        L.append("## 二、问题 1（典型日）")
        L.append("")
        L.append("| 量 | 数值 |")
        L.append("|---|---|")
        for k, v in N["Q1"].items():
            if isinstance(v, float):
                L.append(f"| {k} | {v:,.4f} |")
            elif k == "残差":
                L.append(f"| 残差 | {v} |")
            elif k in ("表1", "表2"):
                L.append(f"| {k} | {v} |")
        L.append("")
        L.append("表 1 指定时段购电量（kWh）：" + "；".join(
            f"{a}={b:,.4f}" for a, b in q1["table1"]))
        L.append("")
        rows, s0, s24 = q1["table2"]
        L.append("表 2 充放电量（kWh）：" + "；".join(
            f"{a} 充{b:,.4f}/放{c:,.4f}" for a, b, c in rows) + f"；0:00={s0:,.4f}，24:00={s24:,.4f}")
        L.append("")
    L.append("## 三、容量/功率/效率与报童")
    L.append("")
    if scan:
        L.append("容量扫描（保持 10%–90% DOD）：")
        L.append("")
        L.append("| 容量 kWh | 最优费用 元 | 相对基线节省 元 | 节省率 % |")
        L.append("|---|---|---|---|")
        for r in scan["capacity"]["cap_scan"]:
            L.append(f"| {r['cap']:,} | {r['cost']:,.2f} | {r['saving']:,.2f} | {r['dcost']:.2f} |")
        L.append("")
        L.append(f"仅解除容量上限：{scan['capacity']['no_capacity_limit']:,.2f} 元（多省 "
                 f"{scan['capacity']['gain_no_cap']:.1f}%）；仅解除功率上限："
                 f"{scan['capacity']['no_power_limit']:,.2f} 元（多省 {scan['capacity']['gain_no_pow
']:.1f}%）")
        L.append("")
        L.append("| 充放功率上限 kW | 最优费用 元 | 节省率 % |")
        L.append("|---|---|---|")
        for r in scan["capacity"]["power_scan"]:
            L.append(f"| {r['pmax']:,} | {r['cost']:,.2f} | {r['dcost']:.2f} |")
        L.append("")
        L.append("效率口径对照：")
        L.append("")
        L.append("| 口径 | 最优费用 元 | 节省 元 |")
        L.append("|---|---|---|")
        for r in scan["eta"]:
            L.append(f"| {r['name']} | {r['cost']:,.2f} | {r['saving']:,.2f} |")
        L.append("")
        nb = scan["newsboy"]
        L.append(f"报童经验覆盖率：逐槽缺货频率 {nb['short_slot_rate']*100:.2f}%，"
                 f"缺口电量 {nb['short_energy']:,.2f} kWh，"
                 f"电量满足率 {nb['fill_rate']*100:.2f}%，"
                 f"安全边际合计 {nb['margin_total']:,.2f} kWh（{nb['margin_per_slot']:.2f} kWh/槽）")
        L.append("")
        L.append("紧急购电倍数灵敏度（典型日）：" + "；".join(
            f"k={r['k']} 边际 {r['margin']:.0f} kWh/期望紧急 {r['exp_emg']:.0f} kWh"
            for r in scan["penalty"]))
        L.append("")
    L.append("## 四、四问主结果")
    L.append("")
    L.append("| 模型 | 总费用 元 | 万元 | 相对基线节省率 | 紧急购电量 kWh |")
    L.append("|---|---|---|---|---|")
    if q2:
        mm = q2["meta"]
        L.append(f"| 问题 2（固定价，两阶段） | {mm['tot_cost']:,.2f} | {wan(mm['tot_cost'])} | "
                 f"{(mm['tot_base']-mm['tot_cost'])/mm['tot_base']*100:.2f}% | {mm['tot_emg_qty']:,.2f}
 |")
    if q3["n4"]:
        mm = q3["n4"]["meta"]
```

```
        L.append(f"| 问题 3（固定价，四次滚动） | {mm['total_cost']:,.2f} | {wan(mm['total_cost'])} | "
                 f"{(mm['base_cost']-mm['total_cost'])/mm['base_cost']*100:.2f}% | {mm['emg_qty']:,.2f} |")
    if q42:
        mm = q42["meta"]
        L.append(f"| 问题 4-2（波动价，两阶段） | {mm['total']:,.2f} | {wan(mm['total'])} | "
                 f"{(mm['base']-mm['total'])/mm['base']*100:.2f}% | {mm['emg_qty']:,.2f} |")
    if q43:
        mm = q43["meta"]
        L.append(f"| 问题 4-3（波动价，四次滚动） | {mm['total_cost']:,.2f} | {wan(mm['total_cost'])} | "
                 f"{(mm['base_cost']-mm['total_cost'])/mm['base_cost']*100:.2f}% | {mm['emg_qty']:,.2f} |")
    L.append("")
    L.append(f"无储能基线：固定价 {q2['meta']['tot_base']:,.2f} 元；波动价 {q42['meta']['base']:,.2f} 元。")
    L.append("")
    L.append("VoI 实验（同一场景集、同一实际轨迹）：")
    L.append("")
    L.append("| 预报时刻数 | 总费用 元 | 紧急购电量 kWh |")
    L.append("|---|---|---|")
    for t, name in (("n1", "仅 0:00（1 次）"), ("n2", "0:00+12:00（2 次）"),
                    ("n4", "0:00/6:00/12:00/18:00（4 次）"), ("perf", "完全信息（上界）")):
        if q3[t]:
            L.append(f"| {name} | {q3[t]['meta']['total_cost']:,.2f} | {q3[t]['meta']['emg_qty']:,.2f} |")
    L.append("")
    with open(os.path.join(NOTES, "数值台账.md"), "w") as fh:
        fh.write("\n".join(L))
    print("\n".join(L))
    return N


if __name__ == "__main__":
    main()
```

源程序：code/run_all.py

```
# -*- coding: utf-8 -*-
"""一键复现：依次求解问题 1–4、写出结果文件、跑审计与数值台账。

用法：python3 run_all.py             （全部）
      python3 run_all.py --fast      （跳过 K 稳健性与 CVaR 前沿等耗时实验）
预期耗时（单机、arm64、macOS）：约 5 分钟（--fast 约 4 分钟）。
"""
from __future__ import annotations

import argparse
import os
import subprocess
import sys
import time

HERE = os.path.dirname(os.path.abspath(__file__))
STEPS = [
    ("问题 1：典型日确定性 LP", [sys.executable, "q1.py"]),
    ("问题 2：固定价两阶段随机规划（334 天）", [sys.executable, "q2.py", "--tag", "main", "--K", "60", "--scen", "strat", "--info", "R3", "--write"]),
    ("问题 3：多预报时刻滚动优化（四次预报）", [sys.executable, "q3.py", "--epochs", "0,6,12,18", "--tag", "n4", "--K", "60", "--write"]),
    ("问题 4-2：波动电价两阶段", [sys.executable, "q4.py", "--mode", "q42", "--K", "64", "--write"]),
    ("问题 4-3：波动电价滚动优化", [sys.executable, "q4.py", "--mode", "q43", "--K", "64", "--write"]),
    ("容量/功率/效率扫描与报童经验覆盖率", [sys.executable, "exp_scan.py"]),
    ("独立审计（A1–A10）", [sys.executable, "audit.py"]),
    ("汇总论文数值台账", [sys.executable, "paper_numbers.py"]),
]
FAST_EXTRA = [
    ("问题 2 对照：完美信息 R1", [sys.executable, "q2.py", "--tag", "cmp_R1", "--K", "60", "--scen", "strat", "--info", "R1"]),
    ("问题 2 对照：典型日预报 R2", [sys.executable, "q2.py", "--tag", "cmp_R2", "--K", "60", "--scen", "strat", "--info", "R2"]),
    ("问题 2 稳健性：K=120", [sys.executable, "q2.py", "--tag", "K120", "--K", "120", "--scen", "strat", "--info", "R3"]),
```

```
    ("VoI: 1 次预报", [sys.executable, "q3.py", "--epochs", "0", "--tag", "n1", "--K", "60"]),
    ("VoI: 2 次预报", [sys.executable, "q3.py", "--epochs", "0,12", "--tag", "n2", "--K", "60"]),
    ("VoI: 完全信息上界", [sys.executable, "q3.py", "--epochs", "0,6,12,18", "--tag", "perf", "--K", "
     60", "--perfect"]),
    ("CVaR 风险—费用前沿", [sys.executable, "q4.py", "--mode", "cvar", "--K", "64", "--step", "6"]),
    ("电价过程检验与情景保真度", [sys.executable, "q4.py", "--mode", "tests", "--K", "64"]),
]


def run(name, cmd):
    t0 = time.time()
    print("=" * 78)
    print(f"[{name}]  {' '.join(cmd)}")
    r = subprocess.run(cmd, cwd=HERE)
    print(f"[{name}] 完成，耗时 {time.time()-t0:.1f}s，退出码 {r.returncode}")
    if r.returncode != 0:
        raise SystemExit(f"步骤失败：{name}")


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--fast", action="store_true", help="跳过对照实验")
    args = ap.parse_args()
    t0 = time.time()
    for name, cmd in STEPS:
        run(name, cmd)
    if not args.fast:
        for name, cmd in FAST_EXTRA:
            run(name, cmd)
    print("=" * 78)
    print(f"全部完成，总耗时 {time.time()-t0:.1f}s。结果文件在 ../results/，中间结果在 ../intermediate/。
     ")


if __name__ == "__main__":
    main()
```

源程序：code/requirements.txt

```
# C 题论文复现环境（实测版本，macOS 15.7.5 / arm64）
numpy==2.4.3
pandas==3.0.1
scipy==1.18.1
openpyxl==3.1.5
matplotlib==3.11.1
scikit-learn==1.9.0
```

源程序：code/plot_figures.py

```
# -*- coding: utf-8 -*-
r"""
论文数据图绘制脚本（fig03 - fig10）
====================================
一次运行生成论文全部 8 张矢量数据图（PDF），输出到 C题论文/figures/。

运行方式
--------
    cd C题论文/code && python3 plot_figures.py
    # 或从工作区根目录：python3 "C题论文/code/plot_figures.py"

硬性口径
--------
* 中文字体：font.sans-serif = ["Songti SC","Heiti SC","PingFang SC","Arial Unicode MS"]，
  axes.unicode_minus = False；**导入 matplotlib 之前**必须把 MPLCONFIGDIR 指向工作区内
  的缓存目录（C题论文/build/mplcache），否则 macOS 上会因字体缓存不可写而报错。
* 输出：PDF 矢量（pdf.fonttype=42 嵌入 TrueType 子集），bbox_inches="tight"。
* 图内**不写标题**（论文由 LaTeX \caption 给题），只写坐标轴标签与单位；
  每条曲线/子图自带图例、单位与关键数值标注，可脱离正文阅读。
* 数字来源：C题论文/intermediate/*.json（q1 / exp_scan / q2_main / q3_n1 / q3_n2 /
  q3_n4 / q3_perf / q4_price / q4_cvar）与原始附件（附件 1、附件 4）。
  除"分位带"直接由附件 4 原始矩阵计算外，图中所有数字均由中间结果 JSON 复算得到，
  不做任何人工录入。
```

```
24
25  图的清单（编号与文件名固定）
26  -------------------------
27      fig03_q1_dual_price.pdf          问题 1 最优性证书：电价 vs 储电边际价值 + 充放电量
28      fig04_q1_capacity_power.pdf      容量-功率双向扫描
29      fig05_cost_waterfall.pdf         问题 1 成本分解瀑布图
30      fig06_voi.pdf                    信息价值 VoI（预报时刻数 1/2/4/完全信息）
31      fig07_q2_emergency.pdf           问题 2 全年逐日紧急购电 + 报童两界
32      fig08_q4_price.pdf               附件 4 电价剖面（分位带 + 分时骨架）+ 残差 ACF
33      fig09_cvar_frontier.pdf          CVaR 风险-费用前沿（依赖 q4_cvar.json，缺失则跳过）
34      fig10_strategy_mechanism.pdf     问题 1 最优策略机制（PV 盈余/负荷/电价/SOC/充放电）
35  """
36  from __future__ import annotations
37
38  # ---------------------------------------------------------------------------
39  # 0. 环境（必须在 import matplotlib 之前完成）
40  # ---------------------------------------------------------------------------
41  import os
42
43  HERE = os.path.dirname(os.path.abspath(__file__))
44  PAPER_DIR = os.path.dirname(HERE)                    # C题论文/
45  WORKSPACE = os.path.dirname(PAPER_DIR)               # 工作区根目录
46  MPL_CACHE = os.path.join(PAPER_DIR, "build", "mplcache")
47  os.makedirs(MPL_CACHE, exist_ok=True)
48  os.environ["MPLCONFIGDIR"] = MPL_CACHE               # 字体缓存必须可写
49
50  import json
51  from datetime import datetime
52
53  import numpy as np
54
55  import matplotlib
56  matplotlib.use("Agg")
57  import matplotlib.pyplot as plt
58  from matplotlib.dates import MonthLocator, DateFormatter, date2num
59  from matplotlib.ticker import MultipleLocator, FuncFormatter
60
61  # ---------------------------------------------------------------------------
62  # 1. 全局样式
63  # ---------------------------------------------------------------------------
64  FONT_STACK = ["Songti SC", "Heiti SC", "PingFang SC", "Arial Unicode MS"]
65  plt.rcParams.update({
66      "font.family": "sans-serif",
67      "font.sans-serif": FONT_STACK,
68      "axes.unicode_minus": False,
69      "pdf.fonttype": 42,          # 嵌入 TrueType 子集，矢量且中文不缺字
70      "ps.fonttype": 42,
71      "savefig.bbox": "tight",
72      "figure.dpi": 120,
73      "font.size": 9,
74      "axes.labelsize": 9,
75      "axes.titlesize": 9,         # 本脚本不写标题，仅防意外
76      "xtick.labelsize": 8,
77      "ytick.labelsize": 8,
78      "legend.fontsize": 7.4,
79      "axes.grid": True,
80      "grid.alpha": 0.28,
81      "grid.linewidth": 0.5,
82      "axes.linewidth": 0.8,
83      "legend.framealpha": 0.92,
84      "lines.linewidth": 1.3,
85  })
86
87  # 统一配色
88  C_PRICE = "#1f4e9c"      # 电价
89  C_MU = "#c0392b"         # 储电边际价值
90  C_CHG = "#2e86de"        # 充电
91  C_DIS = "#e67e22"        # 放电
92  C_PV = "#e8a33d"         # 光伏
93  C_LOAD = "#34495e"       # 负载
94  C_SOC = "#16a085"        # 储电量
95  C_GREY = "0.55"
96
```

```
# 主口径常数（与 config.py 一致；此处仅用于图上阈值线，不参与任何求解）
DT = 1.0 / 6.0           # h
ETA_C = ETA_D = 0.9      # 单边效率（假设 A2，往返 81%）
E_MAX = 5000.0 * DT      # kWh/槽，充放电量上限 833.3333
S_MIN, S_MAX, S_INIT = 1200.0, 10800.0, 6000.0

INTERMEDIATE = os.path.join(PAPER_DIR, "intermediate")
FIGDIR = os.path.join(PAPER_DIR, "figures")
ATTACH = os.path.join(WORKSPACE, "CUMCM2026Problems(1)", "C题", "附件")

FIG_SOURCES: dict[str, str] = {}   # 文件名 -> 数据来源说明


# ---------------------------------------------------------------------------
# 2. 工具函数
# ---------------------------------------------------------------------------
def load_json(name: str, required: bool = True):
    """读取 intermediate/<name>；required=False 时缺失返回 None。"""
    path = os.path.join(INTERMEDIATE, name)
    if not os.path.exists(path):
        if required:
            raise FileNotFoundError("缺少中间结果文件：%s" % path)
        return None
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)


def fnum(x: float, nd: int = 2) -> str:
    """千分位数字（论文口径：费用保留 2 位小数）。"""
    return ("{:,.%df}" % nd).format(float(x))


def read_attach_matrix(fname: str):
    """读取附件（首列为日期，其后 144 列）为 ndarray（行=日，列=槽位）。"""
    import pandas as pd
    path = os.path.join(ATTACH, fname)
    df = pd.read_excel(path, header=0)
    df = df.rename(columns={df.columns[0]: "date"})
    df = df.set_index("date")
    df.columns = [f"s{j}" for j in range(144)]
    return df.astype(float).to_numpy()


def read_attach1():
    """附件 1：典型日 {电价 元/kWh，小区负载 kW，光伏功率 kW}，144 槽。"""
    import pandas as pd
    df = pd.read_excel(os.path.join(ATTACH, "附件1.xlsx"), header=0)
    df.columns = ["时间", "price", "load", "pv"]
    return (df["price"].to_numpy(float), df["load"].to_numpy(float),
            df["pv"].to_numpy(float))


def slot_hours() -> np.ndarray:
    """槽位左端点（h）：槽 j = [j/6, (j+1)/6)。"""
    return np.arange(144) / 6.0


def step_close(t: np.ndarray, v: np.ndarray):
    """把 144 槽的阶梯序列闭合到 24 h，便于 step/fill_between(step='post')。"""
    return np.append(t, 24.0), np.append(v, v[-1])


def hhmm(hour_float: float) -> str:
    h = int(hour_float)
    m = int(round((hour_float - h) * 60))
    if m == 60:
        h, m = h + 1, 0
    return "%d:%02d" % (h, m)


def note(ax, text: str, loc: str = "upper left", fs: float = 7.2, pad: float = 0.38,
         xy: tuple | None = None):
    """轴内信息框（数值标注，不承担标题作用）。xy 给定时按轴分数坐标自定义位置。"""
```

```
    if xy is None:
        xy = {"upper left": (0.012, 0.975), "upper right": (0.988, 0.975),
              "lower left": (0.012, 0.030), "lower right": (0.988, 0.030)}[loc]
        ha = "left" if "left" in loc else "right"
        va = "top" if "upper" in loc else "bottom"
    else:
        ha, va = "left", "top"
    ax.text(xy[0], xy[1], text, transform=ax.transAxes, ha=ha, va=va,
            fontsize=fs, linespacing=1.45,
            bbox=dict(boxstyle="round,pad=%.2f" % pad, facecolor="white",
                      edgecolor="0.72", linewidth=0.6, alpha=0.93))


def save(fig, name: str) -> str:
    path = os.path.join(FIGDIR, name)
    fig.savefig(path, format="pdf", bbox_inches="tight",
                metadata={"Creator": "plot_figures.py", "CreationDate": None})
    plt.close(fig)
    return path


def hour_axis(ax):
    ax.set_xlim(0, 24)
    ax.xaxis.set_major_locator(MultipleLocator(2))
    ax.xaxis.set_major_formatter(FuncFormatter(lambda v, p: "%d" % int(round(v))))
    ax.set_xlabel("时刻（h）")


# ---------------------------------------------------------------------------
# 3. 各图
# ---------------------------------------------------------------------------
def fig03():
    """问题 1 最优性证书：电价 vs 储电边际价值（含死区与两条阈值线）+ 最优充放电量。"""
    q1 = load_json("q1.json")
    A = q1["arrays"]
    price = np.asarray(A["price"], float)
    mu = -np.asarray(A["dual_eq"], float)        # 取负：储电边际价值（元/kWh）
    c = np.asarray(A["c"], float)
    d = np.asarray(A["d"], float)

    t = slot_hours()
    xe, pe = step_close(t, price)
    _, mue = step_close(t, mu)
    th_c = ETA_C * mue                           # 充电阈值 eta_c * mu_t
    th_d = mue / ETA_D                           # 放电阈值 mu_t / eta_d

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(6.9, 5.4), sharex=True,
                                   gridspec_kw=dict(height_ratios=[1.32, 1.0], hspace=0.10))

    # ---- 上：价格与边际价值 ----
    ax1.fill_between(xe, th_c, th_d, step="post", color="0.87",
                     label=r"死区：$\eta_c\mu_t\leq p_t\leq \mu_t/\eta_d$（不充不放）")
    ax1.fill_between(xe, pe, th_c, step="post", where=(pe < th_c - 1e-9),
                     color="#f5c26b", alpha=0.75, linewidth=0,
                     label=r"充电窗口：$p_t<\eta_c\mu_t$")
    ax1.step(xe, pe, where="post", color=C_PRICE, label=r"电价 $p_t$（附件 1，元/kWh）")
    ax1.step(xe, mue, where="post", color=C_MU, label=r"储电边际价值 $\mu_t$（元/kWh）")
    ax1.step(xe, th_c, where="post", color="#d35400", ls="--", lw=1.0,
             label=r"充电阈值 $\eta_c\mu_t$")
    ax1.step(xe, th_d, where="post", color="#7d3c98", ls="--", lw=1.0,
             label=r"放电阈值 $\mu_t/\eta_d$")
    ax1.set_ylabel("电价 / 储电边际价值（元 · kWh$^{-1}$）")
    ax1.set_ylim(0, 1.62)
    ax1.legend(loc="upper left", ncol=3, columnspacing=0.9, handlelength=1.8)
    note(ax1, "死区比 $1/(\\eta_c\\eta_d)=%.4f$；$\\mu_t$ = LP 等式约束对偶量（水价值）"
         % q1["main"]["ratio_cd"], loc="lower right", fs=6.8)

    # ---- 下：最优充放电量（充电为正、放电为负） ----
    w = 1.0 / 6.0
    ax2.bar(t + w / 2, c, width=w, color=C_CHG, label="最优充电量 $c_t$（kWh/槽）")
    ax2.bar(t + w / 2, -d, width=w, color=C_DIS, label="最优放电量 $d_t$（kWh/槽）")
    ax2.axhline(0, color="0.25", lw=0.7)
    ax2.set_ylim(-1080, 1080)
```

```
    ax2.set_ylabel("充放电量（kWh · 槽$^{-1}$）\n充电为正、放电为负")
    hour_axis(ax2)
    ax2.legend(loc="upper left", ncol=2, columnspacing=1.0)

    load = np.asarray(A["load"], float)
    pv = np.asarray(A["pv"], float)
    ch_idx = np.where(c > 1e-6)[0]
    dis_idx = np.where(d > 1e-6)[0]
    idle_idx = np.where((c <= 1e-6) & (d <= 1e-6))[0]
    ok_chg = int(np.sum(price[ch_idx] <= ETA_C * mu[ch_idx] + 1e-9))
    ok_dis = int(np.sum(price[dis_idx] >= mu[dis_idx] / ETA_D - 1e-9))
    idle_in = int(np.sum((price[idle_idx] >= ETA_C * mu[idle_idx] - 1e-9) &
                         (price[idle_idx] <= mu[idle_idx] / ETA_D + 1e-9)))
    viol = ch_idx[price[ch_idx] > ETA_C * mu[ch_idx] + 1e-9]
    viol_sur = int(np.sum(pv[viol] > load[viol]))
    note(ax2, "充电槽 %d（其中 %d 个落在阴影充电窗口内；其余 %d 个中有 %d 个处于 PV 盈余槽，"
              "充电由零成本光伏供给）\n"
              "放电槽 %d（%d 个落在放电阈值上方）；空闲槽 %d（%d 个落在死区内）\n"
              "充电量 %s kWh，放电量 %s kWh，充放比 %.4f = $1/(\\eta_c\\eta_d)$"
         % (len(ch_idx), ok_chg, len(viol), viol_sur, len(dis_idx), ok_dis,
            len(idle_idx), idle_in, fnum(c.sum()), fnum(d.sum()), c.sum() / d.sum()),
         loc="lower left", fs=6.5)

    for ax in (ax1, ax2):
        ax.set_xlim(0, 24)
    FIG_SOURCES["fig03_q1_dual_price.pdf"] = (
        "q1.json: arrays.price / arrays.dual_eq(取负=水价值) / arrays.c / arrays.d; "
        "阈值线由 config.py 主口径 eta_c=eta_d=0.9 计算")
    return save(fig, "fig03_q1_dual_price.pdf")


def fig04():
    """容量-功率双向扫描：左=容量扫描（最优费用 + 储电边际价值），右=充放功率上限扫描。"""
    scan = load_json("exp_scan.json")["capacity"]
    caps = np.array([r["cap"] for r in scan["cap_scan"]], float)
    costs = np.array([r["cost"] for r in scan["cap_scan"]], float)
    pows = np.array([r["pmax"] for r in scan["power_scan"]], float)
    pcost = np.array([r["cost"] for r in scan["power_scan"]], float)
    pchg = np.array([r["charge"] for r in scan["power_scan"]], float)
    base = float(scan["base"])

    # 储电容量边际价值 = -dC*/dSbar（相邻扫描点中心/单侧差分，单位 元/kWh）
    mcaps = 0.5 * (caps[1:] + caps[:-1])
    mvals = -(np.diff(costs) / np.diff(caps))

    fig, (axL, axR) = plt.subplots(1, 2, figsize=(7.2, 3.4))
    fig.subplots_adjust(wspace=0.46)
    LIMIT_C = "#7d3c98"      # 单一自由度放开的参考线

    # ---- 左：容量扫描（最优费用 + 储电容量边际价值） ----
    axL.plot(caps, costs, "o-", color=C_PRICE, ms=3.6, label="最优购电费 $C^*$（元/日）")
    axL.axhline(base, color=C_GREY, ls=":", lw=1.0,
                label="题面容量 12 000 kWh 基准：%s 元/日" % fnum(base))
    axL.axhline(scan["no_capacity_limit"], color=LIMIT_C, ls="--", lw=1.0,
                label="仅解除容量上限：%s 元/日（再省 %.2f%%）"
                      % (fnum(scan["no_capacity_limit"]), scan["gain_no_cap"]))
    axL.set_xlabel("可用储能容量（kWh，保持 10%–90% DOD）")
    axL.set_ylabel("最优购电费（元 · 日$^{-1}$）")
    axL.set_ylim(24800, 40400)
    axL.set_xlim(0, 50000)

    axL2 = axL.twinx()
    axL2.step(mcaps, mvals, where="mid", color=C_MU, lw=1.3,
              label=r"容量边际价值 $-\partial C^*/\partial\bar S$（元/kWh）")
    axL2.plot(mcaps, mvals, "s", color=C_MU, ms=3.0)
    axL2.set_ylabel(r"边际价值 $-\partial C^*/\partial\bar S$（元 · kWh$^{-1}$）",
                    color=C_MU)
    axL2.tick_params(axis="y", colors=C_MU)
    axL2.set_ylim(0, 1.15)
    axL2.grid(False)
    h1, l1 = axL.get_legend_handles_labels()
    h2, l2 = axL2.get_legend_handles_labels()
    axL.legend(h1 + h2, l1 + l2, loc="upper right", fontsize=6.4)
```

```
    note(axL, "费用曲线分段线性、下凸；边际价值由 %.3f 单调递减至 %.3f 元/kWh"
         % (mvals[0], mvals[-1]), loc="lower left", fs=6.4)

    # ---- 右：充放电功率上限扫描 ----
    axR.plot(pows, pcost, "o-", color=C_PRICE, ms=3.6, label="最优购电费 $C^*$（元/日）")
    axR.axhline(scan["no_power_limit"], color=LIMIT_C, ls="--", lw=1.0,
                label="仅解除功率上限：%s 元/日（再省 %.2f%%）"
                      % (fnum(scan["no_power_limit"]), scan["gain_no_pow"]))
    axR.set_xlabel("充放电功率上限（kW）")
    axR.set_ylabel("最优购电费（元 · 日$^{-1}$）")
    axR.set_ylim(33800, 40800)
    axR.set_xlim(600, 11000)

    axR2 = axR.twinx()
    axR2.plot(pows, pchg, "s--", color=C_CHG, ms=3.2, lw=1.1,
              label="全天充电量（kWh/日）")
    axR2.set_ylabel("全天充电量（kWh · 日$^{-1}$）", color=C_CHG)
    axR2.tick_params(axis="y", colors=C_CHG)
    axR2.set_ylim(8000, 32000)
    axR2.grid(False)
    h1, l1 = axR.get_legend_handles_labels()
    h2, l2 = axR2.get_legend_handles_labels()
    axR.legend(h1 + h2, l1 + l2, loc="upper right", fontsize=6.4)
    axR2.text(4200, 23400, "充电量饱和（7 500 kW）：%s kWh/日" % fnum(pchg[-1]),
              fontsize=6.3, color="#1a5276", ha="left", va="bottom")

    FIG_SOURCES["fig04_q1_capacity_power.pdf"] = (
        "exp_scan.json: capacity.cap_scan / power_scan / base / no_capacity_limit / "
        "no_power_limit / gain_no_cap / gain_no_pow；边际价值为扫描费用的离散差分")
    return save(fig, "fig04_q1_capacity_power.pdf")


def fig05():
    """问题 1 成本分解瀑布图：无储能基线 -> 储能削峰填谷收益 -> 最优购电费。"""
    q1 = load_json("q1.json")["main"]
    base = q1["base_cost"]
    opt = q1["cost"]
    save_amt = q1["saving"]

    fig, ax = plt.subplots(figsize=(4.9, 3.9))
    xs = [0, 1, 2]
    ax.bar(0, base, width=0.56, color="0.72", edgecolor="0.35", linewidth=0.7)
    ax.bar(1, -save_amt, bottom=base, width=0.56, color="#27ae60",
           edgecolor="#1e8449", linewidth=0.7)
    ax.bar(2, opt, width=0.56, color=C_PRICE, edgecolor="#16365c", linewidth=0.7)

    # 连接线
    ax.plot([0.28, 1.72], [base, base], ls="--", lw=0.8, color="0.45")
    ax.plot([1.28, 2.28], [opt, opt], ls="--", lw=0.8, color="0.45")

    ax.text(0, base + 1400, "%s 元" % fnum(base), ha="center", fontsize=7.6)
    ax.text(1, base + 1400, "-%s 元" % fnum(save_amt), ha="center", fontsize=7.6,
            color="#1e8449")
    ax.text(2, opt + 1400, "%s 元" % fnum(opt), ha="center", fontsize=7.6,
            color=C_PRICE)

    ax.set_xticks(xs)
    ax.set_xticklabels(["无储能基线 $C^{base}$\n（净负荷全额购电）",
                        "储能削峰填谷收益\n（价差套利 + 光伏时移）",
                        "最优购电费 $C^*$\n（含储能，日周期）"], fontsize=7.4)
    ax.set_ylabel("日购电费用（元 · 日$^{-1}$）")
    ax.set_ylim(0, 72000)
    note(ax, "$C^{base}=\\sum_t p_t\\max(L_t-PV_t,0)\\Delta t$\n"
             "口径 C-01 / C-02，附件 1 典型日（全年平均日）\n"
             "节省率 %.2f%%（%s 元/日）"
         % (q1["saving_rate"] * 100, fnum(save_amt)), loc="upper right", fs=6.8)
    FIG_SOURCES["fig05_cost_waterfall.pdf"] = (
        "q1.json: main.base_cost / main.cost / main.saving / main.saving_rate")
    return save(fig, "fig05_cost_waterfall.pdf")


def fig06():
    """信息价值 VoI：预报时刻数 1/2/4/完全信息 的总费用（左轴）与紧急购电量（右轴）。"""
```

```
    m1 = load_json("q3_n1.json")["meta"]
    m2 = load_json("q3_n2.json")["meta"]
    m4 = load_json("q3_n4.json")["meta"]
    mp = load_json("q3_perf.json")["meta"]
    metas = [m1, m2, m4, mp]
    cost = np.array([m["total_cost"] for m in metas]) / 1e4      # 万元
    emg = np.array([m["emg_qty"] for m in metas])                # kWh
    labels = ["1 次\n0:00", "2 次\n0:00 + 12:00",
              "4 次\n0:00 / 6:00 / 12:00 / 18:00", "完全信息\n（上界）"]
    x = np.arange(4)

    fig, ax = plt.subplots(figsize=(6.7, 4.2))
    axb = ax.twinx()
    axb.bar(x, emg, width=0.42, color="#aed6f1", edgecolor="#5499c7",
            linewidth=0.7, label="紧急购电量（kWh，右轴）")
    axb.set_ylabel("紧急购电量（kWh）", color="#2471a3")
    axb.tick_params(axis="y", colors="#2471a3")
    axb.set_ylim(0, 760000)
    axb.grid(False)
    for xi, v in zip(x, emg):
        axb.text(xi, v + 9000, fnum(v, 0), ha="center", fontsize=6.8, color="#1a5276")

    ax.plot(x, cost, "o-", color=C_MU, ms=5.0, lw=1.6, label="总费用（万元，左轴）")
    for xi, v in zip(x, cost):
        ax.annotate(fnum(v), (xi, v), textcoords="offset points", xytext=(0, 7),
                    ha="center", fontsize=7.0, color=C_MU)
    for i in range(3):
        dcost = (cost[i] - cost[i + 1]) * 1e4
        ax.annotate("-%s 元" % fnum(dcost, 0),
                    ((x[i] + x[i + 1]) / 2, (cost[i] + cost[i + 1]) / 2),
                    textcoords="offset points", xytext=(0, -14), ha="center",
                    fontsize=6.8, color="0.25")

    ax.axvline(2, color="0.35", ls="--", lw=1.0)
    ax.annotate("本题实际：4 次预报", xy=(2.0, 1418.0), xytext=(1.92, 1423.0),
                ha="right", fontsize=7.0, color="0.25",
                arrowprops=dict(arrowstyle="->", color="0.45", lw=0.7))
    ax.set_xticks(x)
    ax.set_xticklabels(labels, fontsize=7.2)
    ax.set_xlim(-0.55, 3.55)
    ax.set_ylim(1384.5, 1445.0)
    ax.set_xlabel("预报时刻数（同一场景集、同一实际轨迹，结果窗口 %d 天）"
                  % m1["n_days"])
    ax.set_ylabel("总费用（万元）", color=C_MU)
    ax.tick_params(axis="y", colors=C_MU)
    h1, l1 = ax.get_legend_handles_labels()
    h2, l2 = axb.get_legend_handles_labels()
    ax.legend(h1 + h2, l1 + l2, loc="upper right", fontsize=6.8)

    voi = m1["total_cost"] - mp["total_cost"]
    got = m1["total_cost"] - m4["total_cost"]
    note(ax, "可获得信息价值上界（1 次 → 完全信息）：%s 元\n"
             "4 次已实现 %.1f%%；再加密最多再省 %s 元（%.2f%%）\n"
             "无储能基线 %s 元（%s 万元，超出纵轴范围）"
         % (fnum(voi), got / voi * 100, fnum(m4["total_cost"] - mp["total_cost"]),
            (m4["total_cost"] - mp["total_cost"]) / m4["total_cost"] * 100,
            fnum(m1["base_cost"]), fnum(m1["base_cost"] / 1e4, 2)),
         loc="upper left", fs=6.6)
    FIG_SOURCES["fig06_voi.pdf"] = (
        "q3_n1.json / q3_n2.json / q3_n4.json / q3_perf.json 的 meta."
        "total_cost、meta.emg_qty、meta.base_cost")
    return save(fig, "fig06_voi.pdf")


def fig07():
    """问题 2 全年逐日紧急购电量 + 报童两界（缺货概率 1/10 与 1/5）。"""
    q2 = load_json("q2_main.json")
    meta = q2["meta"]
    nb = load_json("exp_scan.json")["newsboy"]
    dates = [datetime.strptime(r["date"][:10], "%Y-%m-%d") for r in q2["daily"]]
    g = np.array([r["emg_qty"] for r in q2["daily"]], float)

    fig, ax = plt.subplots(figsize=(7.3, 3.3))
```

```
    xs = date2num(dates)
    ax.bar(xs, g, width=0.82, color="#aed6f1", edgecolor="#5499c7",
           linewidth=0.35, label="逐日紧急购电量 $\\sum_t g_t$（kWh/日）")

    k = 7
    roll = np.convolve(g, np.ones(k) / k, mode="same")
    ax.plot(xs, roll, color="#1a5276", lw=1.2, label="%d 日滑动平均（kWh/日）" % k)

    q80 = float(np.percentile(g, 80))
    q90 = float(np.percentile(g, 90))
    ax.axhline(q80, color="#e67e22", ls="--", lw=1.1,
               label="报童界 1/5：日 %d%% 分位 = %s kWh" % (80, fnum(q80)))
    ax.axhline(q90, color="#c0392b", ls="-.", lw=1.1,
               label="报童界 1/10：日 %d%% 分位 = %s kWh" % (90, fnum(q90)))

    month_ticks = [d for d in dates if d.day == 1]          # 仅 2025 年各月 1 日，避免出现次年 1 月
    ax.set_xticks(date2num(month_ticks))
    ax.set_xticklabels([d.strftime("%m月") for d in month_ticks])
    ax.set_xlim(date2num(dates[0]) - 2, date2num(dates[-1]) + 2)
    ax.set_ylim(0, 5800)
    ax.set_xlabel("日期（2025-02-01 至 2025-12-31，共 %d 天）" % len(dates))
    ax.set_ylabel("紧急购电量（kWh · 日$^{-1}$）")
    ax.legend(loc="upper left", fontsize=6.4, ncol=1)
    note(ax, "逐槽缺货频率 %.2f%%（报童两界 1/10 – 1/5 的下侧）\n"
             "电量满足率 %.2f%%；全年紧急购电量 %s kWh\n"
             "有紧急购电天数 %d/%d；单日最大 %s kWh（%s）"
         % (nb["short_slot_rate"] * 100, nb["fill_rate"] * 100,
            fnum(meta["tot_emg_qty"]), meta["days_with_emg"], len(dates),
            fnum(g.max()), dates[int(np.argmax(g))].strftime("%Y-%m-%d")),
         loc="upper right", fs=6.4)
    FIG_SOURCES["fig07_q2_emergency.pdf"] = (
        "q2_main.json: daily[].emg_qty（334 天）、meta.tot_emg_qty、meta.days_with_emg；"
        "exp_scan.json: newsboy.short_slot_rate / newsboy.fill_rate")
    return save(fig, "fig07_q2_emergency.pdf")


def fig08():
    """附件 4 电价剖面（分位带 + 分时骨架）+ 残差 ACF(1 – 6)。"""
    pt = load_json("q4_price.json")["price_tests"]
    # 分时骨架：优先用附件 1 原始剖面，缺失时退回 q1.json 的电价数组
    try:
        skel, _, _ = read_attach1()
        skel_src = "附件 1"
    except Exception:
        skel = np.asarray(load_json("q1.json")["arrays"]["price"], float)
        skel_src = "q1.json arrays.price（附件 1 同源）"

    band_ok, M = False, None
    try:
        M = read_attach_matrix("附件4.xlsx")
        band_ok = M.shape[1] == 144 and M.shape[0] > 0
    except Exception:
        band_ok = False

    t = slot_hours()
    xe, ske = step_close(t, skel)

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(6.9, 5.2),
                                   gridspec_kw=dict(height_ratios=[1.5, 1.0], hspace=0.30))

    if band_ok:
        q = np.percentile(M, [10, 25, 50, 75, 90], axis=0)
        qe = np.hstack([q, q[:, [-1]]])
        ax1.fill_between(xe, qe[0], qe[4], step="post", color="#d6e4f0",
                         label="附件 4 电价分位带：10 – 90%% / 25 – 75%%（%d 天 × 144 槽）"
                               % M.shape[0])
        ax1.fill_between(xe, qe[1], qe[3], step="post", color="#9ec5e8",
                         label="_nolegend_")
        ax1.step(xe, qe[2], where="post", color="#1a5276", lw=1.2,
                 label="附件 4 逐槽中位数电价（元/kWh）")
    ax1.step(xe, ske, where="post", color=C_PRICE, lw=1.7, ls="--",
             label="分时骨架（%s 电价剖面，元/kWh）" % skel_src)
    ax1.set_ylabel("电价（元 · kWh$^{-1}$）")
```

```
    ax1.set_ylim(0, 2.4)
    ax1.legend(loc="upper left", fontsize=6.3, ncol=1)
    hour_axis(ax1)
    ax1.set_xlabel("")
    note(ax1, "全年电价均值 %s、最小 %s、最大 %s 元/kWh\n"
              "骨架与附件 4 逐时段均值最大偏差 %.2e 元/kWh"
         % (fnum(pt["price_mean"], 4), fnum(pt["price_min"], 4),
            fnum(pt["price_max"], 4), pt["skeleton_is_mean"]),
         loc="upper right", fs=6.6)

    acf = np.asarray(pt["C2_acf_lag1_6"], float)
    lags = np.arange(1, len(acf) + 1)
    ax2.bar(lags, acf, width=0.5, color="#5dade2", edgecolor="#2471a3",
            linewidth=0.7, label="日内电价残差自相关 ACF($k$)，$k$ 以 10 min 为步长")
    for xi, v in zip(lags, acf):
        ax2.text(xi, v + 0.018, "%.4f" % v, ha="center", fontsize=6.8, color="#1a5276")
    ax2.axhline(0, color="0.3", lw=0.7)
    ax2.set_ylim(0, 1.18)
    ax2.set_xticks(lags)
    ax2.set_xlabel("滞后阶数 $k$（10 min）")
    ax2.set_ylabel("自相关系数")
    ax2.legend(loc="upper right", fontsize=6.8)
    dacf = np.asarray(pt["C2b_daily_acf"], float)
    note(ax2, "日级残差 ACF：lag1 %.4f、lag2 %.4f、lag3 %.4f\n"
              "峰段/谷段波动比 %.4f（%.4f / %.4f 元/kWh）"
         % (dacf[0], dacf[1], dacf[2], pt["C3_sd_ratio"],
            pt["C3_sd_peak"], pt["C3_sd_night"]),
         loc="lower left", fs=6.6)
    FIG_SOURCES["fig08_q4_price.pdf"] = (
        "q4_price.json: price_tests.C2_acf_lag1_6 / C2b_daily_acf / C3_sd_ratio / "
        "C3_sd_peak / C3_sd_night / price_mean / price_min / price_max / skeleton_is_mean；"
        "分位带与骨架由附件 4、附件 1 原始数据计算")
    return save(fig, "fig08_q4_price.pdf")


def fig09(cvar=None, out_name="fig09_cvar_frontier.pdf"):
    """CVaR 风险-费用前沿：lambda 扫描下的期望费用 / CVaR95 / 真实轨迹回测费用。

    q4_cvar.json 由 code/q4.py --mode cvar 生成，结构：
        {"days": [...], "frontier": [{"lam","exp_cost","real_cost","cvar","emg_qty"}, ...]}
    缺失时跳过本图（由 main() 报告），不影响其余 7 张图。
    """
    if cvar is None:
        cvar = load_json("q4_cvar.json", required=False)
    if not cvar:
        return None
    fr = cvar["frontier"]
    lam = np.array([r["lam"] for r in fr], float)
    exp_c = np.array([r["exp_cost"] for r in fr], float)
    cvar_v = np.array([r["cvar"] for r in fr], float)
    real_c = np.array([r["real_cost"] for r in fr], float)
    emg = np.array([r["emg_qty"] for r in fr], float)
    ndays = len(cvar.get("days", []))

    fig, (axL, axR) = plt.subplots(1, 2, figsize=(7.3, 3.4))
    fig.subplots_adjust(wspace=0.56)

    axL.plot(lam, exp_c, "o-", color=C_PRICE, ms=3.4, label="情景期望费用（元/日）")
    axL.plot(lam, cvar_v, "s-", color=C_MU, ms=3.4, label="CVaR$_{0.95}$（元/日）")
    axL.set_xlabel(r"风险厌恶系数 $\lambda$")
    axL.set_ylabel("费用 / 风险（元 · 日$^{-1}$）")
    axL.legend(loc="upper left", fontsize=6.8)

    axL2 = axL.twinx()
    axL2.plot(lam, real_c, "^--", color="#16a085", ms=3.4, lw=1.1,
              label="真实轨迹回测费用（元/日，n=%d）" % ndays)
    axL2.set_ylabel("回测费用（元 · 日$^{-1}$）", color="#16a085")
    axL2.tick_params(axis="y", colors="#16a085")
    axL2.grid(False)
    h1, l1 = axL.get_legend_handles_labels()
    h2, l2 = axL2.get_legend_handles_labels()
    axL.legend(h1 + h2, l1 + l2, loc="upper left", fontsize=6.6)
    note(axL, "紧急购电量 %.1f → %.1f kWh/日\n（$\\lambda$ 由 0 增至 %.1f）"
```

```
          % (emg[0], emg[-1], lam[-1]), loc="lower right", fs=6.6)

    # 前沿散点：期望费用 - CVaR
    axR.plot(exp_c, cvar_v, "-", color="0.55", lw=1.0, zorder=1)
    sc = axR.scatter(exp_c, cvar_v, c=lam, cmap="viridis", s=26, zorder=3)
    for i, (xv, yv, lv) in enumerate(zip(exp_c, cvar_v, lam)):
        if i % 2:                    # 每隔一个点标注，避免标签互相压叠
            continue
        axR.annotate(r"$\lambda$=%.1f" % lv, (xv, yv), textcoords="offset points",
                     xytext=(4, 3), fontsize=6.2, color="0.25")
    # 拐点：单位期望费用增量带来的 CVaR 下降最大
    if len(lam) >= 3:
        slope = -np.diff(cvar_v) / np.maximum(np.diff(exp_c), 1e-9)
        k = int(np.argmax(slope))
        axR.scatter([exp_c[k + 1]], [cvar_v[k + 1]], marker="*", s=150,
                    facecolor="none", edgecolor="#c0392b", linewidth=1.2,
                    label=r"边际风险下降最快点 $\lambda$=%.1f" % lam[k + 1])
        axR.legend(loc="lower left", fontsize=6.6)
    cb = fig.colorbar(sc, ax=axR, pad=0.02)
    cb.set_label(r"$\lambda$", fontsize=7)
    cb.ax.tick_params(labelsize=6.5)
    axR.set_xlabel("情景期望费用（元 · 日$^{-1}$）")
    axR.set_ylabel("CVaR$_{0.95}$（元 · 日$^{-1}$）")
    FIG_SOURCES[out_name] = (
        "q4_cvar.json: frontier[].lam / exp_cost / cvar / real_cost / emg_qty，"
        "days（真实轨迹回测子样本，n=%d）" % ndays)
    return save(fig, out_name)


def fig10():
    """问题 1 最优策略机制：PV 盈余/负荷、电价与储电量、充放电量（三子图共享时间轴）。"""
    q1 = load_json("q1.json")
    A = q1["arrays"]
    price = np.asarray(A["price"], float)
    load = np.asarray(A["load"], float)
    pv = np.asarray(A["pv"], float)
    c = np.asarray(A["c"], float)
    d = np.asarray(A["d"], float)
    S = np.asarray(A["S"], float)

    t = slot_hours()
    xe, le = step_close(t, load)
    _, pve = step_close(t, pv)
    _, pe = step_close(t, price)

    surplus = pv > load
    sur_slots = np.where(surplus)[0]
    h_sur = (sur_slots.min() / 6.0, (sur_slots.max() + 1) / 6.0)   # PV 盈余窗口
    cheap = np.where(price <= 0.45)[0]                             # 低价时段（谷段）
    h_cheap = (cheap.min() / 6.0, (cheap.max() + 1) / 6.0)
    peak = np.where(price >= 1.25)[0]                              # 高价时段（峰段）
    h_peak = (peak.min() / 6.0, (peak.max() + 1) / 6.0)

    fig, axes = plt.subplots(3, 1, figsize=(6.9, 7.1), sharex=True,
                             gridspec_kw=dict(height_ratios=[1.0, 1.0, 1.0], hspace=0.16))
    axA, axB, axC = axes

    # 三条竖直参考带（跨子图对齐）
    for ax in axes:
        ax.axvspan(h_sur[0], h_sur[1], color="#f7dc6f", alpha=0.20, zorder=0)
        ax.axvspan(h_cheap[0], h_cheap[1], color="#85c1e9", alpha=0.20, zorder=0)
        ax.axvspan(h_peak[0], h_peak[1], color="#ec7063", alpha=0.13, zorder=0)

    # ---- (a) 功率 ----
    axA.fill_between(xe, le, pve, step="post", where=(pve > le), color=C_PV,
                     alpha=0.45, linewidth=0, label="光伏盈余 $PV_t-L_t$（kW）")
    axA.step(xe, le, where="post", color=C_LOAD, lw=1.4, label="小区负载 $L_t$（kW）")
    axA.step(xe, pve, where="post", color=C_PV, lw=1.4, label="光伏实际功率 $PV_t$（kW）")
    axA.set_ylabel("功率（kW）")
    axA.set_ylim(-600, 11600)
    axA.legend(loc="upper left", fontsize=6.6, ncol=1)
    sur_energy = float((np.maximum(pv - load, 0) * DT).sum())
    note(axA, "光伏盈余时段 %s–%s（%d 槽）：盈余电量 %s kWh\n"
```

```
                "该窗口内充电 %s kWh，占全天充电量 %.1f%%"
          % (hhmm(h_sur[0]), hhmm(h_sur[1]), len(sur_slots), fnum(sur_energy),
             fnum(c[surplus].sum()), c[surplus].sum() / c.sum() * 100),
          loc="upper right", fs=6.6)

    # ---- (b) 电价与储电量 ----
    axB.step(xe, pe, where="post", color=C_PRICE, lw=1.4,
             label="电价 $p_t$（附件 1，元/kWh）")
    axB.set_ylabel("电价（元 · kWh$^{-1}$）")
    axB.set_ylim(0.30, 1.62)
    axB.legend(loc="upper left", fontsize=6.8)
    axB2 = axB.twinx()
    axB2.step(xe, np.append(S, S[-1]), where="post", color=C_SOC, lw=1.5,
              label="储电量 $S_t$（kWh，右轴）")
    axB2.text(0.3, 5700, "起止储电量 %s kWh（$S_0=S_{144}$，日周期）"
              % fnum(q1["main"]["s0"], 0), fontsize=6.4, color=C_SOC,
              ha="left", va="top")
    axB2.axhline(S_MAX, color=C_SOC, ls=":", lw=1.0)
    axB2.axhline(S_MIN, color=C_SOC, ls=":", lw=1.0)
    axB2.axhline(S_INIT, color="0.45", ls=":", lw=0.9)
    axB2.plot([0, 24], [S_INIT, S_INIT], "o", color=C_SOC, ms=3.4, zorder=5)
    axB2.set_ylim(0, 12600)
    axB2.set_ylabel("储电量（kWh）", color=C_SOC)
    axB2.tick_params(axis="y", colors=C_SOC)
    axB2.grid(False)
    axB2.text(7.5, 10950, "上界 %s kWh（触界 %d/144 槽）"
              % (fnum(S_MAX, 0), int(np.sum(S >= S_MAX - 1e-6))),
              fontsize=6.4, color=C_SOC, ha="left", va="bottom")
    axB2.text(13.5, 1300, "下界 %s kWh（触界 %d/144 槽）"
              % (fnum(S_MIN, 0), int(np.sum(S <= S_MIN + 1e-6))),
              fontsize=6.4, color=C_SOC, ha="left", va="bottom")
    h1, l1 = axB.get_legend_handles_labels()
    h2, l2 = axB2.get_legend_handles_labels()
    axB.legend(h1 + h2, l1 + l2, loc="upper left", fontsize=6.6)

    # ---- (c) 充放电量 ----
    w = 1.0 / 6.0
    axC.bar(t + w / 2, c, width=w, color=C_CHG, label="充电量 $c_t$（kWh/槽）")
    axC.bar(t + w / 2, -d, width=w, color=C_DIS, label="放电量 $d_t$（kWh/槽）")
    axC.axhline(0, color="0.25", lw=0.7)
    axC.axhline(E_MAX, color="0.45", ls=":", lw=0.9)
    axC.axhline(-E_MAX, color="0.45", ls=":", lw=0.9)
    axC.set_ylim(-1400, 1400)
    axC.set_ylabel("充放电量（kWh · 槽$^{-1}$）\n充电为正、放电为负")
    hour_axis(axC)
    axC.legend(loc="upper left", fontsize=6.6, ncol=1)
    note(axC, "全天充电 %s kWh、放电 %s kWh，充放比 %.4f\n"
              "功率打满上限（±%s kWh/槽）：充电 %d/144 槽、放电 %d/144 槽\n"
              "（黄：PV 盈余段；蓝：谷段低价；红：峰段高价）"
         % (fnum(c.sum()), fnum(d.sum()), c.sum() / d.sum(), fnum(E_MAX, 0),
            int(np.sum(c >= E_MAX - 1e-6)), int(np.sum(d >= E_MAX - 1e-6))),
         loc="upper right", fs=6.4)

    for ax in axes:
        ax.set_xlim(0, 24)
    FIG_SOURCES["fig10_strategy_mechanism.pdf"] = (
        "q1.json: arrays.price / arrays.load / arrays.pv / arrays.S / arrays.c / "
        "arrays.d，main.s0；阴影窗口由数组本身界定（PV>L 盈余段、p_t 0.45 谷段、"
        "p_t 1.25 峰段）")
    return save(fig, "fig10_strategy_mechanism.pdf")


# ---------------------------------------------------------------------------
# 4. 主程序
# ---------------------------------------------------------------------------
def main():
    os.makedirs(FIGDIR, exist_ok=True)
    made, skipped = [], []

    jobs = [("fig03_q1_dual_price.pdf", fig03),
            ("fig04_q1_capacity_power.pdf", fig04),
            ("fig05_cost_waterfall.pdf", fig05),
            ("fig06_voi.pdf", fig06),
```

```
            ("fig07_q2_emergency.pdf", fig07),
            ("fig08_q4_price.pdf", fig08),
            ("fig09_cvar_frontier.pdf", fig09),
            ("fig10_strategy_mechanism.pdf", fig10)]

    for name, fn in jobs:
        try:
            path = fn()
        except Exception as exc:            # 单图失败不阻断其余图
            skipped.append((name, "%s: %s" % (type(exc).__name__, exc)))
            print("[失败] %-34s %s: %s" % (name, type(exc).__name__, exc))
            continue
        if path is None:
            skipped.append((name, "依赖的中间结果缺失（q4_cvar.json 未生成）"))
            print("[跳过] %-34s 依赖的中间结果缺失" % name)
        else:
            made.append((name, os.path.getsize(path)))

    print("\n" + "=" * 78)
    print("已生成矢量 PDF: ")
    for name, size in made:
        print("  %-34s %9d 字节   来源: %s" % (name, size, FIG_SOURCES.get(name, "?")))
    if skipped:
        print("未生成: ")
        for name, why in skipped:
            print("  %-34s %s" % (name, why))
    print("输出目录: %s" % FIGDIR)
    print("=" * 78)
    return made, skipped


if __name__ == "__main__":
    main()
```

附录 F　数据字典

表 27　附件字段、单位与时标约定

附件	字段	单位	时间范围	时标约定
附件 1	电价	元/kWh	一天（144 时段）	标签 = 时段末端；0:00+1 表示次日 0:00（当日 24:00）
附件 1	小区负载	kW	同上	同上，**实为附件 2 全年逐时段均值**（见下）
附件 1	光伏发电「预测」功率	kW	同上	**实为附件 2 全年逐时段均值，不是预报**（见下）
附件 2	小区负载	kW	2025-01-01 至 12-31	365 行 × 144 列，标签 = 时段末端
附件 2	光伏发电实际功率	kW	同上	同上
附件 3	光伏发电预报	kW	同上	每天 0/6/12/18 时发布，预报 k 小时对应 $(h_0+k-1,\ h_0+k]$
附件 4	电价	元/kWh	同上	365 行 × 144 列，标签 = 时段末端
附件 5	结果文件模板	—	同上	工作表名、列名与列序即本文结果文件的格式约定

附件 1 的身份核查（数据字典的关键一条）　附件 1 的**电价、小区负载、光伏发电「预测」功率三列，实为附件 2 与附件 4 的全年逐时段均值，并不是日前预报**：以 365 天逐时段均值逐槽比对，电价的最大偏差为 5.15×10^{-5} 元/kWh （`intermediate/q4_price.json` 的 `skeleton_is_mean` 字段），负载的最大偏差为 1×10^{-4} kW（`paper_numbers.json`），光伏的最大偏差为 0.0431 kW。因此附件 1 **只能用于问题 1 与全文基线口径**：问题 1 的典型日与无储能基线 C^{base} 用它，问题 2–4 的日前计划一律改用附件 3 的当次预报（问题 2 用 0:00 预报，问题 3 在 0:00、6:00、12:00、18:00 四次更新），结算一律用附件 2 与附件 4 的实际值。若把附件 1 当作日前预报，计划就脱离了该时刻真正可得的信息，属信息结构的口径错误。另有一个窗口口径需同时说明：只用结果窗口 2025-02-01 至 12-31 的 334 天求均值时，电价的最大偏差约 3.1×10^{-2} 元/kWh，即**窗口内均值并不完全等于附件 1**，故问题 2–4 也不能把附件 1 当作窗口内的均值曲线使用。

附录 G　长推导

7.1　对偶推导与阈值律

记式 (2a) 的乘子为 α_t（电量平衡的影子价格，在正常购电时段等于电价，见式 (26)），式 (2b) 的乘子为 ν_t（储电量的边际价值，即水价值），储电量上下界乘子为 $\lambda_t^{\pm}\ge0$，充放电功率界乘子为 $\varrho_t^c,\varrho_t^d\ge0$。

把电量平衡写成 $(L_t - PV_t)\Delta t + c_t + s_t - E_t - d_t = 0$，拉格朗日函数为

$$\begin{aligned}\mathcal{L} = &\sum_t p_t E_t + \sum_t \alpha_t \Big[(L_t - PV_t)\Delta t + c_t + s_t - E_t - d_t\Big] \\ &+ \sum_t \nu_t \Big[S_t - S_{t-1} - \eta_c c_t + \frac{d_t}{\eta_d}\Big] + \sum_t \lambda_t^{+}\big[S_t - \overline{S}\big] + \sum_t \lambda_t^{-}\big[\underline{S} - S_t\big] \\ &+ \sum_t \varrho_t^{c}\big[c_t - \overline{P}\Delta t\big] + \sum_t \varrho_t^{d}\big[d_t - \overline{P}\Delta t\big]\end{aligned} \tag{20}$$

对各决策变量求平稳性条件（E_t, s_t, c_t, d_t 对 $t = 1, \dots, 144$；S_t 对 $t = 1, \dots, 143$，因为日周期约束把 $S_{144} = S_0 = 6000$ kWh 固定）：

$$\frac{\partial \mathcal{L}}{\partial E_t} = p_t - \alpha_t \ \geq 0, \qquad E_t\big(p_t - \alpha_t\big) = 0 \tag{21}$$

$$\frac{\partial \mathcal{L}}{\partial s_t} = \alpha_t \ \geq 0, \qquad s_t\,\alpha_t = 0 \tag{22}$$

$$\frac{\partial \mathcal{L}}{\partial c_t} = \alpha_t - \eta_c\nu_t + \varrho_t^{c} \ \geq 0, \qquad c_t\big(\alpha_t - \eta_c\nu_t + \varrho_t^{c}\big) = 0 \tag{23}$$

$$\frac{\partial \mathcal{L}}{\partial d_t} = -\alpha_t + \frac{\nu_t}{\eta_d} + \varrho_t^{d} \ \geq 0, \qquad d_t\Big(-\alpha_t + \frac{\nu_t}{\eta_d} + \varrho_t^{d}\Big) = 0 \tag{24}$$

$$\frac{\partial \mathcal{L}}{\partial S_t} = \nu_t - \nu_{t+1} + \lambda_t^{+} - \lambda_t^{-} = 0 \tag{25}$$

由式 (21)，购电时段（$E_t > 0$）恒有

$$\alpha_t = p_t \tag{26}$$

即电量平衡的影子价格等于该时段的电价；数值上逐槽复核的最大偏差为 0.000 元/kWh。若充放电功率界均不活跃（$\varrho_t^{c} = \varrho_t^{d} = 0$），式 (23) 与式 (24) 化为

$$\alpha_t - \eta_c\nu_t \ \geq 0, \qquad \frac{\nu_t}{\eta_d} - \alpha_t \ \geq 0 \tag{27}$$

且 $c_t > 0$ 时前式取等、$d_t > 0$ 时后式取等，即得正文命题 2 的阈值律：$\alpha_t \leq \eta_c\nu_t$ **时充电**、$\alpha_t \geq \nu_t/\eta_d$ **时放电**，死区为 $\eta_c\nu_t < \alpha_t < \nu_t/\eta_d$，死区比为 $1/(\eta_c\eta_d) = 1.2346$（单边效率各 0.9、往返 0.81），与式 (4) 一致。由式 (25) 还可得水价值的跨期关系：在储电量上下界均不活跃的连续时段上 $\nu_t = \nu_{t+1}$（水价值恒定）；储电量触及上界时 $\lambda_t^{+} > 0$、$\nu_t < \nu_{t+1}$；触及下界时 $\lambda_t^{-} > 0$、$\nu_t > \nu_{t+1}$。这一关系是图 3 中水价值曲线出现折点的原因。

数值复核（`intermediate/q1_dual2.json`）：储电量双界均不活跃的槽位 97 个；$\nu_t \in [0.4711,\ 1.1305]$ 元/kWh，$\alpha_t \in [0.3713,\ 1.2561]$ 元/kWh；全部 144 槽的充放电阈值律违反次数为 0，互补松弛最大残差为 0.000 元/kWh，式 (26) 的最大偏差为 0.000 元/kWh。

7.2 报童临界分位数的推导

设时段 t 的需求 D_t 为随机变量、分布函数为 F_t，计划购电量为 E_t，过剩成本为 c_o（多买 1 kWh 的损失），缺货成本为 c_u（不足 1 kWh 的额外支出）。期望总成本为

$$\Pi(E) = c_o\,\mathbb{E}\big[(E - D)^{+}\big] + c_u\,\mathbb{E}\big[(D - E)^{+}\big] \tag{28}$$

对 E 求导并令其为零：

$$\frac{\mathrm{d}\Pi}{\mathrm{d}E} = c_o F(E) - c_u\big[1 - F(E)\big] = 0 \quad\Longrightarrow\quad F(E^{*}) = \frac{c_u}{c_u + c_o} \tag{29}$$

本题中不足 1 kWh 需按 $5p_t$ 紧急购电，而正常电价 p_t 已随计划购电量支付，故缺货的额外支出为 $c_u = 5p_t - p_t = 4p_t$；超购的电量若无处可用（超购即弃置），则 $c_o = p_t$，代入式 (29) 得

$$F(E^{*}) = \frac{4p_t}{4p_t + p_t} = \frac{4}{5} = 0.8 \qquad\Longrightarrow\qquad \text{最优缺货概率} = 1 - F(E^{*}) = \frac{1}{5} = 0.2 \tag{30}$$

即「留安全边际」的临界分位数为 4/5、最优缺货概率为 1/5。若超购电量能转入储能、留待后续更高价时段使用（单位回收价值 r_t），则 $c_o = p_t - r_t$，临界分位数相应上移：取全年均价 $p = 0.7662$ 元/kWh

与全年最低价 $p^{low} = 0.3713$ 元/kWh，得 $c_o \approx 0.395$ 元/kWh，$F(E^*) \approx 0.886$，最优缺货概率降至约 1/10。此即正文式 (9) 与表 12 的来源。

实测对照（`paper_numbers.json` 的「报童经验」）：主口径下逐槽缺货频率为 8.55%，缺口电量 172 015.82 kWh，电量满足率 99.04%，安全边际合计 576 299.67 kWh （11.98 kWh/槽），落在 1/10 与 1/5 两界**之下侧**——因为储能同时承担了对冲功能，报童结论只作方向性近似与上下界使用，不作精确刻画。

附录 H　随机种子清单

本工程**只在四处使用随机数**，四问的线性规划求解、主口径的场景削减、审计与数值汇总均为确定性过程（同一输入必得同一输出）。逐项清单如表 28。

表 28　随机种子清单（代码中的全部随机源；其余过程确定）

用途	随机机制	种子	说明
问题 4 联合情景合成（条件对照）	分块 Bootstrap，按时刻对齐 6 h 块重采样（`q4.build_joint_scenarios(mode="boot")`）	`numpy.random.default_rng(seed=7)`	电价块与光伏预报残差块成对抽取，保住日内形状与相关性；一次生成后固定使用
场景削减的对照路径	k-means 初始化（`lp_engine.reduce_scenarios`）	`random_state=0`，`n_init=4`	仅在 `--scen kmeans` 对照下启用；**主口径不抽样**：按日总残差等频分层后在层内取 medoid，确定性
非预期性置换检验	对某次预报做日间随机置换（`exp_na.py`）	`numpy.random.default_rng(seed=6)`，对照用 `seed=1`	检验「置换 000 预报后槽 0–35 完全不变、槽 36–71 改变」，用于排除信息泄漏
全局灵敏度的采样	Sobol 低差异序列置乱（`scipy.stats.qmc.Sobol(scramble=True)`，`exp_sobol.py`）	费用目标 `seed=42`；节省率目标 `seed=43`	置乱只改变采样点的取法，不改变参数区间与模型本身

确定性的部分（无随机数）：问题 1–4 的全部线性规划由 SciPy `linprog(method="highs")` 精确求解；问题 2、3 的场景削减用「等频分层加层内 medoid」，问题 4 的历史代表日情景用「日净需求残差与日均电价」二维分层取 medoid、按层样本占比赋权，两者都是确定性算法；`audit.py`、`paper_numbers.py` 与 `exp_scan.py` 不含随机数。因此论文的每一个数字在同一环境下重跑即可复现。

诚实说明 ：本工程**未开展「多种子重复生成情景」的稳健性实验**，故不报告费用的均值、标准差与 95% 置信区间，也不列出 20 个种子。本工程对这一缺口的处理是：把随机性限制在只影响未来情景、不影响已实现时段决策的合成与采样环节（分块 Bootstrap 只作为问题 4 联合情景的条件对照，主口径用确定性的分层 medoid），并以同源对照替代多种子重复——同一固定情景集下的信息结构对照（R1、R2、R3）、场景数对照（$K = 60$ 与 $K = 120$）、终端条件对照与安全边际消融，四者的结论方向一致。若需要多种子区间，只需改动 `build_joint_scenarios` 的 `seed` 参数并重跑 `run_all.py`，其余环节无需修改。

附录 I　验证与灵敏度明细

本节汇集正文第九章与第三章各表的完整明细，便于复核；正文已给出各表的结论与判据。

附录 J　场景保真度与电价检验明细

本节给出正文第五章场景集与第七章电价过程检验、CVaR 风险—费用前沿的完整明细表。

附录 K　验证映射、风险结果与局限明细

本节给出正文第九章验证层级映射、第八章问题 4 逐日期结果与第十章局限清单的完整表格。

附录 L　附件结构与问题 4 逐日期结果明细

本节给出四个附件的字段结构与问题 4 在四个指定日期的逐项结果。

【问题 4 逐日期结果】

表 29　独立审计项、阈值与实测结果（内部解：问题 1、2、3 的 10 min 级解；交付物：result1、result2、result3、result4-2、result4-3 共五个结果文件；窗口 2025-02-01 至 2025-12-31，共 334 天）

项	检查内容与阈值	内部解实测	交付物实测	判定
A1	功率/电量平衡残差（式 (2a)）：$\max_t \lvert E_t + PV_t\Delta t + d_t + g_t - L_t\Delta t - c_t - s_t\Delta t\rvert \le 10^{-6}$ kWh	问题 1 1.49×10^{-13}；问题 2、3 3.41×10^{-13} kWh	交付物无逐槽 L, PV 列，平衡只能在内部解上核验（汇总列对拍见 A10）	PASS
A2	储电量递推（式 (2b)）：$\max_t \lvert S_t - S_{t-1} - \eta_c c_t + d_t/\eta_d\rvert \le 10^{-6}$ kWh	问题 1 9.09×10^{-13}；问题 2、3 1.09×10^{-11} kWh	问题 1 3.64×10^{-12}；问题 2、3、4-2、4-3 为 $2.96/2.72/2.11/2.61\times10^{-4}$ kWh，为写入取整所致（见下注）	PASS[注]
A3	储电量边界：越界数 $=0$（区间 $[1200, 10800]$ kWh）	0	0	PASS
A4	充放功率边界：问题 1–3 按单槽上限 833.3333 kWh；交付物按块级上限 $24\overline{P}\Delta t = 20\,000$ kWh	0	0	PASS
A5	充放互斥：$\min(c_t, d_t) \le 10^{-6}$	违反槽数 0（充放同时槽数 0）	交付物只有 4 h 聚合量，块内可同时有充放电（问题 1 的 4:00–8:00 块即如此），故块级不可检验	PASS
A6	周期/终端：问题 1 要求 $S_0 - S_{144} = 0$；问题 2–4 要求跨日衔接且 $S_T \ge S_0$	问题 1 周期偏差 0.000；问题 2、3 末值低于初值的天数为 0	五个结果文件末值低于初值的天数均为 0	PASS
A7	非负性：所有电量与费用 $\ge -10^{-6}$	负购电槽数 0	负购电量数 0	PASS
A8	费用复算：计划 + 紧急 + 调整相关，相对误差 $\le 10^{-4}$	问题 1、2、3 的相对误差均为 0	结果文件无调整与紧急费用的分项列，交付物层只能核验计划购电费（最大相对偏差 4.32×10^{-7}，问题 4-2）；四项费用的复算在内部解上完成（相对误差 0）	PASS
A9	日期与结构：334 天、工作表名/列名/列序与模板一致	334 天	五个文件均为 334 行 ×147 列，日期范围 2025-02-01 至 2025-12-31，工作表名与模板逐项一致；紧急购电行数 1156/1190/1193/1187	PASS
A10	全天汇总列与逐时段求和一致：相对误差 $\le 10^{-4}$	—	全天购电量最大偏差 0.0012 kWh（问题 4-3）；全天购电费最大相对偏差 4.32×10^{-7}（问题 4-2）	PASS

[注] 交付物 A2 的 10^{-4} kWh 量级残差可由写入精度完全解释：`results_io.py::soc_sheet_frame()` 对 6 个块级充放电量与 0:00/24:00 两个储电量按 4 位小数写入，取整误差上界为 $6\times5\times10^{-5} + 2\times5\times10^{-5} = 4\times10^{-4}$ kWh，与实测值一致；`result1.xlsx` 由 `q1.py::write_result1()` 写入未取整的块级值，故其同一残差为 3.64×10^{-12} kWh。该残差不是物理越界（A3、A4 越界数均为 0）。

表 30　基线与消融对照矩阵（费用单位：元，电量单位：kWh；全部为 2025-02-01 至 2025-12-31 共 334 天、附件 1 固定电价口径；" 相对 C^{base}" 以无储能基线 16 407 319.63 元（口径 C-01 变体 V1）为分母；" 相对主模型" 以问题 2 的两阶段主模型为基准）

编号	配置（新增或移除的模块）	总费用	相对 C^{base}	紧急购电量	结论
B0	无储能：净负荷全额购电（闭式 口径 C-01）	16 407 319.63	0	—	基线
R2	典型日预报：计划依据改为附件 1 的日均形状	23 484 646.63	−43.14%	2 525 548.76	比无储能基线还贵 7 077 327.00 元，不可用
E0	无安全边际：确定性等价（$K=1$、零残差情景）	17 398 559.68	−6.04%	1 156 618.02	移除安全边际后反而高于无储能基线，**安全边际的边际价值** 3 133 767.54 **元**
R1	完美信息：0:00 知当日真值	13 411 560.90	18.26%	0.00	预测误差成本 853 231.24 元（下界）
M	主模型：0:00 一次预报 + 两阶段随机规划（$K=60$）	**14 264 792.13**	13.06%	172 015.82	主方案（基准行）
N2	两次预报：0:00 + 12:00	14 131 564.16	13.87%	122 524.83	比主模型省 133 227.98 元
N4	四次滚动：0/6/12/18 + 非预期性	13 932 402.95	15.08%	64 522.09	比主模型省 332 389.18 元（(见 C-Q3-01)）
PI	完全信息上界：四个时刻均知真值	13 874 340.39	15.44%	6 134.76	可获得信息价值的上界；四次预报已实现 85.13%
FY	全年单一 LP：完美信息、单一储电量轨迹、无日决策解耦（192 384 个变量）	12 228 166.56	25.47%	—	全局下界；**日决策解耦代价** 1 183 394.34 **元**（9.68%）
K120	场景数加倍（$K=120$，其余同主模型）	14 235 163.95	13.24%	141 707.94	费用仅降 29 628.19 元（−0.21%），结论稳健

表 31　单参数灵敏度与 Sobol 全局灵敏度（参数扰动档位、关键指标实测与”结论是否改变”的判断；单参数扫描除终端条件外均在同一组典型日参数下求解，Sobol 指数 $N = 256$、输出量已逐行注明；费用单位：元，电量单位：kWh）

参数	扰动档位	关键指标实测	Sobol S_1	结论是否改变
紧急购电倍数 κ	2、3、5（基准）、8、10	典型日安全边际 671.42/1 416.50/2 232.49/3 027.42/3 334.50 kWh；期望紧急购电 1 246.33/779.87/428.32/242.87/187.27 kWh；典型日总费用 39 224.54/39 941.12/40 747.45/41 437.52/41 758.99 元	—	方向不变：κ 越大安全边际越大、缺口越小。反转点在 $\kappa \to 1$：紧急购电与正常电价同价时缺货成本消失、临界分位数退化，”需要安全边际”的结论不再成立
效率口径 η_c, η_d	单边各 0.9（往返 0.81，**主口径**）、往返 0.9 等效单边、仅充电侧 0.9、仅放电侧 0.9	典型日总费用 35 126.95/33 801.06/33 416.53/34 168.64 元，节省 12 925.10/14 250.99/14 635.52/13 883.44 元，**极差** 13.23%	0.0695（对全天购电费）	区间内结论不变（四种读法下储能均有利可图），但节省幅度必须带效率口径，否则同一模型可报出相差 13.23% 的数
储电量运行区间 $[\underline{S}, \overline{S}]$	保持 10%–90% DOD：容量 7 200、9 600、12 000（基准）、14 400、19 200、28 800、48 000 kWh	典型日总费用 38 640.45/36 700.45/35 126.95/33 875.57/31 692.43/28 854.53/27 601.80 元，相对无储能基线的节省率由 19.59% 单调升到 42.56%	上限 0.6731；下限 0.1099（对全天购电费）	区间内结论不变：容量摆幅始终是主要瓶颈，仅解除容量上限可再省 22.14%
充放功率上限 $\overline{P}$	1 250、2 500、5 000（基准）、7 500、10 000 kW	典型日总费用 40 166.17/36 390.84/35 126.95/35 027.13/35 010.59 元，节省率 16.41%/24.27%/26.90%/27.11%/27.24%	0.0263（对全天购电费）	区间内结论不变：功率在 5000 kW 以上边际很小（0.28%、0.47%），但收紧到 2 500 kW 时节省率由 26.90% 降到 24.27%、1 250 kW 时降到 16.41%
终端条件（口径 C-08）	窗口：$S_T \geq S_0$（ge，主口径）/ 日环状（eq）/ 自由终端（free）；典型日：日周期 / 自由环状	问题 2 完美信息档 13 411 560.90（ge 与 eq 同值）/ 13 427 548.51（free，+0.12%）；主模型档 14 264 792.13（ge）/ 14 276 816.39（free，+0.08%）；典型日 35 126.9486（日周期）/ 35 118.5986（自由环状，−8.35 元，−0.02%）	—	三组差异均不超过 0.12%，结论不变；窗口口径下自由终端应略贵，说明终端约束并非靠”掏空”储能省钱，故主口径取 $S_T \geq S_0$（数据见 `intermediate/q2_term_*.json`）
场景数 K	60（主口径）、120	总费用 14 264 792.13/14 235 163.95 元（−0.21%）；紧急购电量 172 015.82/141 707.94 kWh（−17.62%）；有紧急购电天数 296/277	—	费用结论不变；分项指标（紧急购电量、触发天数）对场景分辨率敏感，引用时须带 K

表 32　光伏预报偏差的逐整点结构（样本为 2025 年每天 0:00 发布的 24 h 预报与附件 2 对应实际功率的逐槽配对，$n = 52\,560$；bias 单位 kW，“极值整点”给出该组内绝对偏差最大的整点及其取值）

整点分组	整点范围	组内 bias 区间	极值整点
夜间（光伏无有效出力）	0–4 时	0.00 至 +0.09	4 时 +0.09
上午（系统性**高估**）	5–10 时	+214.21 至 +733.00	7 时 +733.00
正午过渡	11 时	+7.21	—
下午（系统性**低估**）	12–18 时	−801.87 至 −106.46	15 时 −801.87
傍晚之后（预报空洞）	19–23 时	−0.15 至 0.00	19 时 −0.15

表 33　联合情景的保真度检验（对象：64 个代表历史日与 64 个分块 Bootstrap 情景；参照：2025-02-01 至 12-31 的 334 天实际数据；单位：相对偏差为%，相关系数为无量纲量；样本：64 情景 × 144 槽）

检验指标	代表历史日（主口径）	分块 Bootstrap（对照）	判读
电价均值相对偏差	0.08%	3.41%	主口径的均值几乎无偏
最大分位相对差（1%/5%/25%/50%/75%/95%/99%）	6.99%（1% 分位）	20.4%（1% 分位）	两者都在低价尾部误差最大，对照口径的尾部偏差约为主口径的 3 倍
情景 $\rho(p, \text{净负荷})$	0.2597（实际 0.2620）	无法按日配对	主口径 **保留**了联合结构
日净需求残差均值保持	−715.85 kWh（全样本 −712.23，相对差 −0.51%）	不适用（块级拼装，无法按日配对）	净需求侧的边缘分布同样保真

表 34　储能容量与充放功率上限的双向扫描（问题 1，典型日；费用单位：元）

容量（kWh）	最优购电费	相对主口径	充放功率上限（kW）	最优购电费	相对主口径
7 200	38 640.45	−10.00%	1 250	40 166.17	−14.35%
9 600	36 700.45	−4.48%	2 500	36 390.84	−3.60%
12 000	35 126.95	0.00%	5 000	35 126.95	0.00%
14 400	33 875.57	+3.56%	7 500	35 027.13	+0.28%
19 200	31 692.43	+9.78%	10 000	34 960.59	+0.47%
28 800	28 854.53	+17.86%	—	—	—
48 000	27 601.80	+21.42%	—	—	—

表注：”相对主口径”以主口径最优费用 35 126.95 元（容量 12 000 kWh、功率 5000 kW）为分母，正号表示更省；容量扫描同步移动储电量上下限以保持 10%–90% 放电深度。另做两个单自由度放开实验：仅解除储电量上限（充放功率维持 5000 kW）为 27 351.55 元，相对主口径多省 22.1%；仅解除充放功率上限（容量维持 12 000 kWh）为 34 544.43 元，相对主口径多省 1.7%。

表 35　场景集的三层构造与稳健性检验（残差样本：2025-02-01 至 2025-12-31，334 天 × 144 槽）

层次	保留的结构	本文做法	证据
第 1 层	残差水平分布（含尾部）	按日总残差 $s_d=\sum_t r_{d,t}$ 等频分层，主口径 $K=60$ 层	每层均有代表，尾部区间不被均值化抹平
第 2 层	残差的日内形状与时段相关	层内取 medoid（真实历史日向量），不给中心化或平滑	代表场景逐槽可复现，形状与真实日一致
第 3 层	层间概率权重	$W_\omega=n_\omega/334$，$\sum_\omega W_\omega=1$	场景集一次性存盘固定，问题 3/4 与消融共用同一份
稳健性	层数 K 的影响	$K=40/60/120$ 各求解一次全窗口	紧急购电量 238 484 / 172 015.82 / 141 707.94 kWh；总费用 1 437.17 / 1 426.48 / 1 423.52 万元，极差 13.65 万元（0.96%）

表 36　电价过程的四项统计检验（数据：附件 4 实时电价，2025-01-01 至 12-31，$n=52\,560$；电价与残差单位：元/kWh，自相关为无量纲量；偏差检验的参照剖面为附件 1 电价列）

编号	检验对象	检验方法与统计量	结果	对建模的含义
T1	确定性日内骨架是否存在	逐时段均值与逐时段标准差的对比（变异系数）	附件 1 剖面与附件 4 逐时段均值的最大偏差 5.15×10^{-5} 元/kWh（全年 365 天口径；仅结果窗口 334 天口径为 3.07×10^{-2}）；残差逐时段标准差均值 0.1238 元/kWh，变异系数 16.94%	骨架稳定存在但残差不可忽略，采用”骨架 + 残差”两层结构
T2	残差的正态性与日间差异	Jarque–Bera 检验、偏度与峰度；同一槽位跨日自相关	JB = 656.70（$p=2.51\times10^{-143}$）拒绝正态；偏度 −0.169、峰度 −0.431；日间自相关 ACF(1–3) = 0.513/0.148/0.144	残差非正态、日间相关性弱：不做正态参数化，改用非参数分层重采样；骨架不按月分段
T3	日内自相关与波动结构	残差自相关 ACF(1–6)；峰段与谷段残差标准差之比	ACF(1–6) = 0.859/0.666/0.614/0.573/0.494/0.43；峰段（19:00–20:00）残差标准差 0.1634 元/kWh 为谷段（03:00–04:00）0.0580 元/kWh 的 **2.81 倍**	情景须按时刻对齐分块，同时保留日内相关与按时段异方差
T4	与负载、光伏的相关性	Pearson 相关系数（池化与日聚合两个尺度）	池化：与负载 +0.508、与净负荷 +0.260、与光伏 +0.028；日聚合：与负载 +0.898、与光伏 −0.343	**决定必须生成电价与净需求的联合情景**，否则系统性低估”高价时段同时缺电”的尾部风险

表 37　风险前沿三个代表点的情景内指标与真实轨迹回测（对象：联合情景集上按 λ 求解的三组解；范围：2025-02-01 至 12-31 中每 6 天取 1 天的 56 个代表日；单位：费用元/日、电量 kWh/日；样本：64 个代表情景，$\alpha=0.95$）

代表点	情景内期望费用	情景内 95% CVaR	真实轨迹回测费用	紧急购电量
$\lambda=0$（期望最优）	41 995.59	52 176.36	45 694.27	480.19
$\lambda=0.3$（推荐）	42 168.43	50 814.83	45 778.03	324.60
$\lambda=1$（近似最坏）	42 411.56	50 510.46	45 952.85	313.83

表 38　验证链条 L1–L5 的实施方式、通过标准与本文实测结果（全部检验针对 2025-02-01 至 2025-12-31 共 334 天的结果窗口，问题 1 另含典型日）

级	名称	做法	通过标准	本文实测结果
L1	退化与恒等式单测	无储能退化由闭式给出并与模型基线对拍；充放比与 $1/(\eta_c\eta_d)$ 对拍；周期条件、充放互斥、非负性逐槽断言	相对误差 $\le10^{-6}$；越界数 = 0	充放比偏差 4.44×10^{-16}；周期偏差 0.000；充放同时槽数 0；负购电槽数 0；无储能基线闭式 48 052.05 元与模型基线一致
L2	交叉模型对拍	逐日两阶段求解（334 个子问题、固定初态）与全年单一 LP（192 384 个变量、96 192 条约束）在完美信息下对拍	差距可解释、方向合理	全年单一 LP 12 228 166.56 元，逐日完美信息 13 411 560.90 元，差 1 183 394.34 元（9.68%），差值来自日决策解耦与逐日终端条件，方向为保守
L3	真实数据独立审计	**不共用建模代码的费用函数**，只读原始附件与结果文件重算 10 项	见表 29	A1–A10 逐项结果见表 29：内部解全部通过，交付物层 A2 需按写入精度说明
L4	基线与消融	无储能基线、典型日预报、完美信息、安全边际、场景数、终端条件、日决策解耦共 10 组对照	每个保留模块的移除都伴随可分辨的费用上升	见表 30：安全边际的边际价值 3 133 767.54 元，四次滚动的价值 332 389.18 元
L5	统计与稳健性	场景保真度、场景数稳健性（$K=60/120$）、单参数与 Sobol 全局灵敏度、非预期性置换检验	结论方向不变、关键量给区间	场景均值相对差 0.082%、关键分位最大相对差 6.99%、电价—净需求相关 0.2597 对 0.2620；K 由 60 加倍到 120 时费用仅变 −0.21%；结论方向均不变（单参数与 Sobol 结果见表 31）

表 39　模型局限的四要素表述（假设编号 → 可能偏离现实之处 → 偏离方向 → 验证所需数据 → 部署前校准动作）

假设编号	可能偏离现实之处	偏离方向	验证所需数据	部署前校准动作
A2	充放电效率取单边 0.9（往返 0.81），实际随功率、温度与老化变化	高估收益	电池厂商的效率—功率—温度曲线与运行日志	按效率灵敏度重标定：本文四种口径下典型日节省为 12 925.10–14 635.52 元（极差 13.23%），若往返效率低于 0.7，套利收益须整体下调
A4	忽略自放电与循环衰减	高估长期收益	循环寿命试验数据、衰减—循环次数曲线	在目标函数中加入循环成本 $c_{cyc}\times$ 放电量后重解；在此之前本文的节省率应视为上限
A1	不允许向外部电网售电	高估费用，结果保守	上网电价政策与结算规则、余电上网的合同条款	引入上网电价 θ_t 后重解并比较策略变化；此时盈余处置量 口径 C-06 的地位由” 必要” 变为” 可选”
R3	计划依据中把当日负载视为已知（题面未给负载预报）	低估紧急购电量与总费用	负载预测误差序列（相对 MAE 5%、10%）	按负载水平的全局灵敏度（Sobol 一阶指数 0.4711）重新设置安全边际，并对 5%、10% 两档负载误差重跑问题 2、3
A8	结果窗口首日 0:00 的储电量由 2025 年 1 月预热递推得到	影响初期费用，方向不定	实际运行日志中 2025-02-01 0:00 的储电量	以实测初值替换后重跑窗口前两周，比较初始瞬态长度与费用差
A6	终端条件只要求 $S_T \geq S_0$，未给终端储电量定价	高估末期费用，结果保守	跨日电价信息与结算规则	用终端水价值替代硬边界后重解，比较末两周的费用差（本文日周期与自由环状两口径仅差 8.35 元）
场景集	334 天窗口既用于估计情景分布又用于回放（样本内）	高估稳健性，区间偏窄	不少于 2 年的预报误差历史	按拟合窗 / 验证窗切分重估情景集并做样本外回放（第九章已声明该限制与代偿性证据）

表 40　附件数据规模、结构与本文用途（行数为数据行数，不含表头；单位：功率 kW、电价元/kWh）

附件	内容	数据规模	时间范围	本文用途
附件 1	电价、小区负载、光伏发电“预测”功率	144 行（时间列 + 3 个数据列）	一天（全年平均日）	问题 1 的输入与基线口径；经身份核查后确认为全年平均日，不作日前预报（§3.3）
附件 2	小区负载、光伏发电实际功率（两个工作表）	365 行 × 144 列（另含日期列）	2025-01-01 至 12-31	问题 2–4 的实际值，用于实时结算与回测
附件 3	光伏发电功率预报（整点值、每天 4 次发布）	1460 行 × 24 列（另含日期、发布时刻列）	2025-01-01 至 12-31	问题 2–4 的日前与滚动预报输入
附件 4	实时电价	365 行 × 144 列（另含日期列）	2025-01-01 至 12-31	问题 4 的波动电价输入；其逐时段均值同时构成假设 A10 的“分时骨架”

表 41　波动电价与固定电价下的结果对照（同一实际轨迹：2025-02-01 至 12-31 共 334 天；单位：费用元，电量 kWh；波动价基线 17 026 782.86 元，固定价基线 16 407 319.63 元，均为 口径 C-01 口径）

模型	总费用/元	相对基线节省率	紧急购电量/kWh	与固定价同结构的差
问题 2（固定价，两阶段）	14 264 792.13	13.06%	172 015.82	—
问题 4-2（波动价，两阶段）	15 094 255.84	11.35%	182 759.86	+829 463.70（+5.81%）
问题 3（固定价，四次滚动）	13 932 402.95	15.08%	64 522.09	—
问题 4-3（波动价，四次滚动）	14 722 153.17	13.54%	63 523.11	+789 750.22（+5.67%）